RAG-ו Ollama

ריבונות דיגיטלית בעידן הבינה המלאכותית

Digital Sovereignty in the Age of AI

Author: Dr. Yoram Segal

All rights reserved

הספר מבוסס על הסרטון מיוטיוב:

Ollama Course -- Build AI Apps Locally

https://www.youtube.com/watch?v=GWB9ApTPTv4

11/2025

תוכן העניינים

1 מבוא: העידן החדש של ידע וריבונות דיגיטלית 1

1.1 מבט היסטורי: אימפריות הענן והריכוזיות הדיגיטלית 1

1.2 הצורך בריבונות דיגיטלית: תלות כלכלית, טכנולוגית ואתית 1

1.3 הצגת Ollama: דמוקרטיזציה של הבינה המלאכותית 2

1.4 הצגת RAG: קרקוע עובדתי ופתרון למשבר ההלוצינציה 2

1.5 ריבונות קונטקסטואלית: משילוב Ollama ו-RAG 3

1.6 מבנה הספר: מסע מהתיאוריה לפרקטיקה 3

1.7 סיכום: אוטונומיה דיגיטלית כהצהרה על עתיד 4

2 האימפריות הדיגיטליות והמהפכה הנגדית של Ollama 1

2.1 השלטון הבלתי מעורער של הענן 1

2.2 Ollama כמהפכה נגדית 2

2.3 יתרונות תיאורטיים של Ollama 2

2.4 השוואה ארכיטקטורית: Ollama מול שירותי ענן 3

2.5 ריבונות חומרה מול ריבונות תוכנה 4

2.6 התקנה ראשונית והרצה בסיסית 5

2.7 סיכום: החירות החישובית כניצחון דמוקרטי 6

3 אנטומיה של מודל שפה מקומי: מושגי יסוד וניהול 1

3.1 פרמטרים: צפיפות המחשבה הדיגיטלית 1

3.2 אורך הקשר והטמעה: זיכרון ומשמעות 3

3.3 קוונטיזציה: הפשרה בין דיוק ליעילות 4

3.4 ניהול מודלים: הכלים הפרקטיים של Ollama CLI 5

3.5 טבלת מושגי היסוד: סיכום מושגי הפרק 9

4 עיצוב האישיות הדיגיטלית: הפרקטיקה של Ollama 1

4.1 הנדסת זהות קוגניטיבית: הModelfile- 1

4.1.1 מבנה הModelfile-: הוראות ופרמטרים 1

4.1.2 יצירת המודל המותאם: ollama create 2

4.1.3 דוגמה נוספת: מודל לסיכום מסמכים טכניים 3

4.2 קישוריות לאפליקציות: הREST API- של Ollama 3

4.2.1 נקודות קצה עיקריות: /api/generate ו/api/chat/- 4

4.2.2 דוגמה עם CURL: שאילתה פשוטה 4

4.2.3 דוגמה עם /api/chat: שיחה רב-תורית 4

4.2.4 שליטה על פרמטרים: temperature ו-num$_predict$ 5

4.3 הפשטה אלגנטית: ספריית Python SDK 5

4.3.1 דוגמה: שאילתה פשוטה עם ollama.generate() 5

4.3.2 שיחה רב-תורית עם ollama.chat() 6

4.3.3 ניהול מודלים: ollama.list() 6

4.4 פרויקט מעשי: מארגן רשימת מכולת אוטומטי 7

4.4.1 הבעיה: כאוס הקניות 7
4.4.2 התכנון: שלושה שלבים 7
4.4.3 הקוד המלא: grocery$_organizer.py$ 7
4.4.4 קובץ הקלט: grocery$_raw.txt$ 8
4.4.5 הפלט המצופה: grocery$_organized.txt$ 9
4.4.6 הרחבות אפשריות 9
4.4.7 ניתוח התוצאות: אוטונומיה מלאה 9
4.5 סיכום הפרק: מכלי למומחה 01

5 משבר ההזיה והצורך בזיכרון קונטקסטואלי 1
5.1 הנבואה השבורה 1
5.2 האמת הקבורה בנתונים 1
5.3 הפתרון הארכיטקטוני 2
5.4 מידע מת, זיכרון חי 3
5.5 ששת השלבים של זיכרון ויצירה 3
5.6 כשלון וישועה: השוואת LLM לעומת RAG 5
5.7 הקונטקסט כזהות 6
5.8 הדרך אל האמינות 7

6 בניית ספריית הזיכרון: האינדקס התיאורטי והמעשי 1
6.1 מסע האינדקס: ארבעת השלבים המכריעים 1
6.2 פיצול הטקסט: אמנות החלוקה והשימור 2
6.3 השפה המהותית: המרת טקסט לוקטורים 4
6.4 מאגר הזיכרון הוקטורי: Chroma DB כבית לידע 7
6.5 טבלת רכיבים מקומיים: האקוסיסטם המלא 9
6.6 סיכום: הזיכרון המקומי כזכות יסוד 01

7 ארכיטקטורת האחזור והיצירה: RAG בפועל 1
7.1 תהליך האחזור: מהשאילתה אל הקונטקסט 1
7.2 אופטימיזציה: Multi-Query Retriever 3
7.3 הגנרציה הקונטקסטואלית: עיצוב RAG Prompt 4
7.4 פרויקט מעשי מלא: מערכת RAG מקומית עם Streamlit 7
7.5 דוגמת שימוש: ``מהם הקנסות על אי דיווח?" 11

8 מערכות סוכנים מתקדמות: מבינה יחידנית לאוטומציה קוגניטיבית 1
8.1 מאינטראקציה לאוטומציה: המהפכה השקטה 1
8.2 ארכיטקטורת סוכנים: מהמונוליט לקהילה 1
8.2.1 המושג: מהו סוכן? 1
8.2.2 מסגרות עבודה: Swarm והתזמור הקוגניטיבי 3
8.3 מקרה מבחן מתקדם: סוכנות גיוס מלאכותית 4
8.3.1 תיאור הבעיה: מהמורכבות לאוטומציה 4
8.3.2 ארכיטקטורת הסוכנים: חמישה מומחים וקצב אחד 4
8.3.3 דוגמת ריצה: מקורות חיים לדוח המלצות 7
8.3.4 ניתוח התוצאות: מה למדנו על סוכנים? 21
8.4 אוטונומיה ארגונית: מהגיוס לשירות לקוחות 31
8.4.1 עלות תפעול אפסית: המשמעות הכלכלית 31
8.4.2 פרטיות מלאה: הרקע האתי 41
8.4.3 יישומים נוספים: מהגיוס לשירות לקוחות 41
8.4.4 עתיד האוטומציה המקומית: מהחזון למציאות 41

8.5 סיכום: מהיחיד לקהילה 51

פרק 1

מבוא: העידן החדש של ידע וריבונות דיגיטלית

RAG-ו Ollama

ריבונות דיגיטלית בעידן הבינה המלאכותית

Author: Dr. Yoram Segal

All rights reserved

11/2025

תוכן העניינים

פרק 1

מבוא: העידן החדש של ידע וריבונות דיגיטלית

1.1 מבט היסטורי: אימפריות הענן והריכוזיות הדיגיטלית

במבט היסטורי, העשור השני של המאה ה-12 ייזכר כתקופה של ריכוזיות כוח קיצונית בעולם המחשוב. זוהי תקופה שבה כוח העיבוד של מודלי השפה הגדולים -- Large Language Models, או בראשי תיבות LLMs -- לא רק נולד, אלא גם התבצר כולו בשרתי ענק תחת שליטתן הבלעדית של קומץ "אימפריות ענן" גלובליות. זוהי דיקטטורה דיגיטלית שקטה, שבה הנתונים, השיחות ואפילו האינטליגנציה המלאכותית שאנו בונים -- כולם עוזבים את גבולות הבית והמשרד, עוברים טרנספורמציה דיגיטלית ומועברים אל מחוזות זרים שאנו לא יכולים לשלוט בהם.

ההיסטוריה האנושית מלמדת אותנו שוב ושוב כי ריכוזיות של כוח, בין אם מדובר בכוח כלכלי ובין אם מדובר בכוח טכנולוגי, מובילה בהכרח לשחיקת האוטונומיה האישית והארגונית. בעולם העתיק, אימפריות ריכוזיות שלטו על ידי שליטה בדרכי המסחר ובמקורות המים. בעידן המודרני, הריכוזיות עברה למימד אחר -- למימד הדיגיטלי. כיום, השליטה אינה על דרכים פיזיות אלא על תשתיות מידע ועיבוד. מי ששולט במודלי השפה הגדולים, שולט בפועל ביכולת לייצר ידע, להבין שפה ולקבל החלטות מבוססות נתונים.

בעולם זה של ריכוזיות דיגיטלית, אנו מוצאים את עצמנו תלויים בשירותים כמו ChatGPT של OpenAI, Claude של Anthropic, ו-Gemini של Google. כל אלו הם שירותי ענן מבריקים ועוצמתיים, אך הם גם מהווים נקודת תשלום וסיכון תמידי לדליפת מידע. כל אינטראקציה, כל פיסת נתון וכל קריאת API הופכת למקום שבו אנו חשופים -- הן מבחינה כלכלית והן מבחינה ביטחונית.

אולם, כנגד זרם הענק הזה, מגיח כלי פשוט ומהפכני המכונה Ollama. ככלי קוד פתוח, Ollama מבקש להחזיר את הכוח החישובי לידי המשתמש היחיד, על ידי מתן האפשרות להריץ מודלים מתקדמים של LLMs מקומית -- על המחשב שלך, בבית או במשרד -- ללא עלות שוטפת, ובאופן פרטי לחלוטין. זהו ניצחון של החירות החישובית. זוהי הצהרה על ריבונות דיגיטלית.

1.2 הצורך בריבונות דיגיטלית: תלות כלכלית, טכנולוגית ואתית

מדוע צריכים אנו לדבר על "ריבונות דיגיטלית"? המונח עשוי להישמע גרנדיוזי, אך הוא מדויק. ריבונות, במובן הקלאסי שלה, היא היכולת של ישות -- בין אם מדינה ובין אם פרט -- לקבוע את גורלה ללא תלות חיצונית. כאשר אנו מדברים על ריבונות דיגיטלית, אנו מדברים על היכולת לשלוט במידע שלנו, בכוח החישוב שלנו ובאינטליגנציה המלאכותית שאנו משתמשים בה -- ללא תלות בגופים חיצוניים.

התלות בשירותי ענן מציבה בפנינו שלוש בעיות קיומיות. הראשונה היא **עלות כלכלית**. המודל העסקי של אימפריות הענן הוא תשלום לפי שימוש -- Pay-per-Use. כל קריאה למודל, כל שאילתה שאנו שולחים, מחייבת אותנו בתשלום. עבור ארגונים גדולים או מפתחים שמריצים אפליקציות אינטנסיביות, העלויות יכולות להגיע לסכומים אסטרונומיים. זוהי תלות כלכלית שאינה בת-קיימא

לטווח ארוך.

הבעיה השנייה היא **פרטיות הנתונים**. כאשר אנו שולחים שאילתה למודל שפה מבוסס ענן, הנתונים שלנו עוזבים את המחשב שלנו ונוסעים דרך רשתות ושרתים שאנו לא יודעים מה יעלה בגורל המידע בהם. עבור יישומים רגישים במיוחד -- כגון תוכנות המטפלות במידע פיננסי, רפואי או משפטי -- הסביבה המבוזרת והפרטית היא הכרחית. אי אפשר להסתמך על הבטחות של ספקי השירות בלבד. הפתרון האמיתי הוא שהמידע פשוט לא יעזוב את הסביבה המקומית שלנו.

הבעיה השלישית היא **שליטה טכנולוגית**. כאשר אנו תלויים בשירות ענן, אנו למעשה תלויים בהחלטות של חברה פרטית. אם החברה מחליטה להעלות מחירים, לשנות תנאי שימוש, או אף לסגור את השירות -- אנו חסרי אונים. הריבונות הדיגיטלית מאפשרת לנו להחזיק את השליטה בידיים שלנו. אנו בוחרים איזה מודל להריץ, באיזה אופן להשתמש בו, וכיצד להתאים אותו לצרכים שלנו.

זהו ההקשר הרחב שבתוכו מופיע Ollama. זהו כלי שנועד להחזיר לנו את הריבונות הדיגיטלית, לאפשר לנו לנהל את המודלים שלנו באופן עצמאי, ולהבטיח שהנתונים שלנו נשארים בידיים שלנו.

1.3 הצגת Ollama: דמוקרטיזציה של הבינה המלאכותית

Ollama הוא פרויקט קוד פתוח שמטרתו לפשט באופן דרמטי את תהליך הרצת מודלי שפה גדולים על גבי החומרה האישית של המשתמש. הכלי נוצר תוך הבנה עמוקה של המכשולים שעומדים בפני משתמשים רגילים כאשר הם מנסים להריץ מודלים מקומיים: הורדת משקלי מודל כבדים, הגדרת סביבות פיתוח מורכבות, התמודדות עם בעיות תאימות חומרה, וניהול זיכרון ומעבדים.

Ollama פותר את כל אלו בבת אחת. הוא מספק ממשק אחוד ופשוט -- ממשק שורת פקודה (CLI) וממשק תכנותי (API) -- שמאפשר להוריד מודלים בקלות, להריץ אותם באופן מיידי, ולנהל אותם בצורה יעילה. הפשטות הזו היא לא רק נוחות טכנית, אלא מהפכה אמיתית בנגישות: Ollama מבצע דמוקרטיזציה של הבינה המלאכותית. הוא מאפשר לא רק למומחי AI ולחוקרים, אלא גם למפתחים רגילים, לחובבים ולסטודנטים -- לכל מי שיש לו מחשב עם כוח חישוב סביר -- להריץ מודלי שפה מתקדמים.

דמוקרטיזציה זו משמעותית ביותר כאשר אנו חושבים על העתיד של הבינה המלאכותית. כאשר הכוח החישובי מרוכז בידי מספר חברות גדולות, החדשנות מצטמצמת. רק אותן חברות יכולות להחליט מה אפשר ומה אסור, מה ייפתח ומה יישאר סגור. אך כאשר הכוח עובר לידי הקהילה הרחבה, החדשנות פורצת את כל הגבולות. מפתחים יכולים לנסות רעיונות חדשים, לבנות אפליקציות ייחודיות, ולהתאים את המודלים בדיוק לצרכים שלהם -- ללא צורך בהיתר, ללא צורך בתשלום, וללא צורך לחשוף את הנתונים שלהם.

Ollama אינו רק כלי טכני; הוא גם הצהרה פוליטית-טכנולוגית. הוא אומר: הבינה המלאכותית אינה צריכה להיות קניינם הבלעדי של אימפריות הענן. היא יכולה להיות נחלת הכלל. היא יכולה לרוץ על המחשב שלך, בבית שלך, תחת השליטה שלך.

1.4 הצגת RAG: קרקוע עובדתי ופתרון למשבר ההלוצינציה

אך החירות החישובית לבדה אינה מספיקה. מודלי שפה גדולים, על אף עוצמתם המדהימה, סובלים ממחלה פנימית קשה -- נטייתם להזות. כאשר אנו שואלים מודל שפה שאלה, הוא מייצר תשובה על סמך דפוסים סטטיסטיים שלמד במהלך האימון שלו. אך המודל אינו "יודע" מידע במובן האנושי. הוא אינו מבין אמת או שקר. הוא פשוט מנבא את המילה הבאה הכי סבירה. כתוצאה מכך, כאשר המודל נתקל בשאלה שהוא אינו יודע את התשובה לה, הוא נוטה למלא את הפערים ב"שקרים נעימים" -- תגובות שנשמעות משכנעות וסמכותיות, אך אינן נכונות בהכרח.

תופעה זו ידועה כ-Hallucination -- הלוצינציה. זהו המשבר הגדול ביותר של מודלי השפה הגדולים. הם יכולים לספר לך על ספר שלא קיים, על מאמר מדעי שלא נכתב, על תקנה משפטית שלא נחקקה. והם יעשו זאת בביטחון מלא, בשפה חלקה וברורה, כך שקשה מאוד להבחין בשגיאה.

© Dr. Yoram Segal - כל הזכויות שמורות

זהו המקום שבו RAG -- ראשי תיבות של Retrieval Augmented Generation, או בתרגום חופשי "יצירה מתוגברת באחזור" -- נכנס לתמונה. RAG הוא מנגנון טכנולוגי שמשלב את יכולות הגנרציה המעולות של מודל השפה עם מערכת חיצונית ואמינה לאחזור ידע. הרעיון הוא פשוט אך מהפכני: במקום לתת למודל השפה לענות על שאלה מתוך "הדמיון" שלו, אנו קודם כל מחפשים במאגר מסמכים פרטי את המידע הרלוונטי, ורק אז מבקשים מהמודל לייצר תשובה -- אך תשובה שמבוססת אך ורק על המידע שאחזרנו.

בצורה זו, RAG מספק "קרקוע עובדתי" -- Factual Grounding. התשובה של המודל מקורקעת בראיות ממשיות שאוחזרו ממסמכי מקור אמיתיים. אם המסמך אומר שהקנס על אי דיווח הוא 5,000 שקלים, המודל ישיב בהתאם. אם המסמך אינו מכיל את המידע, המודל יכול להשיב בכנות: "אין לי מידע בנושא זה". זוהי הפחתה דרמטית של הלוצינציות ושיפור משמעותי באמינות.

RAG הופך את ה-AI ממכונה שמנבאת מילים למכונה שמבינה הקשר. הוא הופך את המודל מיוצר תוכן גנרי ליועץ מקצועי המבוסס על נתונים ספציפיים. והכי חשוב -- כאשר משלבים את RAG עם Ollama, אנו משיגים את שני העולמות: חירות חישובית ואמינות קונטקסטואלית. כל המערכת רצה מקומית, הנתונים לא עוזבים את המחשב שלנו, והתשובות מבוססות על המסמכים שלנו.

1.5 ריבונות קונטקסטואלית: משילוב Ollama ו-RAG

הנקודה המרכזית של ספר זה היא ההבנה כי Ollama ו-RAG אינם רק שני כלים נפרדים, אלא שני עמודי תווך של ארכיטקטורה דיגיטלית חדשה -- ארכיטקטורה שאנו מכנים "ריבונות קונטקסטואלית". מהי ריבונות קונטקסטואלית? זוהי היכולת ליצור ידע אמין, מדויק ורלוונטי מתוך הנתונים הפרטיים שלנו, כל זאת תחת קורת גג אחת ותחת שליטה מלאה.

כאשר אנו משלבים את Ollama -- שמספק את החירות החישובית -- עם RAG -- שמספק את הקרקוע העובדתי -- אנו בונים מערכת שהיא גם פרטית וגם אמינה. זוהי מערכת שיכולה לקרוא את המסמכים הפנימיים של הארגון שלנו, להבין את ההקשר העסקי שלנו, ולהשיב על שאלות באופן מדויק -- ללא שהמידע יעזוב את המשרד שלנו, וללא שנשלם על כל שאילתה.

ריבונות קונטקסטואלית משמעה גם שאנו יכולים לבנות מומחים דיגיטליים ייחודיים. לדוגמה, ארגון רפואי יכול לבנות מערכת RAG מקומית שמבוססת על המאמרים המדעיים העדכניים ביותר, על נהלי הטיפול הפנימיים של הארגון, ועל תיאורי מקרים היסטוריים. המערכת הזו תהיה מומחה פרטי שיודע בדיוק את מה שהארגון צריך שהוא ידע -- לא פחות ולא יותר. וזה הכול רץ מקומית, ללא צורך בחיבור לאינטרנט, ללא צורך בשיתוף נתונים עם גופים חיצוניים.

זוהי המשמעות האמיתית של דמוקרטיזציה של ידע: לא רק הנגישות לכוח חישובי, אלא גם היכולת לייצר ידע מותאם אישית, ידע שהוא רלוונטי בדיוק לצרכים שלנו, ידע שאנו שולטים בו במלואו.

1.6 מבנה הספר: מסע מהתיאוריה לפרקטיקה

ספר זה בנוי כמסע של שני שלבים מקבילים: שלב הראשון עוסק ב-Ollama והריבונות החישובית, ושלב השני עוסק ב-RAG והאמינות הקונטקסטואלית. כל שלב מתחיל בתיאוריה פילוסופית-היסטורית ועובר לפרקטיקה טכנית מפורטת.

חלק ראשון: Ollama -- אוטונומיה דיגיטלית וריבונות חישובית

בחלק זה נבחן לעומק את Ollama כפתרון לריכוזיות הדיגיטלית. נתחיל בניתוח ביקורתי של "אימפריות הענן" -- שירותי ה-LLM המבוססים על ענן כמו ChatGPT, Claude ו-Gemini -- ונבין את התלות הכלכלית, הטכנולוגית והאתית שהם יוצרים. לאחר מכן נציג את Ollama כאלטרנטיבה מהפכנית: כלי קוד פתוח שמאפשר להריץ מודלים מתקדמים מקומית, ללא עלות שוטפת וללא פגיעה בפרטיות.

נעמיק בהבנה הטכנית של מודלי השפה: מהם פרמטרים, מהו אורך הקשר (Context Length), מהי הטמעה (Embedding), ומהי קוונטיזציה (Quantization). נלמד כיצד לנהל מודלים באמצעות

ממשק שורת הפקודה (CLI), כיצד ליצור מודלים מותאמים אישית באמצעות Modelfile, וכיצד לשלב את Ollama באפליקציות באמצעות REST API ו-Python SDK.

בסוף החלק נציג פרויקט מעשי שלם: יישום שמארגן רשימת מכולת באמצעות Ollama, כדוגמה לאופן שבו ניתן לבנות אוטומציה מקומית שלמה.

חלק שני: RAG -- זיכרון קונטקסטואלי וקרקוע עובדתי

בחלק השני נעבור למשבר האמינות של מודלי השפה. נבחן לעומק את תופעת ההלוצינציה -- מדוע מודלים נוטים להמציא מידע, ומה ההשלכות של תופעה זו בעולם האמיתי. נציג את RAG כפתרון מהפכני: מנגנון שמשלב יכולות אחזור ויצירה כדי לקרקע את התשובות בראיות ממשיות.

נלמד את הארכיטקטורה המלאה של RAG: כיצד לבצע חלוקת מסמכים (Chunking), כיצד להטמיע טקסט לוקטורים (Embeddings), כיצד לאחסן את הוקטורים במאגר וקטורי (Vector Store) כמו Chroma DB, וכיצד לבצע חיפוש דמיון (Similarity Search) כדי לאחזר את הקונטקסט הרלוונטי.

נעמיק בטכניקות אופטימיזציה כמו Multi-Query Retriever, נלמד כיצד לעצב Prompt נכון למערכת RAG, וכיצד לבנות שרשרת עיבוד שלמה באמצעות LangChain Expression Language (LCEL).

בסוף החלק נציג פרויקט מלא: מערכת RAG מקומית עם ממשק Streamlit, שמאפשרת להעלות מסמך, לאנדקס אותו, ולשאול שאלות -- הכול מקומית ופרטית.

לסיום: מערכות סוכנים מתקדמות

הפרק האחרון יעסוק במערכות סוכנים (Agent Systems) -- הדור הבא של אוטומציה קוגניטיבית. נראה כיצד ניתן לבנות רשת של סוכנים מומחים, כל אחד עם תפקיד ספציפי, שעובדים יחד בתיאום מלא. נציג מקרה מבחן מפורט: סוכנות גיוס מלאכותית שמנתחת קורות חיים, מסננת מועמדים, מבצעת התאמה למשרות ומספקת המלצות -- הכול באמצעות Ollama מקומי, ללא עלות תפעולית ובפרטיות מלאה.

1.7 סיכום: אוטונומיה דיגיטלית כהצהרה על עתיד

בסופו של דבר, ספר זה אינו רק ספר טכני. זהו ספר על חזון. זהו ספר על האמונה כי הבינה המלאכותית צריכה להיות נחלת הכלל, לא קניינם של אימפריות טכנולוגיות. זהו ספר על ההבנה כי אמינות ופרטיות אינן מושגים מופשטים, אלא דרישות קריטיות בעולם שבו AI משתלב בכל תחום של החיים שלנו.

Ollama ו-RAG מציעים לנו דרך חדשה: דרך של אוטונומיה, דרך של אמינות, דרך של שליטה. הם מאפשרים לנו לקחת אחריות על המחשוב שלנו, להפוך את ה-AI ממנביא שקרי התלוי בענן למומחה מקומי, אמין ופרטי -- מומחה שיודע בדיוק את מה שאנו צריכים שהוא ידע, ושומר על המידע שלנו בידיים שלנו.

זוהי המשמעות האמיתית של ריבונות דיגיטלית במאה ה-12: לא רק היכולת לעבד נתונים, אלא לעבדם באופן חופשי, בטוח ומדויק. זהו המסע שאליו אנו מזמינים אתכם בספר זה.

© Dr. Yoram Segal - כל הזכויות שמורות

פרק 2

האימפריות הדיגיטליות והמהפכה הנגדית של Ollama

Ollama ו-RAG

ריבונות דיגיטלית בעידן הבינה המלאכותית

Author: Dr. Yoram Segal

All rights reserved

11/2025

תוכן העניינים

פרק 2

האימפריות הדיגיטליות והמהפכה הנגדית של Ollama

2.1 השלטון הבלתי מעורער של הענן

במבט היסטורי, העשור השני של המאה העשרים ואחת יזכר כתקופה של ריכוזיות כוח קיצונית בעולם המחשוב. לא במקרה התפתח מצב זה: המעבר מחישוב אישי לחישוב ענן היה הכרחי מבחינה טכנולוגית, שכן האימון והפעלה של מודלי שפה גדולים דרשו משאבי חישוב עצומים שעלו על יכולתם של מחשבים אישיים. אך מה שהתחיל כהכרח טכני הפך במהרה למונופול כלכלי ואסטרטגי.

כוח העיבוד של מודלי השפה הגדולים לא רק נולד אלא גם התבצר כולו בשרתי ענק, תחת שליטתן הבלעדית של קומץ "אימפריות ענן" גלובליות. ChatGPT של OpenAI, Claude של Anthropic, Gemini של Google---כל אלה אינם רק כלים טכנולוגיים, אלא שערים דרכם חייבים לעבור מיליארדי משתמשים כדי לגשת ליכולות הבינה המלאכותית המתקדמת. זוהי דיקטטורה דיגיטלית שקטה, שבה הנתונים, השיחות ואפילו האינטליגנציה המלאכותית שאנו בונים, כולם עוזבים את גבולות הבית והמשרד, עוברים טרנספורט אל מחוזות זרים שאנו לא יכולים לשלוט בהם.

המודל העסקי הדומיננטי שנוצר הוא Pay-per-use---תשלום לפי שימוש. כל קריאת API, כל שאילתה, כל אינטראקציה הופכת לנקודת תשלום. עבור משתמש בודד, העלות עשויה להיראות זניחה. אך עבור ארגונים שמעוניינים לשלב יכולות בינה מלאכותית בתהליכי ליבה שלהם---שירות לקוחות אוטומטי, ניתוח מסמכים, סיכום דיווחים---העלות הופכת לחסם משמעותי. חישוב פשוט מגלה כי ארגון שמבצע מאות אלפי קריאות חודשיות עלול למצוא עצמו משלם עשרות אלפי דולרים בשנה, רק כדי לגשת לכלי שהוא עצמו יכול היה להפעיל על חומרה משלו.

מעבר לממד הכלכלי, התלות בשירותי ענן מציבה בפנינו שאלות אתיות ובטיחותיות חריפות. כאשר ארגון שולח נתונים רגישים---מסמכי מטופלים, חוזים משפטיים, נתונים פיננסיים פנימיים---אל שרתי API חיצוניים, הוא מאבד שליטה מלאה על המידע. נכון שרוב החברות מבטיחות הצפנה ופרטיות, אך ההבטחות הללו נשענות על אמון, לא על וודאות טכנולוגית. כל אינטראקציה הופכת לנקודת דליפת מידע פוטנציאלית. ההיסטוריה מלמדת כי ריכוזיות של כוח, בין אם כלכלי או טכנולוגי, מובילה בהכרח לשחיקת האוטונומיה האישית והארגונית.

ישנה גם בעיה נוספת: התלות הטכנולוגית. כאשר ארגון בונה את כל תשתית הבינה המלאכותית שלו סביב API ספציפי של ספק ענן מסוים, הוא הופך לכלוא בתוך האקוסיסטם הטכנולוגי של אותו ספק. שינוי ספק, התאמת מודלים, או מעבר לפתרון עצמאי---כל אלה הופכים למשימות מורכבות ויקרות. הארגון מאבד גמישות אסטרטגית ונכנס למצב של תלות מתמשכת.

2.2 Ollama כמהפכה נגדית

כנגד זרם הענן הזה, מגיח כלי פשוט ומהפכני המכונה Ollama. זהו פרויקט קוד פתוח שנועד לספק אלטרנטיבה יסודית לשיטה הריכוזית. תפקידו של Ollama הוא לפשט באופן דרמטי את תהליך הרצת מודלי שפה גדולים מקומית, על גבי החומרה האישית של המשתמש. על ידי כך, Ollama מבצע דמוקרטיזציה של בינה מלאכותית מתקדמת.

מהו, אם כן, Ollama במהותו? זהו כלי קוד פתוח שמספק ממשק אחיד ופשוט להורדה, ניהול והפעלה של מודלי שפה מתקדמים. הכלי מספק למשתמשים---בין אם הם מפתחים, חוקרים או חובבים---את היכולת להריץ מודלים כמו Llama 3.2, Gemma, Mistral ועוד, ללא צורך להתמודד עם המורכבות הטכנית של הגדרת סביבות Docker, ניהול תלויות ספריות, או תצורת מנועי GPU.

Ollama לוקח את כל המורכבות הזו ומסתיר אותה מאחורי ממשק פשוט ואלגנטי. ההתקנה היא פשוטה: הורדת קובץ התקנה יחיד, הפעלתו, והכלי מוכן לשימוש. ההפעלה של מודל היא פשוטה לא פחות: פקודה אחת בשורת הפקודה, והמודל מתחיל להירד, להיטען, ולהיות מוכן לשימוש אינטראקטיבי. זוהי הפשטות שמאפשרת לכל אדם שיש לו מחשב עם מעבד סביר וכרטיס גרפי בסיסי להפוך למפעיל עצמאי של בינה מלאכותית מתקדמת.

הפילוסופיה שמנחה את Ollama היא ברורה: החזרת הכוח החישובי לידי המשתמש היחיד. לא יותר תלות בשרתים מרוחקים, לא יותר תשלומים מתמשכים, לא יותר החשת נתונים רגישים לגופים חיצוניים. זוהי מהפכה נגדית---מעבר מאימפריות ריכוזיות לריבונות מבוזרת.

באופן טכני, Ollama מתפקד כשרת מקומי. כאשר מפעילים אותו, הוא רץ ברקע על גבי המכונה המקומית ומאזין לבקשות בפורט מקומי (בדרך כלל `43411:tsohlacol`). משתמשים יכולים לתקשר עם Ollama בשלוש דרכים: באמצעות שורת פקודה אינטראקטיבית, באמצעות REST API עבור אפליקציות, או באמצעות ספריות SDK פשוטות בשפות תכנות כמו Python. גמישות זו הופכת את Ollama לא רק לכלי לשימוש אישי, אלא למנוע חישוב שניתן לשלב בתוך אפליקציות ייצור אמיתיות.

2.3 יתרונות תיאורטיים של Ollama

המעבר להפעלת מודלים מקומית אינו רק שינוי טכני; זוהי הצהרה על ריבונות דיגיטלית, המניבה יתרונות מהותיים בכלכלה, באבטחה, בביצועים ובגמישות.

כלכלה חדשה---אפס עלויות שוטפות. המודל העסקי של אימפריות הענן, כפי שציינו, הוא תשלום לפי שימוש. Ollama מבטל לחלוטין את הדילמה הכלכלית הזו. ברגע שהמודל מותקן על המחשב המקומי, המשתמש יכול להימנע מעלויות קבועות הקשורות לקריאות API או לשימוש בשרת. לאחר ההשקעה הראשונית בחומרה---מעבד חזק או כרטיס גרפי מתאים---ההוצאות השוטפות הופכות לאפסיות.

ניקח דוגמה קונקרטית: ארגון המעוניין לנתח מדי חודש עשרות אלפי מסמכי לקוחות ולסכם אותם באופן אוטומטי. באמצעות API של שירות ענן, כל סיכום עשוי לעלות מספר סנטים. אם הארגון מעבד 100,000 מסמכים בחודש, העלות החודשית יכולה להגיע לאלפי דולרים. לעומת זאת, אם הארגון משקיע 3,000 דולר בשרת מקומי עם כרטיסי גרפי מתאימים ומפעיל את Ollama, הוא משלם פעם אחת ומפעיל את העיבוד ללא עלות נוספת. החישוב הכלכלי הופך למשכנע ביותר לאורך זמן.

ריבונות הנתונים---פרטיות מלאה. זוהי אבן הפינה האתית של Ollama. הרצת המודלים באופן מקומי מבטיחה שהנתונים שלנו לא צריכים להישלח לשרתים חיצוניים. כאשר משתמשים במודלים מבוססי ענן, כל שאילתה, גם אם היא מכילה מידע רגיש, עוברת דרך רשתות ושרתים שאנו לא יודעים מה יעלה בגורל המידע בהם. עבור יישומים רגישים במיוחד---כגון תוכנות המטפלות במידע פיננסי או רפואי---הסביבה המכונסת והפרטית שמספק Ollama היא הכרחית.

ניקח למשל מרפאה רפואית שמעוניינת להשתמש במודל שפה לסיכום אוטומטי של תיקי מטופלים. שליחת התיקים הללו, המכילים מידע רגיש ומוגן על פי חוק, אל שרתי API חיצוניים עלולה להיות

© Dr. Yoram Segal - כל הזכויות שמורות

בעייתית מבחינה משפטית ואתית. לעומת זאת, שימוש ב-Ollama מאפשר למרפאה לעבד את המידע במלואו על גבי שרת פנימי, מאחורי חומת אש, ללא כל חשיפה חיצונית. זוהי ריבונות נתונים אמיתית.

מהירות מיידית---הפחתת זמני השהיה. הרשתות הדיגיטליות, על אף מהירותן, סובלות תמיד מזמן השהיה. כאשר כל החישובים מתבצעים על החומרה המקומית של המשתמש, החיבור מבוטל, וזמני התגובה הופכים להיות כמעט מיידיים. זה מביא לביצועים מהירים יותר באופן משמעותי, במיוחד עבור אפליקציות אינטראקטיביות.

כאשר משתמש שולח שאילתה לשרת API מרוחק, הנתונים חייבים לעבור דרך רשת האינטרנט, להגיע לשרת, להתעבד, ולחזור. כל שלב מוסיף זמן השהיה. אפילו עם חיבור אינטרנט מהיר, זמן השהיה יכול להגיע למאות אלפיות השנייה (milliseconds). לעומת זאת, כאשר המודל רץ מקומית, זמן ההשהיה נעלם כמעט לחלוטין. התגובה מתחילה להתקבל תוך אלפיות שניות בודדות. עבור יישומים שדורשים תגובות מיידיות---כמו צ'אטבוטים אינטראקטיביים או כלי עזר לכתיבה בזמן אמת---השיפור בחוויית המשתמש הוא דרמטי.

גמישות חסרת תקדים והתאמה אישית. Ollama מספק למשתמש את ניהול המודלים, המאפשר להוריד בקלות ולעבור בין מודלי שפה גדולים שונים. יתרה מכך, הרצה מקומית מאפשרת גמישות רבה יותר בהתאמה אישית או אפילו כוונון עדין של המודלים כדי להתאימם לצרכים ספציפיים, ללא המגבלות שמטילים ספקי צד שלישי.

למשל, מפתח שמעוניין להשוות בין ביצועיהם של מודלים שונים למשימה ספציפית יכול להוריד מספר מודלים ולהריץ אותם במקביל. Ollama מאפשר החלפה מהירה בין מודלים באמצעות פקודה אחת. מפתח יכול להתנסות ב-Llama 3.2, לעבור ל-Gemma, ולנסות גם את Mistral---כל זאת בתוך דקות. באופן זה, מפתחים יכולים לבחור את המודל המתאים ביותר למשימה הספציפית שלהם.

יתרה מכך, Ollama תומך ביכולת ליצור מודלים מותאמים אישית באמצעות קבצי Modelfile. קובץ זה מאפשר למפתח להגדיר את הזהות הקוגניטיבית של המודל---להכתיב את התפקיד שלו, לקבוע את רמת היצירתיות, ולהוסיף הוראות קבועות מראש. למשל, ניתן ליצור מודל שתפקידו הוא לנתח רק סנטימנט של טקסטים, או מודל שמתמחה במתן תשובות תמציתיות בסגנון טכני. כל זה נעשה באופן מקומי, פרטי, וללא מגבלות חיצוניות.

2.4 השוואה ארכיטקטורית: Ollama מול שירותי ענן

כדי להבהיר את ההבדל המהותי שמביא Ollama לארכיטקטורה הדיגיטלית, נציג השוואה מפורטת בין גישת Ollama המקומית לבין שירותי LLM מבוססי ענן. ההשוואה מתמקדת בחמישה ממדים קריטיים: עלות השימוש, פרטיות נתונים, זמני השהיה, ניהול מודלים, ושליטה מלאה.

כפי שניתן לראות בטבלה 2.1, ההבדלים בין שתי הגישות הם מהותיים. מודל העלות של Ollama הוא חד-פעמי: השקעה בחומרה פעם אחת, ואחר כך שימוש ללא עלות נוספת. לעומת זאת, מודל הענן הוא תמידי ומצטבר---כל שימוש עולה כסף. מבחינת פרטיות, Ollama מציע ריבונות מלאה: הנתונים לא עוזבים את המכונה המקומית. בענן, הפרטיות נשענת על אמון בספק השירות.

זמני ההשהיה הם אחד היתרונות הבולטים של Ollama. ביצוע מקומי על כרטיס גרפי מהיר מביא לתגובות כמעט מיידיות. לעומת זאת, שירותי ענן תלויים ברשת ובמרחק הפיזי של השרת, מה שמוסיף השהיות משמעותיות. ניהול המודלים הוא גמיש לחלוטין ב-Ollama: ניתן להוריד, למחוק ולהחליף מודלים בקלות. בענן, המודלים מנוהלים על ידי הספק, והמשתמש אינו יכול להתאים או לשנות אותם.

התלות היא גורם נוסף: Ollama מאפשר עבודה מקומית, אפילו ללא חיבור לאינטרנט (לאחר הורדת המודלים). שירותי ענן דורשים תמיד חיבור פעיל, והם תלויים בזמינות של השרת החיצוני. אם השרת נופל או החיבור מופרע, העבודה נעצרת.

טבלה זו מדגישה כי Ollama אינו רק חלופה זולה יותר, אלא הצעה ארכיטקטורית שונה לחלוטין---מעבר מריכוזיות תלויה לריבונות עצמאית.

טבלה 2.1: השוואה: ארכיטקטורת Ollama מקומית מול שירותי LLM מבוססי ענן

Dimension	**Ollama (Local)**	**Cloud LLM (OpenAI, Anthropic)**
Cost Model	Free after initial hardware investment	Pay-per-use (per API call)
Data Privacy	Fully local; data never leaves the machine	Data sent to external servers; privacy relies on trust
Latency	Very low; immediate execution on CPU/GPU	Moderate to high; depends on network and server distance
Model Management	Full control: download, delete, switch models easily	Limited; models managed by cloud provider
Customization	Full flexibility: fine-tuning, Modelfiles, custom prompts	Limited; constraints imposed by provider
Dependency	Independent; no internet required after download	Fully dependent on internet and provider availability

2.5 ריבונות חומרה מול ריבונות תוכנה

ההופעה של Ollama מחייבת אותנו לבחון מחדש את מושג הריבונות בעידן הבינה המלאכותית. Ollama אינו משחרר אותנו לחלוטין מהאימפריות הטכנולוגיות, שכן מודלי הליבה החזקים---כגון Llama של Meta או Gemma של Google---עדיין מפותחים על ידי גופים אלו. עם זאת, Ollama משחרר אותנו מהצורך לשלם ולהיות תלויים בהם לצורך השימוש השוטף במודל.

משמעות הדבר היא שהתחרות בעולם הבינה המלאכותית עוברת מבעלות על תשתית החישוב---הענן---לבעלות על איכות המודל הפתוח. Ollama יוצר תלות הדדית בריאה: מודלי קוד פתוח חזקים (התוכנה) מקבלים גישה חופשית לקהל רחב, ומשתמשים מקבלים פלטפורמה קלה להפעלה (החומרה). התוצאה היא דמוקרטיזציה של הבינה המלאכותית המתקדמת, שהופך להיות פרקטי, בטוח ומותאם אישית עבור מגוון רחב של יישומים.

אולם, חשוב להבין כי הריבונות המלאה מתקבלת רק כאשר משתמשים במודלים שהם באמת קוד פתוח. מודלים כמו Llama 3.2 או Gemma מפורסמים בקוד פתוח, אך הם עדיין מפותחים על ידי חברות גדולות. הקוד זמין, המשקלים זמינים, אך ההשקעה באימון המודלים הללו היא עצומה ונשארת בידי הגופים הגדולים. לכן, הריבונות שמציע Ollama היא ריבונות על התשתית והשימוש, לא בהכרח על הפיתוח הבסיסי של המודלים עצמם.

עם זאת, הריבונות הזו היא משמעותית ביותר. היא מאפשרת למשתמשים לבחור מתי, איך ובאיזו תדירות להשתמש במודלים, ללא תלות בזמינות שרתים חיצוניים או במדיניות תמחור משתנה. היא מאפשרת למפתחים לבנות אפליקציות שאינן תלויות בקיומה של חברה מסוימת או בהמשך תמיכתה בשירות מסוים. זוהי ריבונות חישובית אמיתית.

בעתיד, ככל שקהילת הקוד הפתוח תמשיך להתפתח, סביר שנראה יותר ויותר מודלים שמפותחים באופן קהילתי לחלוטין, ללא תלות בחברות גדולות. פרויקטים כמו BLOOM או Falcon מראים כי זה אפשרי. Ollama מספק את התשתית שתאפשר למודלים הללו להגיע בקלות לידי המשתמשים הסופיים, ובכך להשלים את המהפכה הדמוקרטית.

כל הזכויות שמורות - Dr. Yoram Segal ©

2.6 התקנה ראשונית והרצה בסיסית

כדי להבין את הפשטות שמביא Ollama, נדגים את תהליך ההתקנה וההרצה הבסיסית. התהליך הוא פשוט להפליא ומתאים לכל רמות המשתמשים.

התקנת amallO. ההתקנה היא ישירה וקלה:

```
# On macOS or Linux:
curl -fsSL https://ollama.com/install.sh | sh

# On Windows:
# Download the installer from https://ollama.com/download
# and run the installation wizard
```

Listing 2.1: תונוש תוכרעמ לע Ollama תנקתה

לאחר ההתקנה, Ollama רץ ברקע כשירות מערכת. ניתן לוודא שהוא פעיל על ידי פתיחת דפדפן והיכנסות לכתובת `43411:tsohlacol//:ptth`. אם השרת פעיל, תופיע הודעת אישור.

הרצת מודל ראשון. כעת נוכל להריץ את המודל הראשון שלנו. ניקח למשל את Llama 3.2, מודל קל משקל ועוצמתי:

```
ollama run llama3.2:1b
```

Listing 2.2: Ollama סע ושאר לדומ תצרה

פקודה זו מבצעת שלוש פעולות באופן אוטומטי: תחילה, אם המודל עדיין לא הורד, Ollama מוריד אותו מהשרת המרכזי. שנית, הוא טוען את המודל לזיכרון. שלישית, הוא פותח מעטפת אינטראקטיבית שבה ניתן להתחיל לשוחח עם המודל. כל זה קורה באופן חלק ופשוט.

ברגע שהמעטפת האינטראקטיבית נפתחת, המשתמש יכול להתחיל לשאול שאלות:

```
>>> What is the capital of France?
The capital of France is Paris.

>>> Explain the concept of machine learning in one sentence.
Machine learning is a type of artificial intelligence
that enables computers to learn from data and improve
their performance over time without being explicitly programmed.

>>> /bye
```

Listing 2.3: Ollama סע תיביטקארטניא החישל המגוד

כפי שניתן לראות, התגובות מגיעות במהירות ובדיוק. המודל רץ באופן מקומי, והשיחה מתנהלת ללא כל תלות ברשת חיצונית.

ניהול מודלים בסיסי. Ollama מספק פקודות פשוטות לניהול המודלים:

```
# List all downloaded models
ollama list

# Show detailed information about a model
ollama show llama3.2:1b --info

# Remove a model to free up space
ollama rm llama3.2:1b
```

Listing 2.4: Ollama-ב סילדומ לוהינ תודוקפ

© Dr. Yoram Segal - כל הזכויות שמורות

הפקודה `tsil amallo` מציגה רשימה של כל המודלים המותקנים, יחד עם גודלם ותאריך ההתקנה. הפקודה `wohs amallo` מספקת פרטים טכניים מפורטים על המודל: מספר הפרמטרים, אורך ההקשר, אורך ההטמעה, ועוד. והפקודה `mr amallo` מאפשרת למחוק מודלים שאינם בשימוש כדי לפנות שטח אחסון.

הפשטות הזו היא המפתח להצלחתו של Ollama. משתמשים אינם צריכים להיות מומחי מערכות או מפתחים מנוסים כדי להשתמש בבינה מלאכותית מתקדמת. הכלי מסתיר את המורכבות הטכנית ומציג ממשק נקי, אינטואיטיבי ויעיל.

2.7 סיכום: החירות החישובית כניצחון דמוקרטי

Ollama אינו רק כלי טכנולוגי; הוא הצהרה פוליטית וארכיטקטורית על העתיד של הבינה המלאכותית. במקום לקבל את ההנחה שבינה מלאכותית מתקדמת חייבת להיות מרוכזת בידי גופים גדולים ונגישה רק באמצעות שירותי ענן יקרים, Ollama מציע חזון שונה לחלוטין: בינה מלאכותית שהיא מבוזרת, נגישה, פרטית וחופשית.

היתרונות הם ברורים: עלות אפסית לאחר השקעה ראשונית, פרטיות מלאה על הנתונים, מהירות מיידית ללא השהיות רשת, גמישות מלאה בניהול ובהתאמת מודלים, ועצמאות מוחלטת מתלות בספקים חיצוניים. אלה אינם יתרונות שוליים; אלה יסודות הריבונות הדיגיטלית.

כמובן, Ollama אינו פתרון מושלם לכל מצב. יש משימות שעדיין דורשות מודלים ענקיים שרק שרתי ענן עוצמתיים יכולים להריץ. אך עבור מכלול רחב של משימות---ניתוח טקסטים, סיכום מסמכים, תרגום, מענה על שאלות, סיוע בכתיבה---מודלים מקומיים כמו Llama 3.2 או Gemma מספקים ביצועים מעולים. ככל שכרטיסי הגרפי הופכים חזקים יותר וזולים יותר, ומודלים הופכים יעילים יותר, הטווח של משימות שניתן לבצע באופן מקומי רק יתרחב.

המהפכה הנגדית של Ollama היא חלק ממגמה רחבה יותר של דמוקרטיזציה טכנולוגית. כשם שהמחשב האישי שחרר את כוח העיבוד מידי מרכזי המחשוב הענקיים של שנות השישים, כך Ollama משחרר את כוח הבינה המלאכותית מידי מרכזי הענן של שנות העשרים. זהו ניצחון של החירות החישובית---החזרת השליטה על הכלים הדיגיטליים שלנו לידינו.

בפרק הבא, נצלול לעומק הטכני ונבחן את האנטומיה הפנימית של מודל שפה מקומי: מהם פרמטרים, מהו אורך הקשר, מהי קוונטיזציה, וכיצד ניתן לנהל מודלים באופן יעיל. הבנת היסודות הללו חיונית למי שמעוניין לנצל את מלוא הכוח של Ollama ולבנות אפליקציות ריבוניות אמיתיות.

© Dr. Yoram Segal - כל הזכויות שמורות

פרק 3

אנטומיה של מודל שפה מקומי: מושגי יסוד וניהול

Ollama ו-RAG

ריבונות דיגיטלית בעידן הבינה המלאכותית

Author: Dr. Yoram Segal

All rights reserved

11/2025

תוכן העניינים

פרק 3

אנטומיה של מודל שפה מקומי: מושגי יסוד וניהול

כדי שאדם יוכל לשלוט בכלי שלו, עליו להבין את האנטומיה הפנימית שלו. מודלי שפה אינם קופסאות שחורות מיסטיות המרחפות במרחבי הענן, אלא מבנים מתמטיים מורכבים שניתן למדוד, להבין ולשלוט בהם. כשם שרופא צריך להבין את האנטומיה האנושית כדי לרפא, כך מפתח או חוקר המבקש לרתום את כוחם של מודלי שפה צריך להכיר את המרכיבים הפנימיים שלהם, את המשתנים המגדירים את התנהגותם, ואת הפרמטרים הקובעים את ביצועיהם.

בפרק זה נחשוף את השכבות הפנימיות של מודל שפה מקומי. נבחן את הפרמטרים שהם הלב הפועם של המודל, את אורך הקשר שקובע את יכולת הזיכרון שלו, את ההטמעה הוקטורית המאפשרת לו להבין משמעות, ואת הקוונטיזציה המאפשרת לו לרוץ על חומרה צנועה. מעבר להבנה התיאורטית, נלמד כיצד לנהל מודלים אלה בפועל באמצעות כלי CLI פשוטים אך עוצמתיים ש־Ollama מספק. ההבנה הזו היא התשתית להגשמת החירות החישובית שאליה אנו שואפים.

3.1 פרמטרים: צפיפות המחשבה הדיגיטלית

אם נאלץ לבחור מושג בודד המגדיר את עוצמתו של מודל שפה, זה יהיה מספר הפרמטרים שלו. הפרמטרים הם המשקלים וההטיות הפנימיים שהמודל לומד במהלך תהליך האימון המסיבי שלו על מיליארדי מילים מהאינטרנט. כל פרמטר הוא מספר עשרוני זעיר המייצג את הכוח של קשר בודד ברשת הנוירונים המלאכותית. יחד, מיליארדים של משקלים אלה יוצרים את היכולת הקוגניטיבית של המודל, את הצפיפות של המחשבה הדיגיטלית שלו.

כאשר אנו קוראים על מודל Llama 3.1 45B, האות B מייצגת Billion—מיליארד. כלומר, מודל זה מכיל 45 מיליארד פרמטרים. בהשוואה, מודל Llama 3.2 3B מכיל רק 3 מיליארד פרמטרים. ההבדל אינו קוסמטי. ככל שמספר הפרמטרים גדול יותר, כך גדלה מורכבות המודל, עומק ההבנה שלו, ויכולתו לתפוס דפוסים עדינים בשפה. מודל בעל 70 מיליארד פרמטרים יכול להבין קשרים סמנטיים מורכבים יותר, לזכור הקשרים ארוכים יותר, ולהפיק תגובות מדויקות ומתוחכמות יותר ממודל בעל 3 מיליארד פרמטרים.

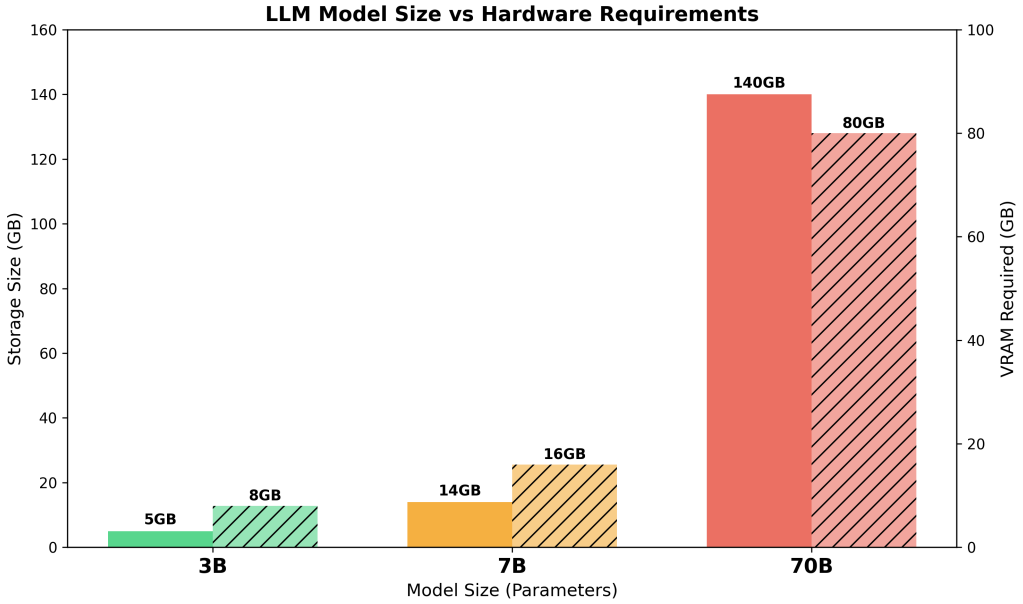
אך כאן מגיחה מציאות קשה: צפיפות המחשבה באה במחיר. כל פרמטר צריך להישמר בזיכרון המחשב, וכל חישוב שהמודל מבצע דורש לעבור על מיליארדי הפרמטרים הללו. מודל Llama 3.1 45B דורש כ־229 GB של שטח אחסון פנוי, וכרטיס גרפי עוצמתי בעל זיכרון VRAM רב כדי להריץ אותו בזמן סביר. זהו Tradeoff מובהק, פשרה שאי אפשר להתחמק ממנה: ביצועים מול משאבים, דיוק מול נגישות, כוח מול פרקטיות.

לכן, עבור משתמש ביתי רגיל או ארגון קטן, מודל כמו Llama 3.2 3B מציע איזון מיטבי. הוא מספק ביצועים סבירים להבנת שפה, יצירת תוכן, ואף ביצוע משימות מורכבות כמו סיכום מסמכים או מענה על שאלות, כל זאת תוך צריכת משאבים המתאימה למחשבים אישיים מודרניים. הבחירה במספר הפרמטרים המתאים אינה רק טכנית—היא פילוסופית. האם אנו מחפשים את המודל החכם

ביותר, או את המודל הזמין ביותר? האם אנו מוכנים להשקיע בחומרה יקרה כדי לקבל דיוק מקסימלי, או שנסתפק בדיוק טוב בשביל עצמאות מלאה?

תפקידו של Ollama הוא להפוך את הפשרה הזו לפשוטה. באמצעות פקודה אחת פשוטה, משתמש יכול להוריד ולהתנסות במודלים בגדלים שונים, לבדוק את הביצועים על החומרה שלו, ולבחור את נקודת האיזון האישית שלו. אין צורך בידע טכני מעמיק או בניסויים מסובכים—Ollama מנהל את הכל מאחורי הקלעים, ומאפשר למשתמש להתמקד במה שחשוב: השימוש בפועל במודל.

איור 3.1 ממחיש את הקשר בין מספר הפרמטרים לבין גודל המודל ודרישות החומרה. כפי שניתן לראות, הגידול במספר הפרמטרים מביא לגידול מעריכי בגודל הקובץ ובדרישות הזיכרון. המעבר מ־3 מיליארד פרמטרים ל־7 מיליארד מכפיל את הגודל פי שניים לערך, אך המעבר ל־70 מיליארד דורש קפיצה משמעותית בתשתית החומרה. זהו הביטוי הקונקרטי של הפשרה שתיארנו: יותר אינטליגנציה פירושה יותר משאבים.



איור 3.1: השוואת גדלי מודלים: הקשר בין מספר הפרמטרים לדרישות חומרה

טבלה 3.1 מציגה דוגמאות קונקרטיות של מודלים פופולריים המוצעים על־ידי Ollama, את מספר הפרמטרים שלהם, גודל הקובץ המשוער, והמלצות לחומרה מינימלית. הטבלה מדגימה כיצד בחירת המודל משפיעה ישירות על הנגישות והפרקטיות של השימוש המקומי.

טבלה 3.1: דוגמאות למודלים זמינים ב־Ollama ודרישות החומרה שלהם

שם המודל	פרמטרים	גודל	זיכרון מינימלי
Llama 3.2 1B	1B	1.3 GB	4 GB RAM
Llama 3.2 3B	3B	2.0 GB	8 GB RAM
Llama 3.1 8B	8B	4.7 GB	16 GB RAM
Llama 3.1 45B	45B	229 GB	64 GB RAM + GPU
Llama 3.1 70B	70B	40 GB	80 GB VRAM

כפי שמוצג בטבלה, המודל הקטן Llama 3.2 1B נגיש כמעט לכל מחשב מודרני, בעוד המודלים הגדולים דורשים תשתית חומרה מתקדמת. העובדה ש־Ollama מאפשר להוריד ולנסות כל אחד מהמודלים הללו בפקודה אחת פשוטה היא דוגמה לדמוקרטיזציה של הטכנולוגיה. אין צורך להבין את הפרטים הטכניים של ניהול זיכרון או אופטימיזציות חומרה—Ollama מטפל בכל זה באופן שקוף.

© Dr. Yoram Segal - כל הזכויות שמורות

3.2 אורך הקשר והטמעה: זיכרון ומשמעות

אם הפרמטרים הם כוח המחשבה, אזי אורך הקשר הוא היכולת לזכור. מודל שפה, כמו אדם, לא יכול להבין משפט בודד ללא הקשר. כדי לענות על שאלה באופן מדויק, המודל צריך לזכור את השיחה שקדמה לה, את המסמכים שהוצגו לו, ואת ההקשר הרחב יותר שבו התבקש לפעול. אורך הקשר, או Context Length, מתייחס למספר המקסימלי של יחידות טקסט—Tokens—שהמודל יכול לקלוט ולעבד בבת אחת.

יחידת הטקסט הבסיסית במודלי שפה היא ה־Token. Token יכול להיות מילה, חלק ממילה, או אפילו סימן פיסוק. באופן ממוצע, מילה אנגלית תופסת כ־1.3 Tokens, ומילה עברית תופסת כ־2--3 Tokens בשל המבנה המורפולוגי המורכב יותר של העברית. כאשר מודל מוגדר כבעל אורך קשר של 131,072 Tokens, המשמעות היא שהוא יכול לקרוא ולעבד מסמך שאורכו כ־100,000 מילים אנגליות או כ־50,000 מילים עבריות בבת אחת.

למה זה חשוב? דמיינו שאתם רוצים שהמודל יסכם דוח כלכלי של 80 עמודים. אם אורך הקשר של המודל קצר מדי, הוא פשוט לא יוכל לקרוא את כל הדוח בבת אחת. הוא יתעלם מהעמודים האחרונים, או יאבד את ההקשר המצטבר שנבנה לאורך הדוח. לעומת זאת, מודל בעל אורך קשר ארוך יוכל לקרוא את כל הדוח, להבין את הקשרים בין פרקים שונים, ולספק סיכום מקיף ומדויק.

אורך הקשר הופך לקריטי במיוחד ביישומי RAG, שעליהם נרחיב בפרקים הבאים. במערכת RAG, המודל מקבל לא רק את שאלת המשתמש, אלא גם פיסות רבות של מידע רלוונטי שאוחזרו ממסמכי מקור. אם אורך הקשר קצר, המערכת תוכל להעביר רק כמות מוגבלת של מידע קונטקסטואלי למודל, מה שמפחית את הדיוק והאמינות. לעומת זאת, אורך קשר ארוך מאפשר למודל לקבל עושר של ראיות ממקורות רבים, ולבנות תשובה מקורקעת היטב.

מודלי Llama 3.1 ו־Llama 3.2 מציעים אורך קשר של עד 131,072 Tokens, מה שהופך אותם לאידיאליים ליישומי RAG מקומיים. ניתן להעביר למודל מסמכים שלמים, מספר פרקים מספר, או עשרות פיסות קונטקסט, והמודל יוכל לעבד את כולם ביחד. זוהי יכולת שעד לפני שנים ספורות הייתה זמינה רק במודלים ענקיים שרצו על תשתיות ענן יקרות.

לצד אורך הקשר, מושג נוסף חיוני להבנת המודל הוא Embedding Length—אורך ההטמעה. בעוד אורך הקשר מתייחס לכמות הטקסט שהמודל יכול לקרוא, אורך ההטמעה מתייחס לעושר הייצוג הפנימי של כל יחידת טקסט. כאשר מודל קורא מילה, הוא אינו שומר אותה כמחרוזת של אותיות. במקום זאת, הוא ממיר אותה לוקטור—סדרה של מספרים—המייצג את המשמעות הסמנטית של המילה.

דמיינו שכל מילה היא נקודה במרחב רב־ממדי. מילים בעלות משמעות דומה יהיו קרובות זו לזו במרחב זה, בעוד מילים שונות יהיו רחוקות. אורך ההטמעה הוא מספר הממדים במרחב הזה. מודל בעל אורך הטמעה של 3,072 ממדים יכול לייצג ניואנסים סמנטיים עדינים יותר ממודל בעל 768 ממדים. ככל שאורך ההטמעה גדול יותר, כך המודל מסוגל לתפוס הבדלים דקים בין מילים, להבין הקשרים מורכבים, ולהפיק תגובות מדויקות יותר.

הנוסחה שמתארת את מימד ההטמעה של מודל Transformer קלאסי היא:

$$d_{\text{model}} = n_{\text{heads}} \times d_{\text{head}} \tag{3.1}$$

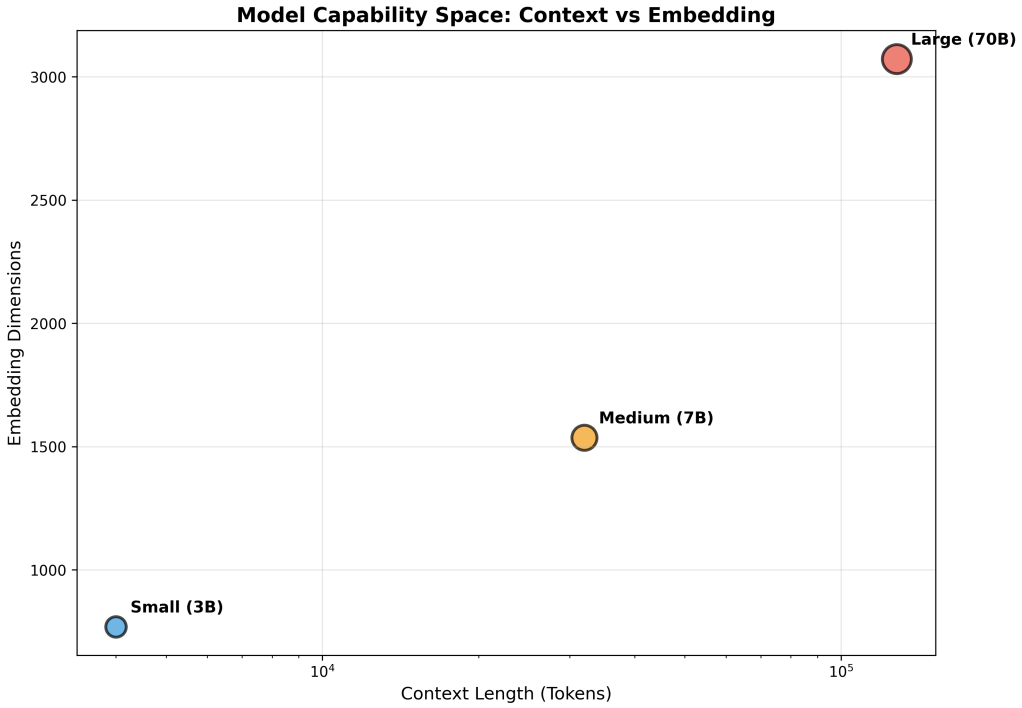
כאשר d_{model} הוא מימד ההטמעה הכולל, n_{heads} הוא מספר ראשי הקשב (Attention Heads), ו־d_{head} הוא מימד כל ראש. נוסחה זו מראה כיצד הארכיטקטורה הפנימית של המודל קובעת את עושר הייצוג הסמנטי שלו.

אורך ההטמעה חיוני גם ליישומי RAG. כאשר מערכת RAG מחפשת מסמכים רלוונטיים, היא ממירה את שאלת המשתמש ואת כל פיסות הטקסט במאגר לוקטורים, ואז מחפשת את הוקטורים הקרובים ביותר במרחב ההטמעה. ככל שמימד ההטמעה גדול יותר, כך החיפוש מדויק יותר, ופיסות הטקסט הנבחרות רלוונטיות יותר לשאלה. מודל כמו nomic-embed-text, שעליו נרחיב בפרקים הבאים, מציע אורך הטמעה של 768 ממדים, מה שמספיק לרוב היישומים המעשיים.

איור 3.2 ממחיש את הקשר בין אורך הקשר לאורך ההטמעה. אורך הקשר קובע כמה מידע המודל יכול לקרוא בבת אחת, בעוד אורך ההטמעה קובע כמה עשיר הייצוג הפנימי של כל פיסת

© Dr. Yoram Segal - כל הזכויות שמורות

מידע. ביחד, שני המימדים הללו קובעים את יכולת ההבנה של המודל.



איור 3.2: המרחב הדו־ממדי של יכולת המודל: אורך קשר מול אורך הטמעה

כפי שניתן לראות באיור, מודל אידיאלי נמצא בפינה הימנית העליונה—אורך קשר ארוך ואורך הטמעה גדול. אך שוב, זהו Tradeoff: מודלים כאלה דורשים יותר זיכרון וכוח חישוב. המשתמש צריך לבחור את המודל המתאים ליכולות החומרה שלו ולדרישות היישום שלו.

3.3 קוונטיזציה: הפשרה בין דיוק ליעילות

עד כה דיברנו על המאפיינים שהופכים מודלים לחזקים יותר: יותר פרמטרים, אורך קשר ארוך יותר, הטמעה עשירה יותר. אך כל אלה באים במחיר של משאבים. מה אם נוכל לדחוס את המודל, להפחית את הגודל שלו ואת צריכת המשאבים, מבלי לוותר יותר מדי על הדיוק? זה בדיוק מה שקוונטיזציה, או Quantization, מאפשרת.

קוונטיזציה היא טכניקה מתמטית המפחיתה את הדיוק של המשקלים במודל. במקום לייצג כל משקל כמספר בעל 32 ביטים (32-bit Float), המודל משתמש בייצוג של 8 ביטים, 4 ביטים, או אפילו פחות. הפחתת הדיוק הזו מביאה לשני יתרונות משמעותיים: ראשית, גודל הקובץ של המודל קטן פי כמה, מה שמאפשר להוריד ולאחסן מודלים גדולים על חומרה מוגבלת. שנית, חישובים על מספרים בעלי פחות ביטים מהירים יותר, מה שמביא לזמני תגובה קצרים יותר.

ההשפעה דרמטית. מודל Llama 3.1 8B בדיוק מלא (FP32) תופס כ־32 GB. אותו מודל לאחר קוונטיזציה ל־4 ביטים תופס רק כ־4.7 GB—כמעט פי שבעה פחות. ההפחתה בגודל הזו הופכת מודלים שהיו בלתי נגישים למרוב המשתמשים לזמינים ופרקטיים.

האם יש מחיר? כמובן. הפחתת הדיוק של המשקלים פוגעת במידה מסוימת בביצועי המודל. המודל עלול להיות פחות מדויק בחישובים מתמטיים מורכבים, פחות עקבי בתגובות, ופחות מסוגל להבין ניואנסים דקים. אך במרבית היישומים המעשיים—כגון יצירת תוכן, סיכום מסמכים, מענה על שאלות—ההפסד בדיוק הוא זניח לעומת היתרונות העצומים בנגישות ומהירות.

הנוסחה הבסיסית לחישוב גודל מודל לאחר קוונטיזציה היא:

$$\text{לדוג} = \frac{n_{\text{params}} \times b}{8 \times 10^9} \quad [\text{GB}] \tag{3.2}$$

כאשר n_{params} הוא מספר הפרמטרים במיליארדים, b הוא מספר הביטים לייצוג כל פרמטר, והחלוקה ב־8 היא המרה מביטים לבייטים. לדוגמה, מודל בעל 8 מיליארד פרמטרים עם קוונטיזציה של 4 ביטים יתפוס: $\frac{8\times4}{8} = 4$ GB.

Ollama תומך באופן שקוף במודלים מקוונטזים. כאשר משתמש מוריד מודל דרך Ollama, המערכת אוטומטית מספקת את הגרסה המקוונטזת המתאימה לחומרה שלו. אין צורך להבין את הפרטים הטכניים של קוונטיזציה—Ollama מטפל בכל זה מאחורי הקלעים, ומאפשר למשתמש ליהנות מהיתרונות מבלי להתמודד עם המורכבות.

טבלה 3.2 משווה בין רמות קוונטיזציה שונות של אותו מודל. ניתן לראות כיצד ההפחתה במספר הביטים משפיעה על הגודל, המהירות, והדיוק היחסי.

טבלה 3.2: השוואת רמות קוונטיזציה שונות עבור מודל Llama 3.1 8B

רמת קוונטיזציה	גודל קובץ	מהירות יחסית	דיוק יחסי
FP32 (Full Precision)	32 GB	1.0x (Baseline)	100%
FP16 (Half Precision)	16 GB	1.5x	99%
INT8 (8-bit)	8 GB	2.5x	97%
INT4 (4-bit)	4.7 GB	4.0x	92%

כפי שמוצג בטבלה, קוונטיזציה ל־4 ביטים מפחיתה את הגודל ב־85% ומגבירה את המהירות פי ארבעה, תמורת הפסד של כ־8% בדיוק. זהו Tradeoff מצוין עבור רוב היישומים המעשיים. המשתמש צריך לשקול את הצרכים שלו: אם הוא מחפש דיוק מקסימלי למשימה קריטית, עדיף להשתמש במודל פחות מקוונטז. אם הוא מחפש מהירות ונגישות למשימות יומיומיות, קוונטיזציה אגרסיבית היא הבחירה הנכונה.

איור 3.3 ממחיש את הפשרה בין גודל, מהירות, ודיוק ברמות קוונטיזציה שונות. ניתן לראות בבירור כיצד קוונטיזציה אגרסיבית יותר משפרת את הגודל והמהירות, אך במחיר של דיוק מעט נמוך יותר.

הפשרה הזו אינה רק טכנית—היא פילוסופית. קוונטיזציה מאפשרת לנו לבחור איזה מימד של הביצועים חשוב לנו יותר. האם אנו מעריכים את הדיוק המקסימלי, או את הנגישות והמהירות? האם אנו מוכנים לוותר על כמה אחוזי דיוק כדי שהמודל ירוץ על המחשב הנייד שלנו, במקום לדרוש שרת ייעודי? קוונטיזציה נותנת לנו את החירות לבחור.

3.4 ניהול מודלים: הכלים הפרקטיים של Ollama CLI

כל ההבנה התיאורטית שרכשנו עד כה הופכת למעשית דרך כלי ניהול המודלים של Ollama. ממשק שורת הפקודה, או CLI (Command-Line Interface), הוא הדרך הבסיסית והמהירה ביותר לאינטראקציה עם Ollama. דרכו מתנהלים כל תהליכי הורדת המודלים, הפעלתם, בדיקתם, ומחיקתם. Ollama CLI הוא הממשק שבאמצעותו המשתמש מפעיל את החירות החישובית שלו.

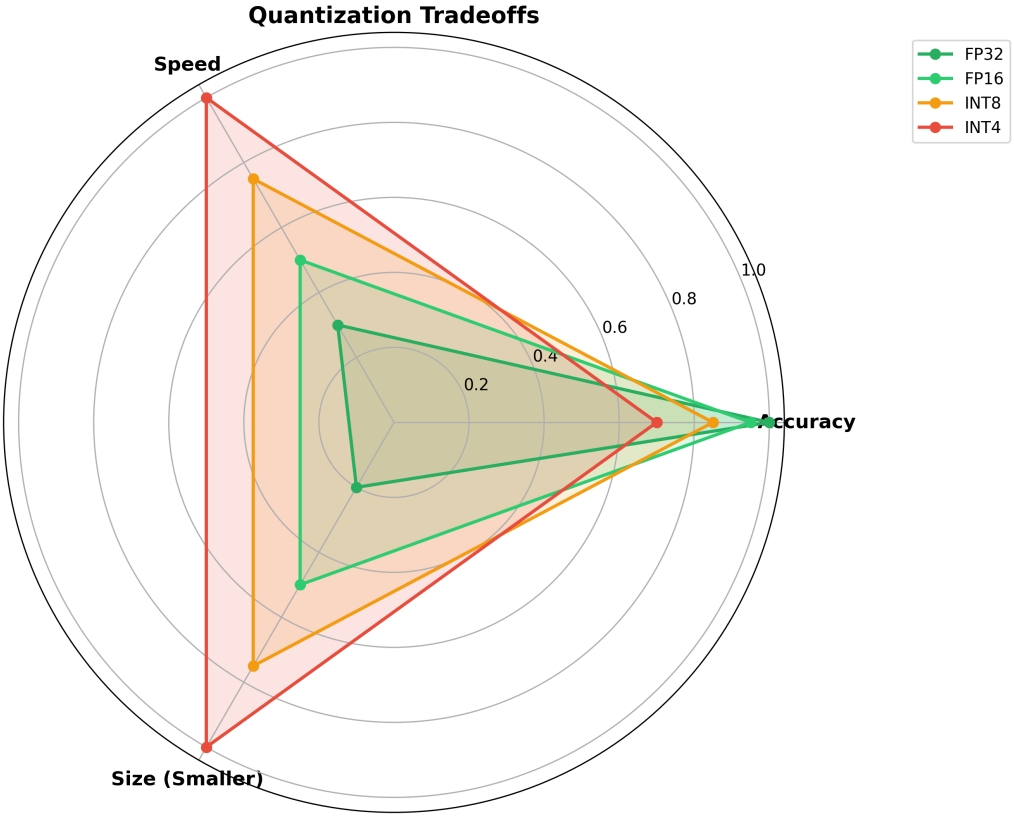
ניתן לחלק את הפעולות הבסיסיות של Ollama CLI לארבע קטגוריות עיקריות: הרצה, רישום, מידע, ומחיקה. כל פעולה מתבצעת באמצעות פקודה פשוטה, אך מאחורי הקלעים Ollama מבצע פעולות מורכבות של ניהול זיכרון, אופטימיזציה, והפעלת המודל.

הרצת מודל: ollama run הפקודה הבסיסית ביותר היא ollama run <model>. פקודה זו מורידה את המודל המבוקש (אם הוא לא קיים עדיין), טוענת אותו לזיכרון, ופותחת Shell אינטראקטיבי שבו המשתמש יכול לתקשר עם המודל בזמן אמת. זוהי הדרך המהירה ביותר להתחיל לעבוד עם מודל חדש.

לדוגמה, כדי להריץ את המודל llama3.2:

```
ollama run llama3.2
```


© rD. maroY lageS - כל הזכויות שמורות



איור 3.3: משולש הפשרות: דיוק מול גודל מול מהירות ברמות קוונטיזציה שונות

הפקודה תוריד את המודל (אם נדרש), ותציג פרומפט אינטראקטיבי:

```
>>> Send a message (/? for help)
```

כעת המשתמש יכול להקליד כל שאלה או בקשה, והמודל יגיב באופן מיידי. לדוגמה:

```
>>> What is artificial intelligence?
```

והמודל יגיב:

```
Artificial Intelligence (AI) is a branch of computer science
that deals with creating computer systems capable of performing
tasks that typically require human intelligence. This includes
learning, reasoning, problem solving, language understanding,
and pattern recognition.
```

הפשטות של הממשק הזה מסתירה את המורכבות הטכנית מאחורי הקלעים. Ollama טוען את המודל לזיכרון, מנהל את הקונטקסט של השיחה, ומייעל את החישובים לפי החומרה הזמינה. המשתמש רואה רק את התוצאה—אינטראקציה חלקה עם מודל שפה מתקדם.

רישום מודלים: ollama list לאחר שהורדנו מספר מודלים, ייתכן שנרצה לראות רשימה של כל המודלים המותקנים במחשב שלנו. הפקודה ollama list מציגה מידע מפורט על כל המודלים המקומיים:

```
ollama list
```

פלט טיפוסי יכול להיות:

```
NAME                ID              SIZE     MODIFIED
```


© Dr. Yoram Segal - כל הזכויות שמורות

```
llama3.2:latest    a80c4f17acd5  2.0 GB  3 days ago
llama3.1:8b        3b72c4536f74  4.7 GB  1 week ago
nomic-embed-text   0a109f422b47  274 MB  2 weeks ago
```

הרשימה מציגה את שם המודל, מזהה ייחודי שלו, הגודל על הדיסק, ומתי הוא הותקן או עודכן לאחרונה. מידע זה מאפשר למשתמש לנהל את משאבי האחסון שלו, לזהות מודלים ישנים שניתן למחוק, ולעקוב אחר הגרסאות השונות של המודלים.

שימו לב שהמודל nomic-embed-text קטן בהרבה מהמודלים האחרים. זה מכיוון שמודלי הטמעה, שתפקידם רק להמיר טקסט לוקטורים, דורשים פחות פרמטרים ממודלי שפה גנרטיביים מלאים. זוהי דוגמה לכך שלא כל המודלים נוצרו שווים—כל מודל מתמחה במשימה ספציפית.

מידע על מודל: ollama show כדי לקבל מידע טכני מפורט על מודל ספציפי, משתמשים בפקודה ollama show <model>. לדוגמה:

```
ollama show llama3.2
```

הפקודה תציג פרטים מקיפים על המודל:

```
Model
  architecture        llama
  parameters          3.2B
  context length      131072
  embedding length    3072
  quantization        Q4_K_M

License
  Meta Llama 3.2 Community License

Parameters
  stop    "<|start_header_id|>"
  stop    "<|end_header_id|>"
  stop    "<|eot_id|>"

System
  You are a helpful assistant.
```

המידע הזה חיוני להבנת יכולות המודל. אנו רואים כאן:

- **ארכיטקטורה**: זה מודל מסוג Llama, המבוסס על ארכיטקטורת Transformer.
- **פרמטרים**: 3.2 מיליארד פרמטרים—מודל בגודל בינוני, מתאים למחשבים אישיים.
- **אורך קשר**: 131,072 Tokens—יכולת זיכרון עצומה, מתאימה למסמכים ארוכים ולייישומי RAG.
- **אורך הטמעה**: 3,072 ממדים—ייצוג סמנטי עשיר.
- **קוונטיזציה**: Q4_K_M—קוונטיזציה ל־4 ביטים, מאוזנת בין גודל לדיוק.

מידע זה מאפשר למשתמש לקבל החלטה מושכלת לגבי המודל. האם הוא מתאים לחומרה שלו? האם אורך הקשר מספיק ארוך למשימה שלו? האם רמת הקוונטיזציה תספק את הדיוק הנדרש?

© rD. maroY lageS - כל הזכויות שמורות

מחיקת מודל: ollama rm לאחר שעבדנו עם מודלים שונים, ייתכן שנחליט למחוק חלק מהם כדי לפנות מקום. הפקודה ollama rm <model> מוחקת מודל מהמחשב:

```
ollama rm llama3.1:8b
```

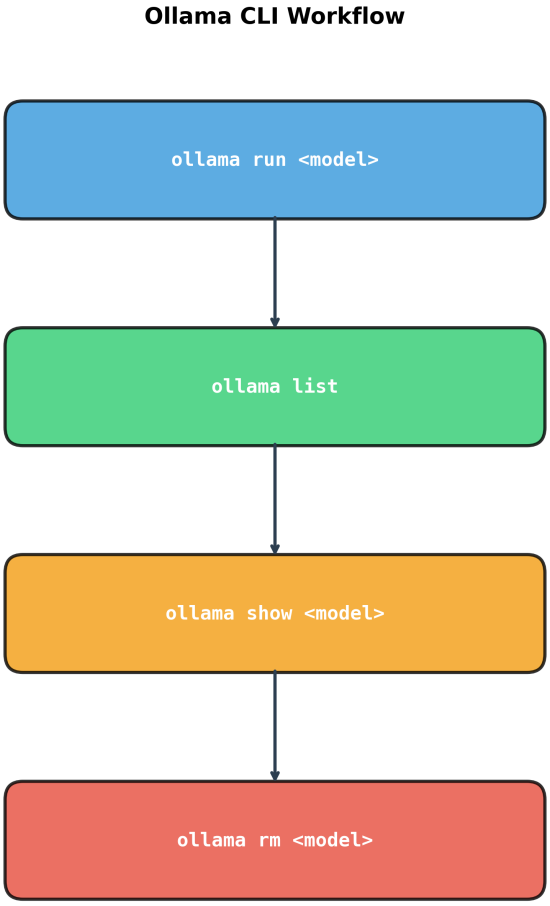
הפלט יהיה:

```
deleted 'llama3.1:8b'
```

המודל נמחק לחלוטין מהדיסק, ומשחרר 4.7 GB של שטח אחסון. פעולה זו הפיכה—ניתן תמיד להוריד שוב את המודל אם נצטרך אותו בעתיד.

ניהול המודלים באמצעות Ollama CLI הוא פשוט, שקוף, ומהיר. המשתמש שולט לחלוטין באיזה מודלים להוריד, להריץ, ולמחוק. אין צורך בממשקים גרפיים מסורבלים או בהגדרות מורכבות. ארבע פקודות פשוטות מספיקות לניהול מלא של מאגר המודלים המקומי.

איור 3.4 ממחיש את זרימת העבודה הטיפוסית עם Ollama CLI. המשתמש מוריד מודל, בודק את המאפיינים שלו, מריץ אותו, ולבסוף מוחק אותו אם אינו נדרש עוד. כל שלב מתבצע בפקודה אחת פשוטה.



איור 3.4: זרימת עבודה טיפוסית עם Ollama CLI: מהורדה למחיקה

© Dr. Yoram Segal - כל הזכויות שמורות

כפי שניתן לראות באיור, הממשק הוא לינארי ואינטואיטיבי. אין תהליכים מורכבים, אין תצורות נסתרות. המשתמש מבצע פעולה, Ollama מבצע אותה, ומדווח על ההצלחה. פשטות זו היא המפתח לדמוקרטיזציה של הטכנולוגיה—כל מי שיודע להשתמש בשורת פקודה יכול לנהל מודלי שפה מתקדמים.

3.5 טבלת מושגי היסוד: סיכום מושגי הפרק

לאחר שעברנו על הרכיבים הפנימיים של מודל שפה מקומי, חשוב לסכם את המושגים המרכזיים באופן מובנה. טבלה 3.3 מציגה את המושגים העיקריים, הגדרתם, משמעותם למשתמש, והשפעתם על בחירת המודל.

טבלה 3.3: מושגי יסוד באנטומיה של מודל שפה מקומי

מושג	הגדרה	משמעות למשתמש	השפעה על בחירה
Parameters	המשקלים וההטיות הפנימיים של הרשת הנוירונית, נמדדים במיליארדים	ככל שיותר פרמטרים, המודל חכם ומדויק יותר	מודל גדול דורש יותר זיכרון וכוח חישוב
Context Length	מספר ה־Tokens המקסימלי שהמודל יכול לעבד בבת אחת	ככל שארוך יותר, המודל יכול לקרוא מסמכים ארוכים יותר	אורך קשר ארוך קריטי ליישומי RAG ושיחות נרחבות
Embedding Length	מספר הממדים בייצוג הוקטורי של כל Token	ככל שגדול יותר, המודל מבין ניואנסים סמנטיים עדינים יותר	הטמעה עשירה משפרת דיוק באחזור ובהבנה
Quantization	הפחתת דיוק המשקלים (למשל, מ־32 ל־4 ביטים)	הקטנת הגודל והגברת המהירות, תמורת הפסד קל בדיוק	מאפשר להריץ מודלים גדולים על חומרה צנועה
Model Size	גודל הקובץ של המודל על הדיסק, במדידות GB	קובע כמה שטח אחסון ניתן לייעד למודלים	צריך לאזן בין גודל המודל לשטח האחסון הזמין

כפי שמוצג בטבלה, כל מושג משפיע ישירות על החלטות המשתמש. הבנת המושגים הללו מאפשרת למשתמש לבחור את המודל המתאים לצרכיו, לחומרה שלו, וליישום שלו. אין מודל אידיאלי אחד—יש מודלים המתאימים לצרכים שונים.

המסקנה המרכזית של הפרק היא שמודלי שפה אינם קופסאות שחורות בלתי חדירות. הם מבנים מובנים, עם פרמטרים מדידים, שניתן להבין, לשלוט בהם, ולהתאים אישית. Ollama מספק את הכלים להפוך את ההבנה התיאורטית הזו למעשית—להוריד, לנסות, להשוות, ולבחור מודלים בקלות ובמהירות.

בפרק הבא נעבור מהאנטומיה התיאורטית לפרקטיקה: נלמד כיצד לעצב את האישיות הדיגיטלית של המודל, כיצד להתאים אותו למשימות ספציפיות, וכיצד לשלב אותו באפליקציות אמיתיות. ההבנה שרכשנו בפרק זה תשמש כתשתית להבנת הפעולות המתקדמות יותר שנבצע בהמשך.

© Dr. Yoram Segal - כל הזכויות שמורות

פרק 4

עיצוב האישיות הדיגיטלית: הפרקטיקה של Ollama

Ollama ו-RAG

ריבונות דיגיטלית בעידן הבינה המלאכותית

Author: Dr. Yoram Segal

All rights reserved

11/2025

תוכן העניינים

פרק 4

עיצוב האישיות הדיגיטלית: הפרקטיקה של Ollama

בראשית הדיגיטל, המכונות היו כלי פסיביים וציותניים. הן חיכו לפקודות ברורות וביצעו אותן בדייקנות רובוטית. אך עם הופעת מודלי השפה הגדולים, התפר הזה נפתח. המכונה כבר אינה רק מבצעת פקודות, היא שוחחת, מציעה, יוזמת. והנה עולה השאלה המהותית: אם המכונה מדברת, מהי הזהות שלה? מהו האופי הקוגניטיבי שאנו רוצים שהיא תגלם?

בפרקים הקודמים ראינו כיצד Ollama משחרר את הכוח החישובי מכבלי הענן ומביא אותו אל תוך הבית והמשרד. ראינו את הפרמטרים הטכניים המגדירים את יכולות המודל: גודל, אורך הקשר, קוונטיזציה. אך כל אלה הם רק המצע החומרי, השלד הטכנולוגי. כעת הגיע הזמן לדון בנשמה: כיצד מעצבים את הזהות הדיגיטלית של המודל? כיצד הופכים אותו ממנבא סטטיסטי אנונימי למומחה ייעודי, בעל אישיות מוגדרת ומטרה ברורה? כיצד משלבים אותו באפליקציות אמיתיות, חיות ונושמות?

פרק זה יוביל אותנו אל תוך הפרקטיקה העמוקה של Ollama. נתחיל בהנדסת הזהות באמצעות Modelfile, המסמך הקונפיגורטיבי המאפשר לנו לתכנת לא רק את תגובות המודל, אלא את עצם דרך חשיבתו. נמשיך אל ממשק הREST API, שער הקישוריות המאפשר לאפליקציות שלנו לשלוח שאילתות ולקבל תשובות באופן תכנותי. נצלול לתוך ספריית Python SDK, שכבת ההפשטה האלגנטית המקלה על המפתח לשלב את המודל בקוד. ולבסוף, נראה את כל אלה מתמזגים לכדי פרויקט מעשי שלם: מארגן רשימת מכולת אוטומטי, המוכיח כי אוטומציה קוגניטיבית מקומית היא לא חזון עתידני, אלא מציאות נוכחית וזמינה.

4.1 הנדסת זהות קוגניטיבית: הModelfile-

כשאנו מורידים מודל מוכן מהמחסן של Ollama, אנו מקבלים ישות גנרית. המודל עונה על שאלות, מייצר טקסט, מסכם מסמכים. אך הוא אינו יודע מי הוא. הוא אינו מכיר את תפקידו הספציפי בתוך המערכת שלנו. האם הוא עוזר שירות לקוחות? מנתח סנטימנט? מתרגם טכני? סופר יצירתי? התשובה: הוא עדיין אף אחד מאלה, והוא כולם בעת ובעונה אחת. זהו כוח פוטנציאלי לא מכוון.

הModelfile הוא הכלי שבאמצעותו אנו מכוונים את הפוטנציאל הזה. זהו קובץ קונפיגורציה טקסטואלי פשוט, אך השפעתו היא עמוקה: הוא מגדיר את תפקיד המודל, את טון הדיבור שלו, את רמת היצירתיות שלו, ואת המגבלות המושגיות שלו. בניגוד לפרומפט חד-פעמי שהמשתמש שולח, הModelfile הוא הוראה קבועה, מוטבעת, המלווה את המודל בכל אינטראקציה. זהו למעשה תעודת הזהות הקוגניטיבית של המכונה.

4.1.1 מבנה הModelfile-: הוראות ופרמטרים

במהותו, הModelfile מורכב משני סוגי הגדרות: הוראת SYSTEM והגדרות פרמטרים טכניים. הוראת הSYSTEM היא הפרומפט הקבוע המוזרק לכל שיחה, והיא מגדירה את התפקיד, הסגנון והמגבלות

של המודל. הפרמטרים, לעומת זאת, שולטים בהתנהגות הגנרטיבית: עד כמה המודל יהיה יצירתי או עובדתי, כמה טוקנים הוא יוכל לייצר, ועוד.

נניח שאנו רוצים ליצור מודל ייעודי לניתוח סנטימנט של ביקורות לקוחות. אנו זקוקים למודל שיהיה תמציתי, עובדתי, ובלתי רגשי. המודל לא צריך לספר סיפורים, לא צריך להיות "יצירתי" – הוא צריך לקרוא ביקורת ולהחזיר תווית פשוטה: positive, negative, או neutral. כיצד מעצבים זהות כזו? באמצעות Modelfile המוגדר כך:

```
FROM llama3.2

SYSTEM """
You are a precise sentiment analyzer.
Your only job is to classify customer reviews into one of three categories:
positive, negative, or neutral.

Be concise. Do not provide explanations unless asked.
Do not generate creative content.
Respond with a single word: positive, negative, or neutral.
"""

PARAMETER temperature 0.1
PARAMETER num_predict 50
```

Listing 4.1: תמגוד Modelfile טנמיטנס חותינל

בקוד זה, כפי שמוצג ברשימה 1.4, אנו רואים כמה רבדים של עיצוב זהות. שורת FROM מציינת את מודל הבסיס – במקרה זה llama3.2, המודל הקל והמהיר שבחרנו. בלוק הSYSTEM מגדיר את התפקיד הקוגניטיבי: "אתה מנתח סנטימנט מדויק". הוא קובע את המשימה (סיווג), את הפורמט הרצוי (תשובה בת מילה אחת), ואת ההגבלות (ללא יצירתיות, ללא הסברים מיותרים).

הפרמטר temperature, המוגדר כאן ל0.1, הוא קריטי. ערך נמוך זה מכוון את המודל לייצר תשובות דטרמיניסטיות, עקביות, ופחות יצירתיות. כאשר temperature קרובה לאפס, המודל בוחר תמיד במילה הסבירה ביותר סטטיסטית, מה שמוביל לדיוק ועקביות גבוהים. לעומת זאת, ערך גבוה (כגון 0.9 או 1.0) היה מעודד וריאציה, הפתעות, ויצירתיות – תכונות לא רצויות כאשר אנו רוצים תשובה חד-משמעית.

הפרמטר $\text{num}_predict$, המוגדר כאן ל50 טוקנים, מגביל את אורך התגובה. זה מונע מהמודל להתפרע ולייצר פסקאות ארוכות, ומכוון אותו להיות תמציתי. בעולם המושלם של ניתוח סנטימנט, התשובה צריכה להיות מילה אחת. 50 טוקנים הם מרווח נדיב, אך מספיק צר כדי לשמור על משמעת.

4.1.2 יצירת המודל המותאם: ollama create

לאחר שכתבנו את הModelfile, השלב הבא הוא ליצור ממנו מודל מותאם חדש. זה נעשה באמצעות הפקודה ollama create, פקודת קסם שלוקחת את הקונפיגורציה הטקסטואלית והופכת אותה למודל פונקציונלי, מוכן לשימוש.

נניח ששמרנו את הModelfile הנ"ל בקובץ בשם SentimentModelfile. היצירה של המודל החדש תיעשה כך:

```
ollama create sentiment-analyzer --file SentimentModelfile
```

Listing 4.2: מ-Modelfile מאתום לדומ תריצי

פקודה זו, כפי שמוצג ברשימה 2.4, יוצרת מודל חדש בשם sentiment-analyzer. מכאן והלאה, כל פעם שנריץ את המודל הזה (באמצעות ollama run sentiment-analyzer), הוא יגיע עם הזהות הקבועה שהגדרנו: מנתח סנטימנט תמציתי, עובדתי, עם temperature נמוכה. המודל זוכר מי הוא.

כל הזכויות שמורות - Dr. Yoram Segal ©

זהו שינוי תודעתי עמוק. במקום לשלוח כל פעם פרומפט ארוך המסביר למודל מהו תפקידו, אנו פשוט מפעילים מודל ייעודי שכבר יודע מה לעשות. זה חוסך זמן, משפר עקביות, ומפחית טעויות. זהו מעבר מאינטראקציה אד-הוק למערכת מושכלת ומתוכננת.

4.1.3 דוגמה נוספת: מודל לסיכום מסמכים טכניים

נניח כעת שאנו עובדים בחברת הייטק, ואנו זקוקים למודל שיסכם מסמכי תיעוד טכני ארוכים. המסמכים מלאי ז'רגון טכני, אך הסיכום צריך להיות ברור, קריא, ומובנה. במקרה זה, נרצה temperature בינונית (לא נמוכה מדי, לא גבוהה מדי), ונרצה להורות למודל להתמקד בנקודות המפתח בלבד.

הModelfile ייראה כך:

```
FROM llama3.2

SYSTEM """
You are a technical documentation summarizer.
Your task is to read lengthy technical documents and produce concise,
structured summaries in bullet points.

Focus on:
- Main objectives
- Key technical decisions
- Risks and dependencies
- Action items

Use clear, professional language. Avoid unnecessary verbosity.
"""

PARAMETER temperature 0.4
PARAMETER num_predict 300
```

Listing 4.3: ינכט מוכיסל Modelfile תמגוד

כאן, כפי שמוצג ברשימה 3.4, הפרמטר temperature מוגדר ל0.4 – גבוה יותר מאשר במודל הסנטימנט, כי אנו רוצים שהמודל יהיה גמיש מספיק לנסח מחדש רעיונות, אך עדיין דיי עובדתי. הפרמטר $\text{num}_predict$ הוא 300 טוקנים, מספיק כדי לאפשר סיכום מפורט, אך לא ארוך מדי.

יצירת המודל:

```
ollama create tech-summarizer --file TechnicalModelfile
```

Listing 4.4: ינכט מוכיס לדומ תריצי

כעת יש לנו שני מודלים מותאמים: sentiment-analyzer וtech-summarizer. כל אחד מהם בעל זהות ייחודית, מותאם למשימה ספציפית. זהו הכוח של הModelfile: היכולת לנהל גלריה שלמה של מומחים קוגניטיביים, כל אחד מכוון לתפקידו.

4.2 קישוריות לאפליקציות: הREST API- של Ollama

הנדסת הזהות באמצעות Modelfile היא רק השלב הראשון. השלב השני הוא אינטגרציה: כיצד אנו מפעילים את המודלים הללו לא באופן ידני מהטרמינל, אלא מתוך אפליקציות אמיתיות – אתרי אינטרנט, שרתים, סקריפטים אוטומטיים? התשובה: ממשק REST API.

© Dr. Yoram Segal - כל הזכויות שמורות

כאשר מריצים את Ollama (על ידי הפעלת השרת באופן ידני או אוטומטי), הוא פותח שרת מקומי בכתובת localhost:11434. שרת זה חושף מספר נקודות קצה (endpoints) המאפשרות לשלוח בקשות HTTP ולקבל תשובות. זהו ממשק סטנדרטי, מוכר, פשוט – כל שפת תכנות יודעת לדבר HTTP, ולכן כל שפת תכנות יכולה לדבר עם Ollama.

4.2.1 נקודות קצה עיקריות: /api/generate ו/api/chat-

שתי נקודות הקצה המרכזיות הן:

- /api/generate – משמשת לגנרציה חד-כיוונית: שולחים פרומפט, מקבלים תגובה.
- /api/chat – משמשת לשיחה רב-תורית: ניתן לשמור הקשר והיסטוריה על פני מספר הודעות.

4.2.2 דוגמה עם CURL: שאילתה פשוטה

הדרך הפשוטה והמהירה ביותר לבדוק את הAPI היא באמצעות CURL, כלי שורת פקודה לשליחת בקשות HTTP. נניח שאנו רוצים לשאול את מודל llama3.2 שאלה פשוטה: "מהי בירת צרפת?"

```
curl http://localhost:11434/api/generate -d '{
  "model": "llama3.2",
  "prompt": "What is the capital of France?",
  "stream": false
}'
```

Listing 4.5: תשקב API הטושפ הטע CURL הצק תדוקנל generate

בקשה זו, כפי שמוצג ברשימה 5.4, שולחת שלושה פרמטרים: את שם המודל (llama3.2), את הפרומפט, ואת הפרמטר stream. הפרמטר stream קובע האם התשובה תוחזר כולה במכה אחת (false) או שתזרום בזמן אמת, מילה אחר מילה (true). עבור אפליקציות רבות, stream: false הוא פשוט יותר, כי הוא מחזיר תשובה שלמה ומוכנה.

התשובה תחזור בפורמט JSON:

```
{
  "model": "llama3.2",
  "created_at": "2025-11-25T10:30:00Z",
  "response": "The capital of France is Paris.",
  "done": true
}
```

Listing 4.6: תבוגת API הצק תדוקנמ generate

השדה response מכיל את התשובה הגנרטיבית. פשוט, ברור, ופרקטי.

4.2.3 דוגמה עם /api/chat: שיחה רב-תורית

נניח כעת שאנו רוצים לנהל שיחה, שבה המודל זוכר מה נאמר קודם. אנו שולחים שתי הודעות: ראשונה – "מהי בירת צרפת?", שנייה – "ומה האוכלוסייה שלה?". המודל צריך להבין שהשאלה השנייה מתייחסת לפריז.

```
curl http://localhost:11434/api/chat -d '{
  "model": "llama3.2",
  "messages": [
    {"role": "user", "content": "What is the capital of France?"},
```


© rD. maroY lageS - כל הזכויות שמורות

```
    {"role": "assistant", "content": "The capital of France is Paris."},
    {"role": "user", "content": "What is its population?"}
  ],
  "stream": false
}'
```

Listing 4.7: /api/chat עם תירות-בר החיש

כאן, כפי שמוצג ברשימה 7.4, אנו שולחים מערך של הודעות (messages), כל אחת עם role (משתמש או עוזר) ו-content. המודל רואה את ההיסטוריה, מבין את ההקשר, ויכול לענות: "ehT stimil ytic eht nihtiw noillim 1.2 yletamixorppa si siraP fo noitalupop."

זהו כוחו של /api/chat: הוא מאפשר בניית ממשקי שיחה חכמים, המתנהלים על פני תורות רבות, תוך שמירת קונטקסט.

4.2.4 שליטה על פרמטרים: temperature ו-num$_predict$

גם דרך הAPI, ניתן לשלוט על פרמטרים כמו temperature. נניח שאנו רוצים תשובה יצירתית מאוד, עם temperature גבוהה:

```
curl http://localhost:11434/api/generate -d '{
  "model": "llama3.2",
  "prompt": "Write a creative opening line for a sci-fi novel.",
  "stream": false,
  "options": {
    "temperature": 0.9,
    "num_predict": 100
  }
}'
```

Listing 4.8: API דרך temperature לע הטילש

כאן, כפי שמוצג ברשימה 8.4, הוספנו את השדה options, המכיל פרמטרים מותאמים אישית. temperature: 0.9 תעודד יצירתיות, וריאציה, והפתעות. זה מושלם למשימות כמו כתיבת פתיחות לספרים, יצירת קמפיינים שיווקיים, או גיבוש רעיונות.

4.3 הפשטה אלגנטית: ספריית Python SDK

למרות שREST API הוא פשוט, הוא עדיין דורש כתיבת בקשות HTTP ידנית, ניהול JSON, וטיפול בשגיאות רשת. זה מייגע ונוטה לטעויות. הפתרון: ספריית Python SDK, המספקת ממשק פייתוני נקי, פשוט וקריא.

התקנת הספרייה היא טריוויאלית:

```
pip install ollama
```

Listing 4.9: תיירפס תנקתה Ollama Python SDK

4.3.1 דוגמה: שאילתה פשוטה עם ollama.generate()

בואו ננסה לבצע את אותה שאילתה שעשינו קודם עם CURL, אך הפעם בקוד פייתוני נקי:

© rD. maroY lageS - כל הזכויות שמורות

```python
import ollama

response = ollama.generate(
    model='llama3.2',
    prompt='What is the capital of France?'
)

print(response['response'])
```

Listing 4.10: ollama.generate() עם הטושפ התליאש

כפי שמוצג ברשימה 01.4, הקוד הוא קצר, ברור, ואידיומטי. הפונקציה ollama.generate() מקבלת את שם המודל ואת הפרומפט, ומחזירה את התשובה. אין צורך לכתוב JSON ידנית, אין צורך לבנות URL, אין צורך לטפל בHTTP headers. הכל מוסתר מאחורי ממשק פשוט.

4.3.2 שיחה רב-תורית עם ollama.chat()

כעת נדגים שיחה עם הקשר:

```python
import ollama

messages = [
    {'role': 'user', 'content': 'What is the capital of France?'},
    {'role': 'assistant', 'content': 'The capital of France is Paris.'},
    {'role': 'user', 'content': 'What is its population?'}
]

response = ollama.chat(
    model='llama3.2',
    messages=messages
)

print(response['message']['content'])
```

Listing 4.11: ollama.chat() עם תירות-בר החיש

כפי שמוצג ברשימה 11.4, אנו מעבירים מערך של הודעות, והמודל משיב בהתאם להקשר. הקוד הוא זהה במבנה לAPI, אך הרבה יותר קריא ופייתוני.

4.3.3 ניהול מודלים: ollama.list()

הספרייה גם מאפשרת לשאול אילו מודלים מותקנים:

```python
import ollama

models = ollama.list()
for model in models['models']:
    print(f"Model: {model['name']}, Size: {model['size']}")
```

Listing 4.12: סינקתומ סילדומ תמישר

כפי שמוצג ברשימה 21.4, הפונקציה ollama.list() מחזירה רשימה של כל המודלים המקומיים. זה שימושי לאפליקציות שצריכות לבחור מודל דינמית, או לבדוק אילו משאבים זמינים.

כל הזכויות שמורות - Dr. Yoram Segal ©

4.4 פרויקט מעשי: מארגן רשימת מכולת אוטומטי

עד כה ראינו כלים נפרדים: Modelfile לעיצוב זהות, REST API לקישוריות, Python SDK לפשטות. כעת הגיע הזמן לשלב את כולם לכדי פרויקט מעשי שלם. נבנה יישום פשוט אך שימושי: מארגן רשימת מכולת, המקבל רשימה לא מאורגנת של פריטי קניה, ומחזיר רשימה ממוינת, מקוטלגת ומעוצבת.

4.4.1 הבעיה: כאוס הקניות

נניח שחזרנו מן השוק עם פתק בכתב יד: "חלב, לחם, עגבניות, בננות, גבינה צהובה, מלפפונים, יוגורט, תפוחים, ביצים, בצל". זוהי רשימה גולמית, לא מסודרת. אם נלך לסופרמרקט עם רשימה כזו, נצטרך לקפוץ בין מחלקות: מוצרי חלב, ירקות, פירות, ובחזרה. זה לא יעיל.

פתרון אנושי: לשבת ולמיין את הרשימה ידנית. פתרון דיגיטלי: להשתמש במודל שפה מקומי, שיקרא את הרשימה, יזהה את הקטגוריות (מוצרי חלב, ירקות, פירות), ימיין אותן, ויחזיר רשימה נקייה ומאורגנת.

4.4.2 התכנון: שלושה שלבים

הפרויקט יתבצע בשלושה שלבים פשוטים:

1. קריאת הרשימה הגולמית מקובץ טקסט.
2. שליחת הרשימה למודל llama3.2 עם פרומפט מדויק, המורה לו לקטלג, למיין ולהציג.
3. שמירת התוצאה המאורגנת לקובץ חדש.

4.4.3 הקוד המלא: $grocery_organizer.py$

```
import ollama

def load_grocery_list(file_path):
    """Load raw grocery list from a text file."""
    with open(file_path, 'r', encoding='utf-8') as f:
        return f.read().strip()

def organize_grocery_list(raw_list):
    """Send the raw list to Ollama and get organized output."""
    prompt = f"""
You are a grocery list organizer.
Given the following list of grocery items:

{raw_list}

Your task:
1. Categorize the items into groups: Dairy, Vegetables, Fruits, Other
.
2. Sort items alphabetically within each category.
3. Present the result in a clear, bulleted format.

Do not add explanations. Just output the organized list.
"""
```


© rD. maroY lageS - כל הזכויות שמורות

```python

    response = ollama.generate(
        model='llama3.2',
        prompt=prompt,
        options={'temperature': 0.2}
    )

    return response['response']

def save_organized_list(organized_list, output_path):
    """Save the organized list to a file."""
    with open(output_path, 'w', encoding='utf-8') as f:
        f.write(organized_list)

def main():
    # File paths
    input_file = 'grocery_raw.txt'
    output_file = 'grocery_organized.txt'

    # Step 1: Load raw list
    print("Loading raw grocery list...")
    raw_list = load_grocery_list(input_file)
    print(f"Raw list:\n{raw_list}\n")

    # Step 2: Organize with Ollama
    print("Organizing with Llama 3.2...")
    organized = organize_grocery_list(raw_list)
    print(f"Organized list:\n{organized}\n")

    # Step 3: Save result
    print(f"Saving to {output_file}...")
    save_organized_list(organized, output_file)
    print("Done!")

if __name__ == '__main__':
    main()
```

Listing 4.13: תלוכמ תמישר וגראמל אלמ דוק

כפי שמוצג ברשימה 31.4, הקוד הוא פשוט, קריא ומודולרי. הפונקציה load$_{g}rocery_{l}ist()$ קוראת את הקובץ הגולמי. הפונקציה organize$_{g}rocery_{l}ist()$ שולחת את הרשימה למודל עם פרומפט מדויק, המכוון את המודל למשימה ספציפית. הפונקציה save$_{o}rganized_{l}ist()$ שומרת את התוצאה.

שימו לב לפרומפט: הוא ברור, ממוקד, ומפורט. הוא מגדיר את המשימה (קטלוג, מיון, הצגה), את הקטגוריות הרצויות (מוצרי חלב, ירקות, פירות, אחר), ואת הפורמט (רשימה עם תבליטים). זהו דוגמה מצוינת להנדסת פרומפט: ככל שהפרומפט ברור יותר, התוצאה מדויקת יותר.

4.4.4 קובץ הקלט: grocery$_{r}aw.txt$

```
milk, bread, tomatoes, bananas, cheese, cucumbers, yogurt, apples,
eggs, onions
```

Listing 4.14: תלוכמ תמישרל ימלוג טלק ץבוק

4.4.5 הפלט המצופה: $\mathbf{grocery}_{organized.txt}$

לאחר הרצת הקוד, הפלט יהיה:

```
**Dairy**
- Cheese
- Eggs
- Milk
- Yogurt

**Vegetables**
- Cucumbers
- Onions
- Tomatoes

**Fruits**
- Apples
- Bananas

**Other**
- Bread
```

Listing 4.15: וגרואמ טלפ ץבוק

הרשימה כעת מסודרת, ברורה, וקלה לשימוש. כל קטגוריה מופיעה בנפרד, הפריטים ממוינים אלפביתית. זה פשוט, אך זה מוכיח את הכוח של אוטומציה מקומית: מידע לא מובנה הופך למידע מובנה, ללא עלות, ללא שליחת נתונים לענן, ובמהירות.

4.4.6 הרחבות אפשריות

פרויקט זה הוא רק נקודת פתיחה. ניתן להרחיב אותו בכמה כיוונים:

- **זיהוי כמויות:** אם הקלט כולל כמויות ("2 חלב, 005 גרם גבינה"), המודל יכול לחלץ אותן ולהציג אותן בנפרד.
- **המלצות תזונתיות:** המודל יכול להציע פריטים חסרים ("אין לך ירקות עלים, שקול להוסיף תרד").
- **אינטגרציה עם מסד נתונים:** ניתן לשמור רשימות קודמות, ולהשוות ביניהן כדי לזהות דפוסי קנייה.
- **ממשק משתמש גרפי:** ניתן לבנות ממשק Streamlit או Flask, שבו המשתמש מעלה רשימה ומקבל תוצאה מיידית.

4.4.7 ניתוח התוצאות: אוטונומיה מלאה

מה שמיוחד בפרויקט זה אינו המורכבות הטכנית – הקוד הוא פשוט. מה שמיוחד הוא העיקרון: אנו בונים אוטומציה קוגניטיבית מלאה, המתבצעת לחלוטין על המחשב המקומי. אין קריאות API לענן, אין עלויות שוטפות, אין דליפת מידע. הרשימה נשארת אצלנו, העיבוד מתבצע אצלנו, התוצאה נשמרת אצלנו. זוהי ריבונות דיגיטלית במיטבה.

© Dr. Yoram Segal - כל הזכויות שמורות

4.5 סיכום הפרק: מכלי למומחה

בפרק זה עברנו מסע מעשי מעולם התיאוריה לעולם הפרקטיקה. ראינו כיצד הModelfile הופך מודל גנרי למומחה ייעודי, בעל זהות קוגניטיבית מוגדרת. ראינו כיצד REST API פותח את השער לקישוריות אפליקטיבית, ומאפשר לכל תוכנה לדבר עם המודל. ראינו כיצד Python SDK מפשט את התהליך, והופך אותו לאלגנטי וקריא. ולבסוף, ראינו כיצד כל אלה מתמזגים לכדי פרויקט מעשי – מארגן רשימת מכולת – המוכיח כי אוטומציה קוגניטיבית מקומית היא לא רק אפשרית, אלא גם פשוטה, זולה, ופרטית.

העידן שבו מודלי שפה היו נחלתם הבלעדית של אימפריות הענן הסתיים. כעת, כל מפתח, כל חובב, כל ארגון, יכולים להריץ מודלים משלהם, לעצב אותם, לשלוט בהם, ולהטמיע אותם באפליקציות אמיתיות. זוהי הדמוקרטיזציה של הבינה המלאכותית במלוא המובן.

בפרק הבא נצלול לעומק הבעיה המהותית של מודלי שפה: ההלוצינציה. נבין מדוע מודלים, על אף עוצמתם, נוטים לשקר בביטחון, ונראה כיצד RAG מציע את הפתרון המהפכני לבעיה זו.

© Dr. Yoram Segal - כל הזכויות שמורות

פרק 5

משבר ההזיה והצורך בזיכרון קונטקסטואלי

RAG-ו Ollama

ריבונות דיגיטלית בעידן הבינה המלאכותית

Author: Dr. Yoram Segal

All rights reserved

11/2025

תוכן העניינים

פרק 5

משבר ההזיה והצורך בזיכרון קונטקסטואלי

5.1 הנבואה השבורה

בראשית המאה העשרים ואחת, האנושות חוותה רגע של אופטימיות טכנולוגית חסרת תקדים. מודלי השפה הגדולים, ה-Large Language Models או LLMs, הבטיחו לנו עולם חדש שבו המכונות לא רק מבינות את שפתנו, אלא גם מסוגלות לחשוב, ליצור ולייעץ. הם הציגו לעצמם כיועצים דיגיטליים, כעוזרים חכמים שיכולים לענות על כל שאלה, לכתוב כל טקסט ולפתור כל בעיה. אך מתחת לפני השטח הזוהר הזה, התגלתה אמת מטרידה: המכונות הללו, על אף עוצמתן הסטטיסטית המדהימה, הן שקרניות מוכשרות.

התופעה שכינינו Hallucination – הזיה או הלוצינציה – אינה באגליטציה bug שניתן לתקן בקלות. היא מובנית בעצם טבען של מערכות אלו. מודל שפה אינו "יודע" דברים במובן האנושי של המילה. הוא אינו מחזיק בידיעות, באמיתות או בהבנה מושגית. במקום זאת, הוא מנבא, בכל רגע נתון, מהי המילה הבאה הסבירה ביותר בהתבסס על דפוסים סטטיסטיים שלמד ממיליארדי משפטים. כאשר שואלים אותו שאלה על נושא שאינו מופיע במאגר האימון שלו – נתון עדכני, רגולציה חדשה, מסמך פנימי של ארגון – המודל עדיין ימשיך לנבא מילים. הוא ימלא את החלל הריק הזה בטקסט שנשמע סמכותי, מדויק ומשכנע, גם אם כל מילה בו היא בדויה.

ניקח דוגמה קונקרטית. נניח שאתה עובד במחלקה המשפטית של חברה גדולה ושואל את המודל: "מהם הקנסות שקבע בנק ישראל על אי דיווח פיננסי לפי התקנה החדשה משנת 2024?" המודל, שלא ראה מעולם את התקנה הזו כי היא פורסמה אחרי תאריך חתך האימון שלו, עדיין יספק לך תשובה. הוא יכתוב: "על פי תקנה 14ב' לחוק הבנקאות המעודכן, הקנס עומד על 500.000 שקל לעבירה ראשונה ועד 000,000,2 שקל לעבירה חוזרת." התשובה נשמעת משכנעת. היא משתמשת בטרמינולוגיה משפטית נכונה, היא מספקת מספרים ספציפיים, והיא מתייחסת לחוק קיים. אך כל פרט בה הוא בדוי. אין תקנה כזו, אין מספרים כאלה, ואין קנסות כפי שתוארו. המודל פשוט ניחש, ואתה, כמשתמש, הולך להסתמך על מידע שקרי זה בדיווח חשוב.

המשבר הזה אינו עניין של נוחות או יעילות. הוא עניין קיומי עבור יישומי AI בתחומים קריטיים. ברפואה, מודל שממליץ על תרופה שגויה עלול לסכן חיים. במערכת המשפט, מודל שמצטט תקדים בדוי יכול להרוס תיק. בפיננסים, מודל שמספק נתונים כוזבים עלול לגרום להפסדים כלכליים עצומים. בכל אחד מהמקרים הללו, הבעיה אינה טכנית בלבד – היא אתית. כיצד נוכל לסמוך על מכונות שאינן מסוגלות להבחין בין אמת לשקר?

5.2 האמת הקבורה בנתונים

האירוניה הגדולה של עידן ה-AI היא זו: ארגונים מחזיקים בהרי זהב של מידע, אך אינם מסוגלים לגשת אליו בצורה אפקטיבית. כל חברה, מוסד או ארגון ממשלתי אוגר מאגרים עצומים של מסמכים – דוחות שנתיים, נהלי עבודה, מדיניות פנימית, תיעוד טכני, פרוטוקולים, מחקרים – שמרביתם נכתבים

פעם אחת ונשכחים לנצח. זהו "ידע מת": מידע קיים, מדויק ורלוונטי, אך בלתי נגיש למי שזקוק לו.

הסיבה לכך היא שהמידע הזה הוא לא מובנה. הוא לא נמצא במאגרי נתונים מסודרים עם שדות, טבלאות ואינדקסים. הוא קבור בקבצי PDF, במצגות PowerPoint, בדוא"ל, בתיקיות שותפות ובארכיונים דיגיטליים שאיש אינו פותח. כאשר עובד חדש מגיע לארגון ומנסה להבין "כיצד אנחנו מטפלים בבעיה X", הוא מוצא את עצמו נודד בין מסמכים, שואל עובדים ותיקים ומנסה לאסוף פיסות מידע פזורות. הידע קיים, אך הוא אינו חי.

מודלי השפה הגדולים הכלליים – ChatGPT, Claude, Gemini – אינם מכירים את הידע הפרטי הזה. הם אומנו על האינטרנט הפתוח, על ספרים, על ויקיפדיה, על מאמרים אקדמיים. הם יודעים הכול על היסטוריה כללית, על מדע פופולרי, על טכנולוגיות נפוצות. אך הם אינם יודעים דבר על הנוהל הספציפי של הארגון שלך לטיפול בתקלות, על המדיניות הפנימית שלך לאישור תקציבים, או על המחקר הקנייני שביצעת בשנה שעברה. עבור מודלים אלו, הידע הארגוני שלך הוא טרה אינקוגניטה – טריטוריה בלתי-ידועה.

וכך, אנחנו מוצאים את עצמנו במצב פרדוקסלי: מחזיקים בכלים חכמים ביותר שיצרה האנושות, אך הם חסרי תועלת כאשר מדובר בידע הספציפי והרלוונטי ביותר עבורנו. זה כמו לשכור את המומחה הטוב ביותר בעולם, רק כדי לגלות שהוא אינו מכיר את החברה שלך, את התעשייה שלך או את האתגרים הייחודיים שלך.

5.3 הפתרון הארכיטקטוני

וכאן נכנסת לתמונה הטכנולוגיה שמכונה Retrieval-Augmented Generation, או בקיצור RAG. זהו שמה הטכני, אך משמעותה הפילוסופית עמוקה בהרבה: היא הופכת את ה-AI ממנבא עיוור לעיתונאי חוקר. במקום לסמוך על זיכרון סטטיסטי בלבד, המערכת לומדת לחפש, לאחזר ולהסתמך על מסמכי מקור אמיתיים.

העיקרון המרכזי של RAG הוא פשוט ומהפכני כאחד: לפני שהמודל עונה על שאלה, הוא תחילה מחפש במאגר פרטי של מסמכים את המידע הרלוונטי ביותר לשאלה זו. רק לאחר שמצא את פיסות המידע המתאימות, הוא משתמש בהן כ"קונטקסט" – כהקשר – לבניית התשובה. התוצאה היא תשובה שאינה מבוססת על ניחושים סטטיסטיים, אלא על ראיות ממשיות.

נחזור לדוגמה שהזכרנו קודם. כעת, במקום לשאול מודל כללי על קנסות בנק ישראל, אנחנו בונים מערכת RAG שיש לה גישה למאגר של כל התקנות והחוזרים שפרסם בנק ישראל. כאשר אתה שואל "מהם הקנסות על אי דיווח?", המערכת תחילה מחפשת בתוך המסמכים האלה את הפסקאות הרלוונטיות. היא מוצאת את החוזר הספציפי משנת 2024, מחלצת את הפסקה שדנה בקנסות, ומעבירה אותה למודל השפה יחד עם השאלה שלך. המודל כעת רואה בדיוק את המידע הנכון, ומשתמש בו כדי לנסח תשובה מדויקת: "על פי חוזר בנק ישראל 4202-51, הקנס הבסיסי עומד על 300.000 שקל, עם אפשרות להכפלה במקרים חמורים." התשובה כעת אינה המצאה – היא ציטוט מבוסס של מידע אמיתי.

זהו "קרקוע עובדתי" – Factual Grounding. המונח, שנשמע טכני, מתאר למעשה תהליך פילוסופי עמוק: הפיכת הטקסט הגנרטיבי ממצוי חופשי בחלל הסטטיסטי לטקסט מעוגן במציאות התיעודית. זה כמו ההבדל בין עיתונאי שממציא עובדות לבין עיתונאי שמצטט מקורות. השניים עשויים לכתוב באותו סגנון, אך האמינות שלהם שונה לחלוטין.

RAG מפחית באופן דרמטי את הסיכון להזיות. המודל כבר אינו צריך לנחש או להמציא. הוא רואה בפניו את המידע הרלוונטי, ותפקידו הוא רק לארגן אותו, לסכם אותו ולהציג אותו בצורה קריאה. אם המידע לא קיים במאגר, המערכת יכולה להגיד זאת במפורש: "לא מצאתי מידע רלוונטי על נושא זה במסמכים שלך." זוהי כנות טכנולוגית – היכולת להודות באי-ידיעה במקום להמציא.

© Dr. Yoram Segal - כל הזכויות שמורות

5.4 מידע מת, זיכרון חי

בעבר, הידע הארגוני היה נחשב ל"נכס סטטי". מסמכים נכתבו, נשמרו בארכיון, והשימוש בהם היה מינימלי. חיפוש מידע היה תהליך ידני, מייגע ולעיתים חסר תוחלת. עובד שחיפש מידע היה צריך לדעת היכן לחפש, באיזה תיקייה, באיזה מסמך, ולעבור עשרות עמודים כדי למצוא את הפסקה הרלוונטית. במרבית המקרים, המידע פשוט לא נמצא, והארגון היה ממציא את הגלגל מחדש.

RAG הופכת את הידע הזה ל"זיכרון חי". במקום מאגר מסמכים פסיבי, אנחנו בונים מערכת שמסוגלת לענות על שאלות, להציע פתרונות ולהמליץ על פעולות מבוססות מידע היסטורי. זה כמו ההבדל בין ספרייה ללא ספרן לבין ספרייה עם ספרן מומחה. בשניהם המסמכים זהים, אך היכולת לגשת למידע ולהבין אותו שונה בתכלית.

היתרון התחרותי של RAG במנעד הארגוני הוא עצום. חברה שמצליחה להפוך את הידע הפנימי שלה לנגיש ושימושי זוכה ביכולת לקבל החלטות מהירות ומדויקות יותר. עובדים חדשים יכולים להתאקלם מהר יותר על ידי שאילת שאלות למערכת ה-RAG ולמידה ממסמכי הארגון. צוותים יכולים להימנע מטעויות חוזרות על ידי חיפוש מקרים קודמים והבנה כיצד הם טופלו. מנהלים יכולים לקבל תובנות אסטרטגיות על ידי ניתוח דוחות היסטוריים והבנת מגמות ארוכות טווח.

והכול – כולו – מתבצע בצורה פרטית ומקומית. שילוב RAG עם Ollama פירושו שהמסמכים שלך לא נשלחים אל שרתי ענן זרים. הם נשארים על החומרה שלך, מתחת לשליטתך המלאה. הידע הארגוני – נכס אסטרטגי קריטי – נשמר בידיך בלבד.

5.5 ששת השלבים של זיכרון ויצירה

כעת, נצלול אל עומק הארכיטקטורה הטכנית של RAG. התהליך מתבצע בשישה שלבים ברורים, המחולקים לשני שלבים עיקריים: אינדקס וגנרציה. נבחן כל שלב ונבין כיצד הוא תורם למכלול הפתרון.

שלב 1: חלוקה – Chunking

המסמכים המקוריים שלנו – קבצי PDF, קבצי טקסט, מצגות – הם לרוב ארוכים מדי כדי להיכנס במלואם לאורך ההקשר של המודל. מודל שפה, כזכור, יכול לקרוא רק כמות מוגבלת של טקסט בכל פעם – זה מה שקראנו Context Length. אם נשלח אליו מסמך של 200 עמודים, הוא פשוט לא יוכל לעבד אותו. לכן, השלב הראשון הוא פיצול המסמכים לקטעים קטנים, שכל אחד מהם מכיל כמות סבירה של טקסט – בדרך כלל כ-1,200 עד 1,500 תווים.

אך הפיצול הזה אינו שרירותי. אנחנו לא פשוט חותכים את הטקסט באמצע משפט או באמצע פסקה. במקום זאת, אנחנו משתמשים בכלים חכמים כמו RecursiveCharacterTextSplitter מתוך ספריית LangChain, שמסוגלים לפצל את הטקסט בצורה סמנטית: בסיום פסקאות, בסיום משפטים, או בנקודות טבעיות של מעבר תוכני. בנוסף, אנחנו מוסיפים "חפיפה" – Overlap – בין הקטעים. כל קטע מכיל גם כמה משפטים מהקטע הקודם. כך, אנחנו מבטיחים שהקשר לא אובד: אם מידע חשוב נמצא על הגבול בין שני קטעים, החפיפה תבטיח שהוא יישמר בשניהם.

התוצאה של שלב זה היא קבוצה של "חתיכות" – Chunks – כל אחת בגודל סביר, כל אחת מכילה הקשר מספיק כדי להיות מובנת בעצמה.

שלב 2: הטמעה – Embedding

כעת אנחנו ניגשים לשלב המרתק ביותר: הפיכת מילים למספרים. המושג Embedding מתאר את התהליך שבו אנחנו ממירים קטע טקסט – משפט, פסקה או חתיכה שלמה – לייצוג מתמטי: וקטור של מספרים. וקטור זה אינו אקראי. הוא נוצר על ידי מודל הטמעה – Embedding Model – שהוכשר להבין את המשמעות הסמנטית של הטקסט ולתרגם אותה למספרים.

מודל ההטמעה לוקח את הטקסט "בנק ישראל קבע קנס על אי דיווח", ומייצר וקטור כמו זה:

```
[0.23, -0.45, 0.78, ..., 0.12]
```

וקטור זה מכיל מאות או אלפי מספרים, כל אחד מהם מייצג מימד אחר של המשמעות. חלק מהמימדים יכולים לייצג נושאים כמו "פיננסים", חלקים אחרים יכולים לייצג "רגולציה", ואחרים

© Dr. Yoram Segal - כל הזכויות שמורות

"משפט". הקסם האמיתי מתרחש כאשר אנחנו מבינים שוקטורים של טקסטים דומים סמנטית נמצאים קרוב זה לזה במרחב הוקטורי.

דמיינו מרחב תלת-מימדי שבו כל נקודה היא משפט. משפטים שעוסקים בנושא דומה יהיו קרובים זה לזה, בעוד משפטים על נושאים שונים יהיו רחוקים. זה בדיוק מה שקורה במרחב הוקטורי, רק במקום שלושה מימדים, יש לנו מאות או אלפי מימדים. המרחק בין וקטורים – שמכונה "דמיון סמנטי" – מודד כמה דומים שני קטעי טקסט במשמעותם, גם אם הם לא משתמשים באותן מילים.

עבור מערכת RAG מקומית, אנחנו משתמשים במודל הטמעה מקומי, כמו nomic-embed-text, שרץ באמצעות Ollama. זהו מודל קל משקל, יעיל, ומסוגל לייצר הטמעות איכותיות ללא צורך בשרת חיצוני.

שלב 3: אחסון – Vector Store

לאחר שכל החתיכות שלנו הפכו לוקטורים, אנחנו צריכים מקום לשמור אותם. כאן נכנס לתמונה מושג חדש: מאגר וקטורי – Vector Database. זהו סוג מיוחד של מסד נתונים שתוכנן במיוחד לאחסון וקטורים ולביצוע חיפושים מהירים של דמיון.

מאגר וקטורי כמו Chroma DB מאחסן שני דברים עבור כל חתיכה: את הוקטור עצמו, ואת הטקסט המקורי שממנו הוא נוצר. כך, כאשר נחפש וקטור דומה, נוכל לאחזר גם את הטקסט המקורי ולהציג אותו למודל השפה.

Chroma DB הוא פתרון אידיאלי למערכות מקומיות: הוא קל משקל, אינו דורש שרת נפרד, ופועל בתוך האפליקציה שלנו. הוא מסוגל לאחסן אלפי או מיליוני וקטורים, ולחפש בהם בזמן שבריר שניה.

בסיום שלב זה, יש לנו "אינדקס" מלא: כל הידע שלנו, כל המסמכים שלנו, מיוצגים כוקטורים במאגר, מוכנים להיות מאוחזרים בכל רגע.

שלב 4: שאילתה והטמעה – Query Embedding

כעת אנחנו עוברים לשלב השני: אחזור וגנרציה. המשתמש שואל שאלה: "מהם הקנסות על אי דיווח?". המערכת לוקחת את השאלה הזו ועושה לה את אותו תהליך הטמעה שעשינו לחתיכות המסמכים. השאלה הופכת אף היא לוקטור.

למה? כי עכשיו אנחנו יכולים להשוות את וקטור השאלה לוקטורים של כל החתיכות במאגר. השאלה והתשובות חיות עכשיו באותו מרחב מתמטי, ואנחנו יכולים למדוד את הקרבה ביניהם.

שלב 5: חיפוש דמיון – Similarity Search

זהו הלב הפועם של RAG. המערכת משווה את וקטור השאלה לכל הוקטורים במאגר, ומחשבת את הדמיון ביניהם. היא מזהה את החתיכות שהן הכי קרובות – כלומר, הכי דומות סמנטית – לשאלה שהמשתמש שאל.

למשל, אם השאלה היא "מהם הקנסות על אי דיווח?", המערכת תמצא חתיכות שמדברות על קנסות, על דיווחים פיננסיים, על רגולציות של בנק ישראל. היא לא מחפשת התאמה מילה במילה – היא מחפשת התאמה משמעותית. כך, גם אם המסמך המקורי כתוב "סנקציות בגין אי מילוי חובות דיווח", המערכת תזהה שזה רלוונטי לשאלה, כי משמעות הטקסטים דומה.

החיפוש מחזיר את, נניח, שלוש עד חמש החתיכות הכי רלוונטיות. אלו הן פיסות הטקסט המקוריות – לא הוקטורים, אלא הטקסט עצמו – שיועברו למודל השפה.

שלב 6: גנרציה – Generation

השלב הסופי: המערכת לוקחת את החתיכות שאוחזרו ואת שאלת המשתמש, ומרכיבה "פרומפט" – הנחיה – למודל השפה. הפרומפט הזה נראה בערך כך:

```
Answer the question based ONLY on the following context:

Context:
[Chunk 1 from document]
[Chunk 2 from document]
[Chunk 3 from document]

Question: What are the penalties for non-reporting?

```

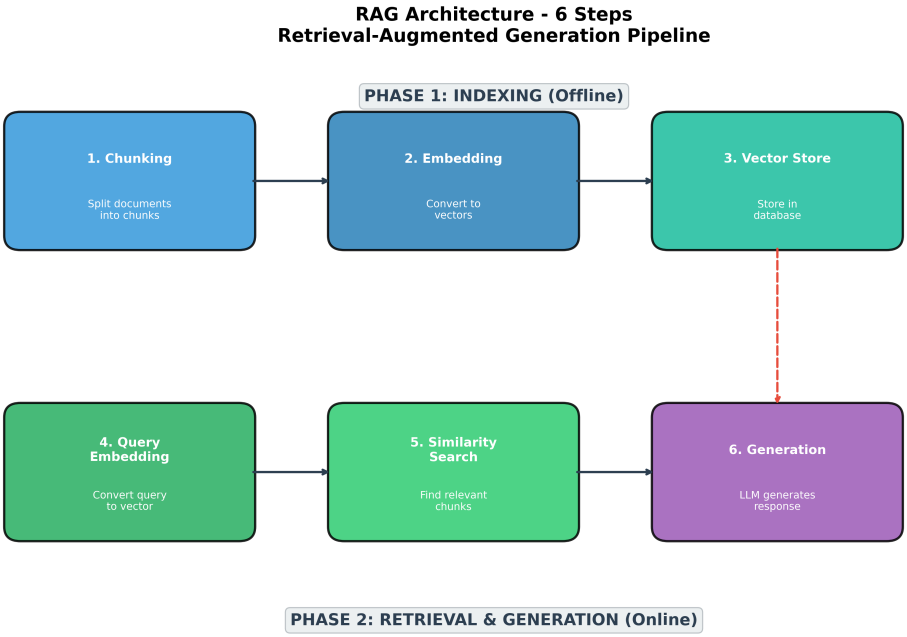

© Dr. Yossi Malz - כל הזכויות שמורות

```
Answer:
```

המודל – במקרה שלנו, Llama 3.2 המקומי – קורא את הקונטקסט, מבין את השאלה, וכותב תשובה המבוססת אך ורק על המידע שסופק לו. ההוראה "txetnoc gniwollof eht no YLNO desab" היא קריטית: היא מאלצת את המודל להישאר בגבולות המידע שאוחזר, ולא להמציא דברים חדשים.

התוצאה היא תשובה מדויקת, מבוססת-ראיות, ומקורקעת במסמכים האמיתיים של הארגון.

איור 5.1 ממחיש את ששת השלבים הללו כזרימה רציפה, החל מהמסמך המקורי ועד לתשובה הסופית שמקבל המשתמש.



איור 5.1: ארכיטקטורת RAG בששת השלבים: מהמסמך המקורי דרך אינדקס וקטורי ועד לתשובה מקורקעת

התרשים מראה כיצד כל שלב תורם למכלול: החלוקה מאפשרת עיבוד של מסמכים גדולים, ההטמעה הופכת משמעות לקואורדינטות, האחסון מאפשר חיפוש מהיר, השאילתה יוצרת גשר בין המשתמש למאגר, החיפוש מזהה את המידע הרלוונטי, והגנרציה מרכיבה הכול לתשובה קריאה ושימושית.

5.6 כשלון וישועה: השוואת LLM לעומת RAG

כדי להבין באמת את ההבדל המהותי בין מודל שפה כללי לבין מערכת RAG, נבחן שני תרחישים קונקרטיים.

תרחיש הכשל: מודל כללי ללא GAR

אתה עובד בחברת סטארט-אפ ישראלית בתחום הפינטק. אתה צריך להבין מהן חובות הדיווח שלך כלפי רשות המסים על עסקאות קריפטו. אתה פונה ל-ChatGPT ושואל: "מהן חובות הדיווח לרשות המסים בישראל על עסקאות קריפטוגרפיה?"

המודל, שאומן על מידע כללי מהאינטרנט, משיב:

> "שיגהל שרדנ, תונכדועמה תונקתה יפ לע .ביסמה תושרל יתנש חווידב תובייח לארשיב היפרגוטפירק תואקסע
> יא .הנשב ח"ש 10,000 לעמ ביחוור לע הלח חווידה תבוח .הנש לכ לש יאמב 31 דיראת דע 'א1234 ספוט
> 50 דע לש סנקל ליבוהל לולע חוויד

© Dr. Yoram Segal - כל הזכויות שמורות

התשובה נשמעת מקצועית ומפורטת. יש בה תאריך ספציפי, מספר טופס, סף כספי וקנס. הבעיה? כל פרט בה הוא בדוי. אין טופס בשם 1,234א', התאריכים שגויים, הסף הכספי לא קיים, והקנס מומצא. המודל ניחש, וייצר מידע שנשמע משכנע אך חסר כל בסיס במציאות. אם תסתמך על מידע זה בדיווח שלך, אתה עלול להפר את החוק, להימצא בסיכון משפטי ולחשוף את החברה לקנסות אמיתיים.

תרחיש ההצלחה: מערכת GAR עם מסמכי רשות המסים

כעת, דמיינו שבנית מערכת RAG שיש לה גישה לכל החוזרים והתקנות של רשות המסים בנוגע לקריפטו. המערכת אינדקסה את כל המסמכים הרלוונטיים, כולל החוזר המעודכן משנת 2024.

אתה שואל את אותה שאלה: "מהן חובות הדיווח לרשות המסים בישראל על עסקאות קריפטוגרפיה?"

המערכת מבצעת חיפוש דמיון, מוצאת את החוזר הרלוונטי, מאחזרת את הפסקאות שדנות בדיווח קריפטו, ומעבירה אותן למודל המקומי. המודל כעת רואה את המידע האמיתי:

> Context: "תבוח .סמ יכרוצל 'סכנ' תווהמ ס־יילטיגיד תועבטמב תואקסע ,5/2024 הסנכה סמ רזוח יפ לע' ו חפסנ תועצמאב עצבתמ חווידה .ילמינימ ָפס אלל ,ילטיגיד סכנ תריכממ עבונה וה חוור לכ לע הלח חווידה יפל תילילפ הריבע הווהמ חוויד יא .(סמ ָקעוי דרד שגומ סא) יאמב 31 וא לירפאב 30 סויל דע ,יתנשה ספוטל ".הסנכה סמ תדוקפל 220 ָפיעס

המודל מרכיב תשובה:

> "ילמינימ ָפס אלל ָוה חוור לכ לע חווידב תוביוחמ היפרגוטפירק תואקסע ,5/2024 הסנכה סמ רזוח יפ לע' .(סמ ָקעוי דרד שגומ סא יאמב 31 וא) לירפאב 30 סויל דע ,יתנשה ספוטל 'ו חפסנ תועצמאב עצבתמ חווידה ".תילילפ הריבע הווהמ חוויד יא

התשובה כעת מדויקת, מבוססת-מקור, וניתן לאמת אותה. אם תרצה, תוכל לבקש מהמערכת גם את הציטוט המדויק או את מספר החוזר למעקב. זוהי תשובה שעליה ניתן לסמוך בהחלטות עסקיות קריטיות.

ההבדל בין שני התרחישים הוא ההבדל בין המצאה מסוכנת לבין ידע אמין. RAG הופכת את המודל ממנבא עיוור לחוקר מושכל, מבעל דעה לבעל ראיות.

5.7 הקונטקסט כזהות

יש משמעות פילוסופית עמוקה בארכיטקטורה של RAG. כאשר אנחנו בונים מערכת שמסתמכת על מאגר מסמכים פרטי, אנחנו למעשה מעניקים למודל השפה "זהות ארגונית". המודל הכללי – ChatGPT, Claude – הוא אדם אוניברסלי, שיודע הכול על העולם אך אינו יודע דבר עליך. המודל המקומי שלך עם RAG הוא מומחה ייעודי, שיודע הכול על הארגון שלך, על הנהלים שלך, על ההיסטוריה שלך.

זוהי "ריבונות קונטקסטואלית" – היכולת להגדיר בעצמך מהו ההקשר שממנו פועל ה-AI שלך. במקום להיות תלוי בידע הכללי של ענקיות הענן, אתה בונה בסיס ידע מותאם אישית, שמשקף את הייחודיות של המקום שבו אתה עובד.

דמיינו מערכת RAG בבית חולים. המאגר שלה כולל את כל הפרוטוקולים הרפואיים של בית החולים, את המחקרים שביצעו הרופאים, את ההנחיות הקליניות המעודכנות. רופא שפונה למערכת ושואל "מהו הפרוטוקול שלנו לטיפול בזיהום דם?" מקבל תשובה שמבוססת בדיוק על הנהלים הספציפיים של בית החולים הזה, לא על מידע כללי מהאינטרנט. זוהי יכולת שמצילה חיים.

או דמיינו מערכת RAG במשרד עורכי דין. המאגר שלה כולל את כל התקדימים המשפטיים שהמשרד טיפל בהם, את הפסיקות הרלוונטיות, את הסיכומים המשפטיים. עורך דין שמכין תיק יכול לשאול "האם טיפלנו בעבר בתיק דומה?" ולקבל תשובה מיידית עם הפניה לתיקים קודמים, לאסטרטגיות שעבדו, ולטיעונים שהצליחו. זוהי יכולת שמשפרת את איכות הייצוג המשפטי.

הקונטקסט הוא הזהות. RAG מאפשרת לכל ארגון, לכל צוות, ולכל אדם לבנות AI שמשקף את הידע הייחודי שלו. זוהי דמוקרטיזציה של הבינה המלאכותית ברמה חדשה: לא רק הכוח לחשב, אלא הכוח להגדיר את ההקשר שממנו נובע החשיבה.

© Dr. Yoram Segal - כל הזכויות שמורות

5.8 הדרך אל האמינות

RAG אינה פתרון מושלם. היא אינה מבטלת לחלוטין את האפשרות להזיות – המודל עדיין יכול לטעות בפרשנות של המסמכים, או לארגן את המידע בצורה לא מדויקת. אך היא מפחיתה את הסיכון באופן דרמטי. היא הופכת את ההזיה ממצב ברירת מחדל למצב חריג. היא מעבירה את הנטל מהמודל – "נחש בעצמך" – למסמכים – "קרא ולמד".

בשילוב עם Ollama, RAG מספקת את החבילה השלמה: חירות חישובית, אמינות קונטקסטואלית ופרטיות מלאה. אתה כבר לא תלוי בשרתי ענן, אתה כבר לא מסתמך על ידע כללי, ואתה כבר לא חושף את המסמכים הרגישים שלך לגופים חיצוניים. אתה בונה מערכת שהיא שלך במלואה: הנתונים שלך, המודל שלך, הזיכרון שלך.

זוהי המהפכה האמיתית של RAG: היא הופכת את ה-AI מכלי כללי וחשוד למומחה אישי ואמין. היא הופכת את הידע הארגוני מנכס קבור לזיכרון חי. והיא הופכת אותנו, המשתמשים, מצרכנים פסיביים של שירותי ענן לבעלים ריבוניים של הטכנולוגיה שלנו.

בפרקים הבאים, נעבור מהתיאוריה לפרקטיקה. נבנה צעד אחר צעד מערכת RAG מקומית שלמה: נלמד כיצד לפצל מסמכים, כיצד ליצור הטמעות, כיצד לאחסן אותן במאגר וקטורי, וכיצד לבצע חיפוש ואחזור יעילים. נראה כיצד הארכיטקטורה התיאורטית שתיארנו כאן הופכת לקוד פועל, לאפליקציה ממשית, ולפתרון עסקי שניתן להטמיע בכל ארגון.

הדרך אל האמינות מתחילה בהבנה שהבעיה קיימת. היא ממשיכה בבחירה של הכלים הנכונים. והיא מגיעה לשיאה ביישום מעשי של הפתרון. אנחנו כבר הבנו את הבעיה. אנחנו כבר בחרנו את הכלים. כעת, הגיע הזמן לבנות.

© Dr. Yoram Segal - כל הזכויות שמורות

פרק 6

בניית ספריית הזיכרון: האינדקס התיאורטי והמעשי

RAG-ו Ollama

ריבונות דיגיטלית בעידן הבינה המלאכותית

Author: Dr. Yoram Segal

All rights reserved

11/2025

תוכן העניינים

פרק 6

בניית ספריית הזיכרון: האינדקס התיאורטי והמעשי

בשנת 2023, כאשר מערכות RAG החלו להפוך לסטנדרט בעולם הבינה המלאכותית, התגלתה אמת פשוטה אך מהפכנית: זיכרון ללא ארגון הוא כאוטי כמו ספרייה ללא מפתח. הידע הגולמי, הנתונים הלא מובנים, המסמכים השוכבים בשקט על הדיסק הקשיח של המחשב שלנו - כולם חסרי תועלת אם אין דרך יעילה למצוא אותם בזמן האמת. זהו בדיוק האתגר שעומד בבסיס בניית מערכת RAG מקומית: הפיכת הידע הפרטי שלנו לזיכרון חי, נגיש ופעיל.

התהליך של בניית ספריית הזיכרון, או במילים טכניות יותר - יצירת האינדקס הוקטורי, הוא היסוד האמיתי של כל מערכת RAG מוצלחת. ללא אינדקס איכותי, המודל שלנו יהיה כעיוור המגשש בחושך, מחפש אחר תשובות שהן ממש מתחת לאפו אך בלתי נגישות. מצד שני, עם אינדקס מובנה היטב, אנו הופכים את מודל השפה המקומי שלנו למומחה תחום אמיתי, המסוגל לענות על שאלות מורכבות תוך שניות ספורות, כל זאת תוך הסתמכות בלעדית על המסמכים הפרטיים שלנו.

6.1 מסע האינדקס: ארבעת השלבים המכריעים

כאשר אנו מתבוננים בתהליך בניית האינדקס הוקטורי, אנו רואים ארכיטקטורה בעלת ארבעה שלבים קריטיים. כל שלב ושלב הוא חוליה הכרחית בשרשרת, וחולשה באחד מהם תפגע באיכות המערכת כולה. השלבים הם: טעינת המסמכים, פיצול הטקסט לחלקים, המרה לייצוג וקטורי, ולבסוף אחסון במאגר מיוחד. התהליך הזה, אף שנשמע טכני ויבש, מייצג למעשה שינוי יסודי באופן שבו אנו מארגנים ומנגישים מידע בעידן הדיגיטלי.

טעינת המסמכים היא השלב הראשון והפשוט ביותר מבחינה קונספטואלית. אנו לוקחים את המסמכים שלנו - בין אם הם קבצי PDF, קבצי טקסט פשוטים, או אפילו מסמכי Word - וקוראים את תוכנם אל תוך זיכרון התוכנית. אך הפשטות הזו מטעה. המסמכים הריאליים שלנו הם לרוב עמודים ועמודים של טקסט רצוף, ואורכם עולה בהרבה על יכולת הקליטה של כל מודל שפה, גם המתקדמים ביותר. מודל כמו Llama 3.2 עשוי להיות בעל אורך קשר של 131,072 tokens, מספר מרשים ללא ספק, אך גם הוא מוגבל. מסמך רגולטורי ממוצע, דוח שנתי של חברה, או אפילו מאמר אקדמי מקיף - כולם עלולים לחרוג בקלות מהגבול הזה.

כאן נכנס לתמונה השלב השני, ואולי החשוב ביותר: פיצול הטקסט. התהליך הזה, המכונה בשפה המקצועית Chunking, אינו פשוט חיתוך מכני של הטקסט למקטעים. זוהי אמנות עדינה של שמירה על הקשר סמנטי תוך יצירת יחידות טקסט עצמאיות וניתנות לניהול. כל חלק שנוצר צריך להיות גדול מספיק כדי להכיל הקשר מלא ומובן, אך קטן מספיק כדי להתאים לאורך הקשר של המודל ולהיות רלוונטי לשאילתא ספציפית. זהו איזון דק בין קיצור יתר, שיגרום לאובדן הקשר, לבין הרחבה יתרה, שתפגע ברלוונטיות.

לאחר הפיצול מגיע השלב השלישי, הטרנספורמציה המהפכנית: המרת הטקסט לייצוג וקטורי. כאן קורה הקסם האמיתי. כל חלק טקסט, שהוא בבסיסו רצף של מילים ותווים, עובר דרך מודל הטמעה

מיוחד - Embedding Model - שממיר אותו לסדרת מספרים, וקטור במרחב רב-מימדי. הרעיון הבסיסי הוא פשוט אך עמוק: טקסטים בעלי משמעות דומה יקבלו וקטורים קרובים במרחב הוקטורי. שני משפטים המדברים על אותו נושא, גם אם הם משתמשים במילים שונות לחלוטין, ימוקמו זה ליד זה בחלל המתמטי הזה. זהו ייצוג סמנטי של השפה, שפה שהמחשב מבין ומסוגל לעבוד איתה ביעילות.

השלב הרביעי והאחרון הוא האחסון. הוקטורים שנוצרו, יחד עם הטקסט המקורי המקביל להם, נשמרים במאגר נתונים מיוחד - Vector Database או Vector Store. זהו לא מאגר נתונים רגיל. מאגר זה מיועד במיוחד לבצע חיפושים מהירים של דמיון בין וקטורים. כאשר נגיע לשלב האחזור, נקליט את שאילתת המשתמש, נהפוך אותה היא עצמה לוקטור, ונבקש מהמאגר למצוא את הוקטורים הקרובים ביותר אליה - כלומר, את פיסות המידע הרלוונטיות ביותר לשאלה שנשאלה.

התהליך המלא, מטעינה ועד אחסון, הוא הליבה הטכנית של מערכת RAG. הוא הופך נתונים לא מובנים לזיכרון מובנה, ידע פסיבי לכוח פעיל. וכעת, לאחר שהבנו את העקרונות, הגיע הזמן לצלול אל תוך הפרקטיקה - אל הקוד, הכלים והטכניקות שיאפשרו לנו לבנות את ספריית הזיכרון שלנו במו ידינו.

6.2 פיצול הטקסט: אמנות החלוקה והשימור

הבעיה היסודית של פיצול טקסט היא פרדוקסלית במהותה. מצד אחד, אנו רוצים חלקים קטנים ומדויקים שיתאימו במדויק לשאילתה של המשתמש. מצד שני, אנו זקוקים להקשר רחב מספיק כדי שהמידע יהיה מובן ושלם. אם נפצל את הטקסט באופן גס מדי, כל חתיכה תהיה ארוכה ולא ממוקדת, מה שיפגע ביכולת החיפוש למצוא את המידע הרלוונטי באמת. אך אם נפצל יותר מדי, נאבד את ההקשר הסובב, ופיסות המידע יהיו חסרות משמעות.

בשנים האחרונות צמחו מספר אסטרטגיות פיצול. השיטה הפשוטה ביותר היא פיצול לפי מספר תווים קבוע - למשל, כל 500 תווים הופכים לחלק נפרד. שיטה זו פשוטה ליישום, אך היא עיוורת לחלוטין למבנה הטקסט. היא עלולה לחתוך משפט באמצע, לפצל פסקה באופן לא הגיוני, ולהרוס את הזרימה הטבעית של הטקסט. שיטה מתקדמת יותר היא פיצול לפי משפטים או פסקאות - גישה שמשמרת טוב יותר את היחידות הסמנטיות הטבעיות של השפה. אך גם כאן יש בעיה: משפט בודד עשוי להיות קצר מדי ולא להכיל הקשר מספיק, בעוד פסקה ארוכה עלולה להיות רחבה מדי ולא ממוקדת.

הפתרון המודרני והאלגנטי ביותר מגיע מתוך הספרייה LangChain, אחת מהספריות המובילות לבניית יישומי RAG. הכלי המרכזי שלה לפיצול טקסט הוא RecursiveCharacterTextSplitter, ושמו מסגיר את הגישה הייחודית שלו. זהו מפצל רקורסיבי, שמנסה לפצל את הטקסט תוך שמירה על המבנה הטבעי ככל האפשר. הוא מחפש תחילה מפרידים "טבעיים" - כמו סימני פסקה חזקים, שורות ריקות, או שינויים של פסקאות - ורק אם לא נמצא מפריד כזה בטווח הרצוי, הוא עובר לפיצול מכני לפי תווים.

הקסם האמיתי של RecursiveCharacterTextSplitter טמון בשני הפרמטרים המרכזיים שלו: chunk_size ו-chunk_overlap. הפרמטר הראשון, גודל החלק, קובע את האורך המקסימלי של כל מקטע. בפרקטיקה, גודל נפוץ הוא 1,000 עד 1,200 תווים - מספיק כדי להכיל פסקה או שתיים של טקסט, מה שמבטיח הקשר סביר. הפרמטר השני, החפיפה, הוא המפתח לשמירת הרצף הסמנטי. כאשר אנו מגדירים חפיפה של 200 עד 300 תווים, אנו מבטיחים שכל חתיכה של טקסט תכיל גם את המשפטים האחרונים של החתיכה הקודמת. זה אומר שאם מידע קריטי נמצא על גבול הפיצול, הוא לא יאבד - הוא יופיע בשתי החתיכות הסמוכות, ובכך נשמר ההקשר המלא.

כדי להמחיש את התהליך בקוד ממשי, נציג כעת את המימוש המלא של טעינת מסמך PDF ופיצולו באמצעות Python. הקוד הבא, כפי שניתן לראות באיור **??**, מדגים את השלבים המלאים: התקנת הספריות הנדרשות, טעינת קובץ ה-PDF, ופיצולו לחלקים בעלי גודל ממוצע של 1,200 תווים עם חפיפה של 300 תווים.

```
# Install required libraries
# pip install langchain langchain-community pypdf

```


© Dr. Yoram Segal - כל הזכויות שמורות

```python
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter

# Step 1: Load PDF document
pdf_path = "path/to/your/document.pdf"
loader = PyPDFLoader(pdf_path)

# Load all pages from file
pages = loader.load()

print(f"Document loaded: {len(pages)} pages")
print(f"First page content preview:\n{pages[0].page_content[:500]}...")

# Step 2: Create text splitter with custom parameters
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1200,        # Maximum chunk size (characters)
    chunk_overlap=300,      # Overlap between chunks (characters)
    length_function=len,    # Function for measuring length
    separators=["\n\n", "\n", " ", ""]  # Priority order for separators
)

# Step 3: Split full document into chunks
chunks = text_splitter.split_documents(pages)

print(f"\nDocument split into {len(chunks)} chunks")
print(f"Average chunk size: {sum(len(chunk.page_content) for chunk in
chunks) / len(chunks):.0f} characters")

# Display example of first chunk
print(f"\n--- First Chunk ---")
print(f"Content: {chunks[0].page_content[:300]}...")
print(f"Metadata: {chunks[0].metadata}")
print(f"Length: {len(chunks[0].page_content)} characters")

# Save chunks to file (optional)
import json

chunks_data = [
    {
        "content": chunk.page_content,
        "metadata": chunk.metadata,
        "length": len(chunk.page_content)
    }
    for chunk in chunks
]

with open("chunks_output.json", "w", encoding="utf-8") as f:
    json.dump(chunks_data, f, ensure_ascii=False, indent=2)

print("\nChunks saved to chunks_output.json")
```

Listing 6.1: LangChain תועצמאב PDF דמסמ לוציפו הניעט

הקוד שלעיל, כפי שמוצג ברשימה 1.6, מבצע את התהליך המלא של טעינה ופיצול. תחילה אנו מייבאים את הכלים הנדרשים מתוך LangChain. לאחר מכן, אנו משתמשים ב-PyPDFLoader כדי לקרוא את קובץ ה-PDF ולהמיר אותו לרשימה של אובייקטי מסמך, כאשר כל עמוד הופך לאובייקט נפרד. השלב החשוב ביותר הוא יצירת המפצל - RecursiveCharacterTextSplitter - עם הפרמטרים המתאימים. אנו מגדירים גודל חלק של 1,200 תווים, חפיפה של 300 תווים, ורשימת מפרידים בסדר

© Dr. Yoram Segal - כל הזכויות שמורות

עדיפויות: תחילה פסקאות (\n\n), אחר כך שורות בודדות (\n), ואז רווחים, ורק בסוף - פיצול מכני לפי תווים.

לאחר הפיצול, אנו מקבלים רשימה של אובייקטי Document, כל אחד מכיל חלק מהטקסט המקורי יחד עם מטה-דאטה שמציינת את מקור החלק (מספר העמוד, שם הקובץ, וכו'). המטה-דאטה הזו תהיה קריטית בשלבים המאוחרים יותר, כשנרצה לעקוב אחר המקור המדויק של כל פיסת מידע שאוחזרה.

התוצאה של התהליך הזה היא לא רק אוסף של חתיכות טקסט, אלא מבנה סמנטי מאורגן שמשמר את ההקשר הטבעי של המסמך המקורי. כל חלק הוא יחידה עצמאית שניתן להבין אותה בפני עצמה, אך יחד הם יוצרים רצף רציף ומקושר של ידע. זהו הבסיס לכל מה שיבוא אחר כך.

6.3 השפה המהותית: המרת טקסט לוקטורים

לאחר שפיצלנו את המסמך שלנו לחלקים מנוהלים, אנו ניצבים בפני האתגר המרכזי של כל מערכת RAG: כיצד למצוא את פיסות המידע הרלוונטיות לשאילתה נתונה? החיפוש הטקסטואלי המסורתי, המבוסס על התאמת מילות מפתח, אינו מספיק. שאלה כמו "מהן המדיניות הפנימיות בנוגע לחופשות?" עשויה להיות רלוונטית למסמך המדבר על "הנחיות חג ונופש", למרות שאין התאמה מילולית ישירה. אנו זקוקים לשיטה שתבין את המשמעות העמוקה, הסמנטית, של הטקסט - ולא רק את המילים המדויקות.

כאן נכנסת לתמונה הטכנולוגיה של Embeddings, או בעברית - הטמעות. זוהי אחת מההמצאות המרתקות ביותר של עידן הבינה המלאכותית המודרני. הרעיון הבסיסי הוא פשוט אך מהפכני: כל פיסת טקסט - בין אם היא מילה בודדת, משפט, או פסקה שלמה - ניתן להמיר לייצוג מספרי בצורה של וקטור, רשימה של מספרים. אך זה לא סתם המרה שרירותית. הוקטורים הללו נוצרים באופן שמשמר את היחסים הסמנטיים בין הטקסטים. מילים בעלות משמעות דומה יקבלו וקטורים קרובים במרחב המספרי, וקטעי טקסט המדברים על אותו נושא ימוקמו זה ליד זה בחלל הווקטורי הרב-ממדי.

לדמיין זאת, בואו נחשוב על מרחב תלת-ממדי פשוט, כמו החדר שבו אנו יושבים. כל נקודה בחדר ניתנת לתיאור על ידי שלושה מספרים: x, y, z. נקודות קרובות במרחב הן קרובות מבחינת הקואורדינטות שלהן. עכשיו, דמיינו מרחב לא של שלושה ממדים, אלא של אלפים - או במקרה של מודלי Embedding מודרניים, מאות או אלפי ממדים. כל מילה, כל משפט, הופכים לנקודה במרחב הענק הזה. וההבטחה המדהימה של מודלי ההטמעה היא שהמרחק הגיאומטרי במרחב הזה משקף את הדמיון הסמנטי: משפטים על אותו נושא יהיו קרובים, אפילו אם הם לא חולקים מילה אחת משותפת.

הדרך שבה מודל ההטמעה לומד להמיר טקסט לוקטורים היא בעצמה סיפור מרתק. המודל עבר אימון על כמויות אדירות של טקסט, בתהליך שבו הוא למד לזהות דפוסים, קשרים ומבנים בשפה. הוא למד שהמילים "מלך" ו"מלכה" קשורות בצורה דומה למילים "גבר" ו"אישה", ולכן הוא יוצר וקטורים שמשמרים את היחס המתמטי הזה. הוא למד שמשפטים על טכנולוגיה יהיו קרובים למשפטים אחרים על טכנולוגיה, גם אם אחד מדבר על "מחשבים" והשני על "תוכנה". זו למידה עמוקה של מבני השפה והמשמעות.

בהקשר של RAG מקומי עם Ollama, אנו רוצים להשתמש במודל הטמעה שהוא בו זמנית חזק ומקומי. אחד המודלים המובילים לכך הוא nomic-embed-text, מודל הטמעה מתקדם שפותח על ידי חברת Nomic AI. מודל זה תומך באורך קשר מרשים של עד 8,192 tokens, מה שאומר שהוא יכול להטמיע קטעי טקסט ארוכים מאוד בבת אחת. ומה שהופך אותו למושלם עבורנו הוא שהוא זמין לחלוטין דרך Ollama, מה שאומר שנוכל להריץ אותו מקומית, ללא עלות, ותוך שמירה על פרטיות מלאה.

השימוש במודל ההטמעה דרך Ollama הוא פשוט להפליא. ספריית ה-Python של Ollama מספקת פונקציה ייעודית - embeddings() - שלוקחת טקסט ומחזירה את הייצוג הווקטורי שלו. הקוד הבא, כפי שמוצג ברשימה 2.6, מדגים את התהליך המלא של הטמעת קטעי טקסט באמצעות nomic-embed-text.

```
# Install Ollama library
# pip install ollama

```


© Dr. Yoram Segal - כל הזכויות שמורות

```python
import ollama
import numpy as np

# Step 1: Download embedding model (first time only)
# ollama pull nomic-embed-text

# Step 2: Embed a single sentence
text_sample = "Ollama is a tool for running large language models locally."

response = ollama.embeddings(
    model='nomic-embed-text',
    prompt=text_sample
)

embedding_vector = response['embedding']

print(f"Original text: {text_sample}")
print(f"Embedding vector length: {len(embedding_vector)} dimensions")
print(f"First 10 dimensions: {embedding_vector[:10]}")
print(f"Vector type: {type(embedding_vector)}")

# Step 3: Embed multiple text segments
texts = [
    "Machine learning is a subset of artificial intelligence.",
    "Deep learning uses neural networks with multiple layers.",
    "Natural language processing enables computers to understand text.",
    "The weather today is sunny and warm.",
    "I enjoy cooking pasta with tomato sauce."
]

embeddings = []
for text in texts:
    response = ollama.embeddings(
        model='nomic-embed-text',
        prompt=text
    )
    embeddings.append(response['embedding'])

print(f"\n--- Embedded {len(embeddings)} texts ---")

# Step 4: Calculate similarity between embeddings (Cosine Similarity)
def cosine_similarity(vec1, vec2):
    """Calculate cosine similarity between two vectors"""
    vec1_array = np.array(vec1)
    vec2_array = np.array(vec2)

    dot_product = np.dot(vec1_array, vec2_array)
    norm1 = np.linalg.norm(vec1_array)
    norm2 = np.linalg.norm(vec2_array)

    return dot_product / (norm1 * norm2)

# Compare similarity between sentences
print("\n--- Similarity Matrix ---")
print("Comparing semantic similarity between sentences:\n")

for i in range(len(texts)):
    for j in range(i + 1, len(texts)):
```


© Dr. Yaron Hadad - כל הזכויות שמורות

```
        similarity = cosine_similarity(embeddings[i], embeddings[j])
        print(f"Text {i+1} <-> Text {j+1}: {similarity:.4f}")
        print(f"  '{texts[i][:50]}...'")
        print(f"  '{texts[j][:50]}...'")
        print()

# Step 5: Find the closest sentence to query
query = "What is AI and machine learning?"
query_response = ollama.embeddings(
    model='nomic-embed-text',
    prompt=query
)
query_embedding = query_response['embedding']

print(f"\n--- Finding closest match to query ---")
print(f"Query: {query}\n")

similarities = []
for i, emb in enumerate(embeddings):
    sim = cosine_similarity(query_embedding, emb)
    similarities.append((i, sim))

# Sort by similarity (high to low)
similarities.sort(key=lambda x: x[1], reverse=True)

print("Top 3 most similar texts:")
for rank, (idx, sim) in enumerate(similarities[:3], 1):
    print(f"{rank}. Similarity: {sim:.4f} - {texts[idx]}")
```

Listing 6.2: תועצמאב טסקט תעמטה Ollama ו-nomic-embed-text

הקוד שלעיל, המוצג ברשימה 2.6, ממחיש את מלוא העוצמה של מודל ההטמעה. תחילה, אנו מטמיעים משפט בודד ורואים שהוא הופך לוקטור באורך מאות ממדים - בדרך כלל 768 או 1,024 ממדים, תלוי במודל. כל מספר בווקטור הזה מייצג תכונה סמנטית מסוימת שהמודל למד במהלך האימון שלו. אנו לא יודעים בדיוק מה כל ממד מייצג - זהו "ידע סמוי" של המודל - אך אנו יודעים שהייצוג הכולל משמר את המשמעות.

בהמשך, אנו מטמיעים חמישה משפטים שונים. שלושת המשפטים הראשונים עוסקים בנושאים קרובים - בינה מלאכותית, למידת מכונה, עיבוד שפה טבעית. שני המשפטים האחרונים עוסקים בנושאים שונים לחלוטין - מזג אוויר ובישול. כעת, אנו משתמשים במדד מתמטי קלאסי בשם Cosine Similarity - דמיון קוסינוס - כדי לחשב עד כמה הוקטורים דומים זה לזה. דמיון קוסינוס מודד את הזווית בין שני וקטורים במרחב הרב-ממדי. ערך של 1.0 אומר שהוקטורים זהים לחלוטין, ערך של 0.0 אומר שהם ניצבים זה לזה (אין ביניהם שום דמיון), וערך שלילי אומר שהם מנוגדים.

כשנריץ את הקוד, נגלה משהו מדהים: המשפטים על בינה מלאכותית ימדדו דמיון גבוה מאוד זה לזה (ערכים של 0.85 ומעלה), בעוד שהדמיון בינם לבין המשפט על מזג האוויר או בישול יהיה נמוך בהרבה (ערכים של 0.2 או פחות). המודל, ללא שהסברנו לו דבר, הבין את הקשר הסמנטי בין המושגים. זה בדיוק מה שאנו רוצים במערכת RAG: כשמשתמש שואל שאלה, אנו נהפוך את השאלה לוקטור, נחפש את הוקטורים הקרובים ביותר במאגר שלנו, ובכך נמצא את פיסות המידע הרלוונטיות ביותר - גם אם הן משתמשות במילים שונות לחלוטין מאלו שבשאלה.

בסוף הקוד, אנו מדגימים תרחיש מציאותי: שאילתת משתמש - "?gninrael enihcam dna IA si tahW" - מוטמעת, ואנו מחפשים את המשפט הקרוב ביותר אליה מתוך הרשימה שלנו. התוצאה, כמובן, תהיה המשפט הראשון - "ecnegilletni laicifitra fo tesbus a si gninrael enihcaM" - שיקבל את הציון הגבוה ביותר. זהו, בקיצור נמרץ, הליבה של כל מערכת RAG: המרת טקסט לוקטורים, חישוב דמיון, ואחזור המידע הרלוונטי ביותר.

6.4 מאגר הזיכרון הוקטורי: Chroma DB כבית לידע

לאחר שהטמענו את כל חלקי הטקסט שלנו לוקטורים, אנו עומדים בפני שאלה מעשית: היכן נאחסן את הוקטורים הללו, וכיצד נבצע חיפוש מהיר ויעיל ביניהם? אחסון הוקטורים כרשימה פשוטה בזיכרון הוא אפשרי למסמכים קטנים, אך כשמדובר במערכת אמיתית עם אלפי או עשרות אלפי מקטעי טקסט, הגישה הזו הופכת לא יעילה ואיטית. אנו זקוקים למאגר נתונים מיוחד, מותאם לאחסון וחיפוש של וקטורים - Vector Database.

מאגר נתונים וקטורי הוא לא כמו מאגר נתונים רלציוני מסורתי שמאחסן טבלאות של מספרים וטקסט. הוא מותאם במיוחד לאחסן וקטורים רב-ממדיים ולבצע עליהם חיפושי דמיון במהירות גבוהה. האלגוריתמים שמאחורי מאגרים כאלו מבוססים על טכניקות מתקדמות כמו HNSW (lacihcrareiH elbagivaN llamS dlroW) או Annoy (srobhgieN tseraeN etamixorppA), שמאפשרות למצוא את הוקטורים הקרובים ביותר לוקטור נתון במהירות, גם כשמדובר במיליוני וקטורים.

בעולם ה-RAG המקומי, אחד המאגרים הפופולריים והידידותיים ביותר הוא Chroma DB. מה שהופך את Chroma לבחירה מושלמת עבור יישומים מקומיים הוא הפשטות והקלילות שלו. Chroma הוא מאגר in-memory - כלומר, הוא פועל ישירות בזיכרון התוכנית - אך הוא גם יודע לשמור את הנתונים על הדיסק כדי שלא נצטרך לבנות את האינדקס מחדש בכל פעם. הוא לא דורש התקנה של שרת נפרד, לא דורש קונפיגורציה מסובכת, והוא משתלב באופן חלק עם LangChain - הספרייה שאנו משתמשים בה לבניית הפייפליין של RAG.

השימוש ב-Chroma DB יחד עם Ollama ו-LangChain הוא פשוט להפליא. הספרייה מספקת מעטפת נוחה שמבצעת את כל העבודה המורכבת מאחורי הקלעים. כל מה שאנחנו צריכים לעשות הוא לספק לה את חלקי הטקסט (שפיצלנו קודם), לציין איזה מודל הטמעה להשתמש, והיא תדאג ליצור את הוקטורים ולאחסן אותם במאגר. הקוד הבא, המוצג ברשימה 3.6, ממחיש את התהליך המלא של יצירת אינדקס וקטורי מקומי.

```python
# Install required libraries
# pip install langchain langchain-community langchain-chroma chromadb ollama pypdf

from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.embeddings import OllamaEmbeddings
from langchain_chroma import Chroma
import os

# Step 1: Load and split document (like previous example)
pdf_path = "path/to/your/document.pdf"
loader = PyPDFLoader(pdf_path)
pages = loader.load()

print(f"Loaded {len(pages)} pages from PDF")

# Step 2: Split text into chunks
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1200,
    chunk_overlap=300,
    length_function=len,
)

chunks = text_splitter.split_documents(pages)
print(f"Split into {len(chunks)} chunks")

# Step 3: Create local embedding model using Ollama
# Ensure nomic-embed-text model is installed: ollama pull nomic-embed-text

```


© rD. maroY lageS - כל הזכויות שמורות

```
embeddings_model = OllamaEmbeddings(
    model="nomic-embed-text",
    base_url="http://localhost:11434"  # Local Ollama server address
)

print("Embeddings model initialized: nomic-embed-text")

# Step 4: Create Chroma vector store
# Define directory to save store on disk
persist_directory = "./chroma_db_storage"

print(f"\nCreating Chroma vector store...")
print(f"Processing {len(chunks)} chunks...")

# Create store: Chroma will automatically embed all chunks
# and store them with their vectors
vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings_model,
    persist_directory=persist_directory,
    collection_name="my_document_collection"
)

print(f"Vector store created successfully!")
print(f"Stored in: {persist_directory}")
print(f"Total chunks indexed: {len(chunks)}")

# Step 5: Test vector search
query = "What are the key benefits of using local LLMs?"

print(f"\n--- Testing Vector Search ---")
print(f"Query: {query}")

# Simple search: returns 3 most similar documents
results = vectorstore.similarity_search(query, k=3)

print(f"\nFound {len(results)} relevant chunks:")
for i, doc in enumerate(results, 1):
    print(f"\n--- Result {i} ---")
    print(f"Content: {doc.page_content[:300]}...")
    print(f"Metadata: {doc.metadata}")

# Step 6: Search with similarity scores
print(f"\n--- Search with Similarity Scores ---")
results_with_scores = vectorstore.similarity_search_with_score(query, k=3)

for i, (doc, score) in enumerate(results_with_scores, 1):
    print(f"\n--- Result {i} (Score: {score:.4f}) ---")
    print(f"Content: {doc.page_content[:200]}...")

# Step 7: Save and reload store
print(f"\n--- Persistence Test ---")
print(f"Vector store is persisted to disk at: {persist_directory}")
print(f"You can reload it later without re-processing the documents.")

# Example of reloading existing store
def load_existing_vectorstore(persist_dir, embeddings_model):
    """Load existing vector store from disk"""
```

```
    vectorstore = Chroma(
        persist_directory=persist_dir,
        embedding_function=embeddings_model,
        collection_name="my_document_collection"
    )
    return vectorstore

# Can load store in next run:
# vectorstore = load_existing_vectorstore(persist_directory,
embeddings_model)

print("\n--- Indexing Complete ---")
print(f"Your local RAG knowledge base is ready!")
print(f"Total documents: {vectorstore._collection.count()}")
```

Listing 6.3: תריצי רגאמ רוטקו ירומקמ ימע Chroma DB ו-Ollama

הקוד המוצג ברשימה 3.6 הוא הלב הפועם של בניית מערכת RAG מקומית. בואו ננתח אותו שלב אחר שלב. תחילה, אנו טוענים ומפצלים את המסמך שלנו - תהליך שכבר הכרנו. לאחר מכן, בשלב הקריטי, אנו יוצרים אובייקט OllamaEmbeddings. אובייקט זה הוא המתווך בין הקוד שלנו לבין מודל ההטמעה שרץ באמצעות Ollama. אנו מציינים את שם המודל - nomic-embed-text - ואת כתובת השרת המקומי של Ollama, שהיא כברירת מחדל localhost:11434.

השלב הבא הוא הקסם האמיתי: יצירת מאגר ה-Chroma. אנו משתמשים בפונקציה Chroma.from_documents(), שמקבלת את רשימת ה-chunks שלנו, את מודל ההטמעה, ותיקייה לשמירת המאגר על הדיסק. מה שקורה מאחורי הקלעים הוא מרתק: Chroma לוקחת כל אחד מהחלקים, שולחת אותו למודל ההטמעה דרך Ollama, מקבלת בחזרה את הוקטור, ומאחסנת גם את הטקסט המקורי וגם את הוקטור במאגר הפנימי שלה. התהליך הזה עשוי לקחת מספר שניות או דקות, תלוי בגודל המסמך ובמהירות המחשב, אך הוא מתבצע רק פעם אחת. לאחר מכן, המאגר נשמר על הדיסק, ובהרצות הבאות נוכל פשוט לטעון אותו ללא צורך לעבד את המסמך מחדש.

לאחר שהמאגר נבנה, אנו יכולים להשתמש בו לחיפוש. הפונקציה similarity_search() היא הפשוטה ביותר: אנו מעבירים לה שאילתה טקסטואלית ומספר תוצאות רצוי (למשל, 3), והיא מחזירה את חלקי הטקסט הרלוונטיים ביותר. מה שקורה מאחורי הקלעים הוא שהשאילתה עצמה מוטמעת לוקטור, ואז מתבצע חיפוש דמיון במאגר למציאת הוקטורים הקרובים ביותר. הפונקציה similarity_search_with_score() מחזירה לנו גם את ציוני הדמיון, כך שנוכל לראות עד כמה כל תוצאה רלוונטית.

היתרון העצום של Chroma הוא שהוא דואג לכל המורכבות הטכנית. אנו לא צריכים לדאוג לאלגוריתמי חיפוש, לאופטימיזציה, או לניהול זיכרון. הוא מספק לנו ממשק פשוט וברור, ומאחוריו מתרחשת כל המאגיה המתמטית והאלגוריתמית שהופכת חיפוש וקטורי למהיר ויעיל.

6.5 טבלת רכיבים מקומיים: האקוסיסטם המלא

כעת, לאחר שעברנו על כל השלבים של בניית האינדקס - פיצול, הטמעה ואחסון - הגיע הזמן לסכם ולהציג את התמונה המלאה. מערכת RAG מקומית שלמה מורכבת מכמה רכיבים מרכזיים, כל אחד ממלא תפקיד קריטי בשרשרת. טבלה 6.1 מציגה את הרכיבים המרכזיים שאנו משתמשים בהם, את התפקיד של כל אחד, ואת היתרונות של השימוש המקומי בהם.

כפי שניתן לראות בטבלה 6.1, כל רכיב ממלא תפקיד ייחודי במערכת. מודל השפה - LLM - הוא המוח המרכזי שמייצר את התשובות. מודל ההטמעה ממיר את הטקסט שלנו, הן את המסמכים והן את השאילתות, לשפה הוקטורית שהמערכת מבינה. המאגר הוקטורי - Chroma DB - הוא הספרייה שבה מאוחסן הידע שלנו באופן מאורגן וניתן לחיפוש. ולבסוף, מסגרת העבודה - LangChain - היא המנצחת שמחברת את כל הרכיבים האלה לפייפליין אחד, עובד וחלק.

© Dr. Yoram Segal - כל הזכויות שמורות

טבלה 6.1: רכיבי מערכת RAG מקומית עם Ollama

יתרונות מקומיות	המודל המומלץ	תפקיד	רכיב
חינם, פרטי, מהיר, הפחתת Latency	Llama 3.2 3B או Llama 3.1 8B	עיבוד השאילתה, יצירת תשובה מבוססת-קונטקסט	**מודל שפה (LLM)**
שליטה מלאה על תהליך האינדקס, איכות גבוהה	nomic-embed-text (דרך Ollama)	המרת טקסט לייצוג וקטורי לצורך חיפוש דמיון סמנטי	**מודל הטמעה (Embedding)**
קל משקל, אין צורך בשרתי ענן או תשלום	Chroma DB (דרך LangChain)	אחסון וקטורים וטקסט מקורי, חיפוש דמיון מהיר ויעיל	**מאגר וקטורי (Vector Store)**
גמישות, שליטה מלאה על כל שלבי התהליך	LangChain Community	תזמור הפייפליין, ניהול שרשרת עיבוד	**מסגרת עבודה (Framework)**

היתרון המכריע של המערכת המקומית הזו הוא האוטונומיה המלאה. כל רכיב פועל על המחשב שלנו, ללא תלות בשירותי ענן חיצוניים. המשמעות היא שאין עלויות שוטפות מעבר לחשמל שהמחשב שלנו צורך. הנתונים שלנו, המסמכים הפרטיים שלנו, לעולם לא עוזבים את המכונה שלנו - הם לא נשלחים לשרת מרוחק, לא עוברים דרך רשת האינטרנט, ואף אחד מלבדנו לא יכול לגשת אליהם. הפרטיות היא מוחלטת. נוסף על כך, זמני התגובה קצרים במיוחד, כי כל החישוב מתבצע מקומית - אין latency של רשת, אין תורים בשרת מרוחק.

הארכיטקטורה הזו מייצגת את הדור החדש של יישומי בינה מלאכותית: מערכות שהן בו זמנית עוצמתיות, זולות, מהירות ופרטיות. זהו שילוב שלפני כמה שנים היה נחשב בלתי אפשרי. והכל בזכות הכלים הפתוחים והחופשיים שפיתחה הקהילה: Ollama, LangChain, Chroma, ומודלים פתוחים כמו Llama ו-Nomic Embed.

6.6 סיכום: הזיכרון המקומי כזכות יסוד

בסיום פרק זה, לאחר שעברנו את כל שלבי בניית האינדקס - מפיצול הטקסט דרך המרה לוקטורית ועד לאחסון במאגר - אנו עומדים בפני מסקנה עמוקה יותר מהטכנולוגיה עצמה. היכולת לבנות ספריית זיכרון מקומית, לארגן את הידע הפרטי שלנו באופן שהופך אותו לנגיש ושימושי, היא לא רק שאלה של נוחות או יעילות. זוהי שאלה של ריבונות דיגיטלית.

במשך שנים רבות, הידע הארגוני והאישי שלנו היה כמו ספרייה סגורה - קיים, אך לא נגיש. מסמכים שוכבים בתיקיות, קבצים מאוחסנים בכונן, אך אין דרך אמיתית לחלץ מהם מידע בזמן אמת בתגובה לשאלה ספציפית. הפתרון המסורתי היה להסתמך על שירותי ענן - להעלות את המסמכים שלנו לשרת מרוחק, לשלם עבור השירות, ולקוות שהפרטיות שלנו נשמרת. אך כעת, עם הכלים שהצגנו בפרק זה, האלטרנטיבה היא ברורה ומעשית: אנו יכולים לבנות את הכל בעצמנו, על המחשב שלנו, ללא תלות חיצונית.

התהליך של פיצול, הטמעה ואחסון אינו רק תהליך טכני. הוא מייצג את ההפיכה של נתונים פסיביים לזיכרון אקטיבי. המערכת שבנינו היא למעשה הרחבה של הזיכרון שלנו - היא מאפשרת לנו לזכור, לחפש, ולהבין כמויות אדירות של מידע בצורה שהמוח האנושי לבדו לא יכול. וכל זה תוך שמירה על השליטה המלאה שלנו על המידע, על התהליך, ועל התוצאות.

בפרק הבא, נעבור משלב בניית האינדקס לשלב השימוש בו - נראה כיצד לאחזר מידע מהמאגר הוקטורי ולשלב אותו בתהליך הגנרציה, כדי ליצור מערכת RAG מלאה ופועלת שמסוגלת לענות על שאלות מורכבות תוך הסתמכות בלעדית על המסמכים הפרטיים שלנו. זהו השלב שבו הזיכרון שבנינו מתעורר לחיים.

© Dr. Yoram Segal - כל הזכויות שמורות

פרק 7

ארכיטקטורת האחזור והיצירה: RAG בפועל

RAG-ו Ollama

ריבונות דיגיטלית בעידן הבינה המלאכותית

Author: Dr. Yoram Segal

All rights reserved

11/2025

תוכן העניינים

פרק 7

ארכיטקטורת האחזור והיצירה: RAG בפועל

כאשר עומדים בפני המשימה הארכיטקטורית של בניית מערכת RAG מקומית ומלאה, אנו למעשה בונים גשר בין שני עולמות: העולם הסטטיסטי והמילולי של מודלי השפה הגדולים, והעולם הממשי, העובדתי והמדויק של הנתונים הארגוניים שלנו. בפרק הקודם בנינו את ספריית הזיכרון --- את האינדקס הווקטורי שהופך מסמכים סטטיים לזיכרון חי וניתן לאחזור. כעת הגיע הזמן להשלים את המעגל: לבנות את שרשרת האחזור והיצירה, את הצינור המלא שמקשר בין שאלת המשתמש לבין התשובה המקורקעת בראיות. זהו המעבר מהתיאוריה אל הפרקטיקה המלאה.

תהליך זה אינו רק טכני; הוא פילוסופי. כאשר אנו מעצבים את ארכיטקטורת האחזור, אנו למעשה מגדירים כיצד הידע יימצא, יישלף ויוצג. כאשר אנו מעצבים את תבנית הPrompt-, אנו למעשה מכתיבים את כללי האמת: מה מותר למודל לומר, ומה אסור. זוהי הנדסת אמינות --- האומנות של הפיכת מכונת סטטיסטיקה חזקה, אך נוטה להזיה, למומחה תחום אחראי ומקורקע בעובדות.

המערכת שנבנה בפרק זה תהיה מערכת RAG מלאה, מקומית לחלוטין, המסוגלת לקבל מסמכי PDF, לפצל אותם, להטמיע אותם, לאחסנם במאגר ווקטורי, לאחזר את הקונטקסט הרלוונטי ביותר בתגובה לשאילתת משתמש, ולבסוף ליצור תשובה מדויקת ומקורקעת בראיות. הכול --- מהאינדקס ועד הגנרציה --- יפעל על החומרה המקומית שלנו, ללא תלות בשירותי ענן, ללא עלות שוטפת, ובפרטיות מוחלטת. זוהי ריבונות מלאה על התהליך כולו.

7.1 תהליך האחזור: מהשאילתה אל הקונטקסט

לאחר שבנינו את מאגר הנתונים הווקטורי --- את ספריית הזיכרון --- הגיע הזמן להפוך אותו לכלי חיפוש פעיל. תהליך האחזור הוא הלב הפועם של מערכת RAG. זהו השלב שבו השאילתה של המשתמש, אותה פיסת טקסט קצרה ולעתים לא מנוסחת באופן מושלם, הופכת למפתח שפותח את דלתות הידע הרלוונטי המוסתר בתוך אלפי עמודים של מסמכים.

המושג המרכזי בתהליך האחזור הוא **דמיון סמנטי** (Semantic Similarity). כזכור, כל פיסת טקסט במאגר שלנו --- כל Chunk --- הומרה לוקטור מספרי רב-ממדי באמצעות מודל ההטמעה nomic-embed-text. וקטורים אלו אינם מייצגים רק את המילים עצמן, אלא את **המשמעות הסמנטית** שלהן. פיסות טקסט בעלות משמעות דומה, גם אם הן משתמשות במילים שונות לחלוטין, ימוקמו קרוב זו לזו בחלל הווקטורי הרב-ממדי.

כאשר משתמש שולח שאילתה --- נניח, ``מהם הקנסות על אי דיווח?" --- השאילתה עצמה עוברת תהליך זהה: היא מוטמעת (Embedded) לוקטור על ידי אותו מודל הטמעה שהטמיע את המסמכים. כעת, במקום לחפש התאמה מדויקת של מילים, המערכת מבצעת **חיפוש דמיון ווקטורי** (Vector Similarity Search): היא משווה את וקטור השאילתה לכל הוקטורים השמורים במאגר, ומוצאת את אלו ה**קרובים ביותר** במרחב הווקטורי. קרבה זו משמעותה דמיון סמנטי --- המערכת מחזירה את פיסות הטקסט שתוכנן הסמנטי הוא הרלוונטי ביותר לשאילתה, גם אם הן לא מכילות את אותן המילים המדויקות.

תהליך זה הוא הבסיס לכוחה של מערכת RAG. הוא מאפשר למערכת להבין את **הכוונה** מאחורי השאילתה, ולא רק את המילים המדויקות. אם משתמש שואל ``מה העונש על הפרת חובת הדיווח?", המערכת תזהה שמדובר בשאילתה דומה סמנטית לשאילתה על קנסות, גם אם המילה ``קנס" לא הופיעה בשאילתה. זהו ההבדל המהותי בין חיפוש מילות מפתח מסורתי לבין חיפוש סמנטי: האחד מחפש התאמה מילולית, השני מחפש התאמה משמעותית.

מבחינה טכנית, ההפיכה של מאגר הנתונים הווקטורי לכלי אחזור היא פשוטה מבחינה קונספטואלית. אנו לוקחים את אובייקט הChroma DB- שיצרנו בפרק הקודם, ממירים אותו לRetriever- --- אובייקט שמטרתו לקבל שאילתה ולהחזיר את המסמכים הרלוונטיים ביותר. המסגרת LangChain מספקת לנו ממשק פשוט לכך: אנו קוראים למתודה as_retriever() על אובייקט מאגר הנתונים, ומקבלים בחזרה Retriever מוכן לשימוש.

הקוד הבא ממחיש את יצירת הRetriever- מהמאגר הווקטורי Chroma DB שבנינו בפרק הקודם:

```
amorhC tropmi serotsrotcev.ytinummoc_niahcgnal morf
sgniddebmEamallO tropmi amallo_niahcgnal morf

ksid morf erots rotcev gnitsixe eht daoL #
("txet-debme-cimon"=ledom)sgniddebmEamallO = ledom_gniddebme
)amorhC = erotsrotcev
,"bd_amorhc/."=yrotcerid_tsisrep
ledom_gniddebme=noitcnuf_gniddebme
(

reveirter ot erots rotcev trevnoC #
tluafed yb sknuhc ralimis tsom 5 pot snruteR #
({5 :"k"}=sgrawk_hcraes)reveirter_sa.erotsrotcev = reveirter

yreuq elpmas a htiw laveirter tseT #
"?gnitroper-non rof seitlanep eht era tahW" = yreuq
(yreuq)ekovni.reveirter = scod_tnaveler

sknuhc deveirter yalpsiD #
:(scod_tnaveler)etaremune ni cod ,i rof
(":{1+i} knuhC"f)tnirp
(tnetnoc_egap.cod)tnirp
(05 * "-")tnirp
```

gnitsiL 1.7: יצירת reveirteR מהמאגר הווקטורי

קוד זה טוען את המאגר הווקטורי שהכנו קודם לכן --- את ספריית הזיכרון השמורה בתיקייה chroma_db/ --- ממיר אותו לRetriever-, ושולח אליו שאילתת בדיקה. הפרמטר k=5 מורה למערכת להחזיר את חמשת פיסות הטקסט הקרובות ביותר לשאילתה במרחב הווקטורי. כל פיסה מוחזרת כאובייקט מסמך, הכולל את תוכן הטקסט המקורי, יחד עם מטא-דאטה כמו מספר עמוד ומקור המסמך.

התוצאה המתקבלת היא רשימה של חמש פיסות טקסט, כל אחת בת כTokens 1200- (בהתאם לגודל הChunk- שהגדרנו באינדקס), שתוכנן הוא הרלוונטי ביותר לשאלה על קנסות ואי דיווח. אלו הן הראיות, הבסיס העובדתי, שעליו תיבנה התשובה הסופית.

אך תהליך האחזור הבסיסי הזה, על אף יעילותו, סובל מבעיה מהותית: הוא **רגיש מאוד לניסוח השאילתה**. אם המשתמש ניסח את השאילתה בצורה לא מדויקת, או השתמש במונחים שונים מאלו המופיעים במסמך, ייתכן שהאחזור יחמיץ חלק מהקונטקסט הרלוונטי. הפתרון לכך הוא טכניקת אופטימיזציה מתקדמת הנקראת Multi-Query Retriever --- מאחזר רב-שאילתות --- שנדון בה בהרחבה בסעיף הבא.

© Dr. Yoram Segal - כל הזכויות שמורות

7.2 אופטימיזציה: Multi-Query Retriever

התהליך הבסיסי של אחזור שתיארנו לעיל פועל היטב כאשר השאילתה מנוסחת בצורה מדויקת וברורה. אך במציאות, משתמשים רבים אינם מנסחים שאילתות באופן אופטימלי. הם עשויים להשתמש במילים לא מדויקות, לכלול מידע מיותר, או להשמיט מידע חיוני. יתרה מכך, לעתים קרובות אותה שאילתה יכולה להיות מנוסחת בדרכים רבות ושונות, כאשר כל ניסוח עשוי להוביל לאחזור של קונטקסט שונה מעט מהמאגר.

כאן נכנסת לתמונה הרעיון המהפכני של Multi-Query Retriever. במקום להסתמך על ניסוח יחיד של השאילתה, המערכת משתמשת בכוחו של מודל שפה מקומי --- Llama 3.2 במקרה שלנו --- כדי **לייצר מספר וריאציות של אותה השאילתה**. כל וריאציה היא ניסוח מחדש של השאילתה המקורית, המבטא את אותה הכוונה במילים שונות, ממוקדת בהיבט מעט שונה, או משתמשת במונחים חלופיים.

התהליך עובד כך: השאילתה המקורית של המשתמש מועברת למודל השפה המקומי, יחד עם הוראה מפורשת: ``צור חמש גרסאות שונות של השאילתה הבאה, כאשר כל גרסה מנסחת את אותה הכוונה במילים שונות". מודל השפה, הודות ליכולת ההכללה והיצירתיות הלשונית שלו, מייצר חמש שאילתות שונות, שכל אחת מהן היא וריאציה תקפה ומשמעותית של השאילתה המקורית.

כעת, במקום לבצע חיפוש אחד במאגר הווקטורי, המערכת מבצעת **חמישה חיפושים נפרדים** --- אחד עבור כל וריאציה. כל חיפוש מחזיר את הפיסות הרלוונטיות ביותר לאותה הוריאציה הספציפית. התוצאה הסופית היא איחוד של כל התוצאות, לאחר סינון כפילויות. בדרך זו, אנו **מרחיבים את שטח החיפוש** בצורה דרמטית: גם אם השאילתה המקורית לא הייתה מנוסחת באופן אופטימלי, סביר להניח שאחת או יותר מהווריאציות שיצר המודל תצליח לאחזר את הקונטקסט הרלוונטי.

זוהי טכניקה חכמה מאוד, שמנצלת את החוזקות של שני סוגי מודלים שונים: את **יכולת הגנרציה והיצירתיות הלשונית** של מודל השפה הגדול, ואת **יכולת החיפוש הסמנטי המדויק** של מודל ההטמעה והמאגר הווקטורי. השילוב ביניהם מייצר תוצאה חזקה ועמידה בפני שגיאות ניסוח.

מבחינה טכנית, LangChain מספקת לנו את הכלי MultiQueryRetriever, שמכמס את כל התהליך הזה לתוך אובייקט אחד. אנו מספקים לו את הRetriever- הבסיסי שיצרנו קודם, ואת מודל השפה המקומי שישמש לייצור הווריאציות, והוא מטפל בשאר: ייצור השאילתות החלופיות, ביצוע החיפושים המרובים, איחוד והסרת כפילויות.

הקוד הבא ממחיש את יצירת Multi-Query Retriever והשימוש בו:

```
amallOtahC tropmi amallo_niahcgnal morf
reveirteRyreuQitluM tropmi yreuq_itlum.sreveirter.niahcgnal morf

noitareneg yreuq rof MLL lacol ezilaitinI #
(3.0=erutarepmet ,"2.3amall"=ledom)amallOtahC = mll

reveirteR yreuQ-itluM etaerC #
yreuq hcae fo sgnisarhp evitanretla 5 etareneg lliw sihT #
)mll_morf.reveirteRyreuQitluM = reveirter_yreuq_itlum
,reveirter=reveirter
mll=mll
(


yreuq a htiw tseT #
"?gnitroper-non rof seitlanep eht era tahW" = yreuq
(yreuq)ekovni.reveirter_yreuq_itlum = scod

(":sknuhc euqinu {(scod)nel} deveirteR"f)tnirp
:(scod)etaremune ni cod ,i rof
(":{1+i} knuhCn\"f)tnirp
srahc 002 tsrif tnirP #  ([002:]tnetnoc_egap.cod)tnirp
```

gnitsiL 2.7: יצירת reveirteR yreuQ-itluM

קוד זה יוצר קודם כל אובייקט ChatOllama, שמייצג את מודל השפה המקומי Llama 3.2. שימו לב לפרמטר `temperature=0.3`: זוהי הגדרה נמוכה יחסית, שמבטיחה שהווריאציות שיווצרו יהיו קרובות במשמעותן לשאילתה המקורית, ולא יסטו לכיוונים יצירתיים מדי שעלולים להרחיק מהכוונה המקורית. לאחר מכן, אנו יוצרים את הMultiQueryRetriever- על ידי הקריאה למתודה הסטטית from_llm(), שמקבלת את הRetriever- הבסיסי ואת מודל השפה.

כאשר אנו קוראים ל()invoke- על הMultiQueryRetriever- עם השאילתה שלנו, מתרחש מאחורי הקלעים תהליך רב-שלבי: המודל יוצר חמש גרסאות של השאילתה (לדוגמה: ``מהם העונשים על אי דיווח?", ``איזה קנסות חלים על הפרת חובת הדיווח?", ``מה התוצאות המשפטיות של אי דיווח?", ``אילו סנקציות מוטלות על מי שלא מדווח?", ``מה העונש הכספי על אי דיווח?"). כל גרסה מוטמעת לוקטור ונשלחת לחיפוש במאגר. התוצאות מאוחדות, כפילויות מוסרות, והתוצאה הסופית היא רשימה מורחבת של פיסות טקסט רלוונטיות.

היתרון הוא משמעותי: **עמידות בפני שגיאות ניסוח**. גם אם המשתמש לא ניסח את השאילתה בצורה אופטימלית, סביר מאוד שאחת או יותר מהווריאציות שיצר המודל תפגע במטרה ותאחזר את הקונטקסט הדרוש. זהו שיפור משמעותי באיכות האחזור, במחיר זניח: מספר קריאות נוספות למודל השפה המקומי (שהוא חינמי ומהיר) ומספר חיפושים נוספים במאגר הווקטורי (שהוא מהיר ביותר).

הטכניקה הזו היא דוגמה מצוינת לאופן שבו אנו יכולים לשלב בין יכולות שונות של מודלים שונים כדי לבנות מערכת חזקה ועמידה יותר. המודל הגדול אינו משמש רק לגנרציה סופית, אלא גם לאופטימיזציה של תהליך האחזור עצמו. זוהי הנדסת מערכות AI חכמה: שימוש בכל כלי למטרה שהוא מצטיין בה, ושילובם לכדי שרשרת עיבוד אחת ויעילה.

7.3 הגנרציה הקונטקסטואלית: עיצוב RAG Prompt

לאחר שאחזרנו את הקונטקסט הרלוונטי מהמאגר הווקטורי --- את אותן פיסות טקסט שתוכנן הסמנטי מתאים לשאילתת המשתמש --- הגיע הזמן לשלב הסופי והקריטי ביותר: **הגנרציה המקורקעת**. זהו השלב שבו מודל השפה הגדול --- Llama 3.2 במקרה שלנו --- יוצר את התשובה הסופית, אך לא בחופשיות מוחלטת, אלא **קרוק בנחיצות** לקונטקסט שאוחזר. זוהי המהות של RAG: גנרציה המוגבלת ומוגדרת על ידי ראיות ממשיות.

העיצוב הקריטי ביותר בשלב זה הוא **תבנית הPrompt-**. הPrompt- הוא למעשה ההוראה המפורשת שאנו נותנים למודל השפה, המגדירה את גבולות סמכותו, את מקורות הידע המותרים לו, ואת האופן שבו עליו להגיב. בRAG-, תבנית הPrompt- היא הכלי המרכזי שמבטיח **קרקוע עובדתי** --- היא זו שמונעת מהמודל לסטות מהקונטקסט שאוחזר ולהזות תשובות מבוססות על הידע הסטטיסטי הכללי שלו.

הניסוח הטיפוסי והקריטי ביותר בתבנית RAG Prompt הוא: **``Answer the question based ONLY on the following context:"**. מילת המפתח כאן היא ONLY --- **רק**. הוראה זו אומרת למודל באופן מפורש ונחרץ: אסור לך לענות על בסיס הידע הכללי שלך; אסור לך לנחש; אסור לך להמציא. עליך לבסס את תשובתך **אך ורק** על פיסות הטקסט שמועברות אליך כקונטקסט. אם התשובה אינה נמצאת בקונטקסט --- עליך לומר זאת במפורש, ולא לספק תשובה.

זוהי הצהרה על **אחריות מודל**. במקום לתת למודל חופש יצירתי מוחלט, אנו מגבילים אותו בכוונה תחילה, ומחייבים אותו לפעול כעיתונאי או כחוקר: לדווח רק על מה שמופיע במקורות, ולא לפרש מעבר למה שכתוב. זוהי הדרך שבה אנו מפחיתים הלוצינציות לכמעט אפס, והופכים את המערכת לאמינה ושקופה.

בנוסף להוראה המגבילה, תבנית הPrompt- כוללת שני רכיבים מרכזיים נוספים:

1. **הקונטקסט עצמו**: פיסות הטקסט שאוחזרו מהמאגר הווקטורי. אלו מועברות למודל כחלק מהPrompt-, בדרך כלל לאחר ההוראה המגבילה. המודל רואה את הקונטקסט המלא לפני

שהוא רואה את השאילתה, כך שהוא יכול להכיר את החומר העובדתי העומד לרשותו לפני שהוא מנסה לענות.

2. **השאילתה של המשתמש**: השאלה עצמה. היא מועברת בנפרד מהקונטקסט, בדרך כלל לאחריו, כך שהמודל מבין בדיוק על מה הוא נשאל.

המבנה הכללי של RAG Prompt נראה כך:

```
Answer the question based ONLY on the following context:

{context}

Question: {question}

If the answer is not in the context, say: "I cannot answer
based on the provided context."
```

תבנית פשוטה זו היא הבסיס לכל הקרקוע העובדתי של RAG. המילים {context} ו{question}- הן משתנים שימולאו בזמן ריצה: הקונטקסט ימולא בפיסות הטקסט שאוחזרו, והשאילתה תמולא בשאלת המשתמש. המודל יקבל את הPrompt- המלא הזה, וייצור תשובה בהתאם.

הניסוח ``If the answer is not in the context, say...'' הוא קריטי. הוא נותן למודל **אפשרות יציאה כבוד**: אם הקונטקסט שאוחזר אינו מספיק כדי לענות על השאלה, המודל לא יאלץ להמציא תשובה, אלא יוכל להודות בכנות שהוא לא יודע. זוהי תכונה חיונית למערכת אמינה: עדיף להודות בחוסר ידע מאשר לספק מידע שגוי.

כעת, נבנה את השרשרת המלאה --- את צינור העיבוד שמקשר בין השאילתה לבין התשובה הסופית. נשתמש בLCEL- (LangChain Expression Language), שפת הביטויים של LangChain, כדי לבנות שרשרת אלגנטית וקריאה. השרשרת תכלול את כל הרכיבים: האחזור, הפרומפט, המודל והעיצוב הסופי של הפלט.

הקוד הבא ממחיש את בניית שרשרת RAG מלאה:

```
etalpmeTtpmorPtahC tropmi stpmorp.eroc_niahcgnal morf
resraPtuptuOrtS tropmi sresrap_tuptuo.eroc_niahcgnal morf
hguorhtssaPelbannuR tropmi selbannur.eroc_niahcgnal morf

etalpmet tpmorp GAR enifeD #
txetnoc gniwollof eht no YLNO desab noitseuq eht rewsnA""" = etalpmet
:

{txetnoc}

{noitseuq} :noitseuQ

:yas ,txetnoc eht ni ton si rewsna eht fI
".txetnoc dedivorp eht no desab rewsna tonnac I"
"""

(etalpmet)etalpmet_morf.etalpmeTtpmorPtahC = tpmorp

stnemucod deveirter tamrof ot noitcnuf repleH #
:(scod)scod_tamrof fed
(scod ni cod rof tnetnoc_egap.cod)nioj."n\n\" nruter
```


© Dr. Yoram Segal - כל הזכויות שמורות

```
# Build GAR chain using LCEL
) = niahc_gar
,scod_tamrof | reveirter_yreuq_itlum :"txetnoc"}
{()hguorhtssaPelbannuR :"noitseuq"
tpmorp |
mll |
()resraPtuptuOrtS |
(

# Test GAR chain
"?gnitroper-non rof seitlanep eht era tahW" = yreuq
(yreuq)ekovni.niahc_gar = esnopser
(esnopser)tnirp
```

gnitsiL 3.7: בניית שרשרת GAR מלאה

קוד זה הוא הלב של מערכת RAG המקומית שלנו. נבין אותו שלב אחר שלב:

ראשית, אנו מגדירים את תבנית הPrompt- באמצעות מחרוזת Python רב-שורתית. התבנית כוללת את ההוראה המגבילה, את משתני הקונטקסט והשאילתה, ואת הוראת היציאה במקרה של חוסר מידע. לאחר מכן, אנו יוצרים אובייקט ChatPromptTemplate מהתבנית, שמטרתו למלא את המשתנים בזמן ריצה.

השורה הבאה מגדירה פונקציית עזר format_docs(), שתפקידה לקחת את רשימת המסמכים המוחזרים (כל אחד אובייקט עם תכונה page_content) ולאחד אותם למחרוזת טקסט אחת, מופרדת בשורות ריקות. זהו הקונטקסט המלא שיועבר למודל.

כעת מגיעה השורה המרכזית: בניית השרשרת באמצעות LCEL. הסימון | הוא אופרטור הצינור של LangChain, שמחבר בין רכיבים בשרשרת. נקרא את השרשרת משמאל לימין:

1. **{"context": ..., "question": ...}**: זהו מילון שמגדיר את המשתנים שיועברו לפרומפט. עבור context, אנו מעבירים את השאילתה דרך multi_query_retriever (שמבצע את האחזור המרובה-שאילתות), ולאחר מכן דרך format_docs (שמעצב את המסמכים). עבור question, אנו משתמשים בRunnablePassthrough()-, שפשוט מעביר את השאילתה המקורית כפי שהיא.

2. **prompt**: הפרומפט מקבל את המשתנים ממלא את התבנית, יוצר פרומפט מלא.

3. **llm**: הפרומפט המלא מועבר למודל השפה המקומי Llama 3.2, שמייצר את התשובה.

4. **StrOutputParser()**: הפלט של המודל מנוקה ומוחזר כמחרוזת טקסט פשוטה.

התוצאה היא שרשרת אחת, שלמה וקריאה, שמקבלת שאילתה ומחזירה תשובה מקורקעת. כאשר אנו קוראים לrag_chain.invoke(query)-, השרשרת מבצעת את כל השלבים האלו באופן אוטומטי: אחזור, עיצוב, פרומפט, גנרציה ועיצוב פלט. הכול בקריאה אחת.

נניח שהשאילתה ``What are the penalties for non-reporting?" נשלחת למערכת, והקונטקסט שאוחזר מהמאגר הווקטורי כולל את הפסקה הבאה ממסמך רגולטורי: ``Failure to report as required by Section 12(a) shall result in a penalty of up to $50,000 per violation. Repeated violations may result in suspension of license." --- התשובה שתיווצר תהיה משהו כמו: ``According to the provided context, the penalties for non-reporting include a fine of up to $50,000 per violation, with repeated violations potentially resulting in suspension of license."

שימו לב לכמה דברים חשובים בתשובה זו:

- המודל מפנה במפורש למקור: ``According to the provided context".

© Dr. Yoram Segal - כל הזכויות שמורות

- המודל מצטט את המידע המדויק מהקונטקסט, ללא תוספות או ניחושים.
- המודל לא מוסיף מידע כללי על קנסות מסוגים אחרים, או על נושאים קשורים, שאינם מופיעים בקונטקסט.

זוהי תשובה **מקורקעת, אמינה ושקופה**. אנו יכולים לאמת אותה על ידי בדיקת המסמך המקור, ולהיות בטוחים שהמודל לא המציא דבר. זהו ההבדל המהותי בין מערכת RAG לבין שימוש ישיר במודל שפה ללא קונטקסט.

7.4 פרויקט מעשי מלא: מערכת RAG מקומית עם Streamlit

כעת, לאחר שהבנו את כל הרכיבים התיאורטיים והטכניים, הגיע הזמן לשלב הסופי: בניית אפליקציה מלאה, עם ממשק משתמש, שמדגימה את כוחה של מערכת RAG מקומית. נבנה אפליקציית Streamlit פשוטה ואלגנטית, שתאפשר למשתמש להעלות קובץ PDF, לבנות עבורו אינדקס ווקטורי באופן אוטומטי, ולאחר מכן לשאול שאלות ולקבל תשובות מקורקעות בתוכן המסמך.

Streamlit הוא מסגרת Python פשוטה ליצירת ממשקי משתמש אינטראקטיביים. הוא מאפשר לנו להפוך סקריפט Python רגיל לאפליקציה מלאה עם ממשק ווב, ללא צורך בידע ב-HTML, CSS או JavaScript. האפליקציה תרוץ על השרת המקומי שלנו, ותהיה נגישה דרך הדפדפן.

האפליקציה שלנו תכלול את התכונות הבאות:

- העלאת קובץ PDF על ידי המשתמש דרך הממשק.
- אינדקס אוטומטי של המסמך: טעינה, פיצול לחתיכות, הטמעה ווקטורית, ואחסון ב-Chroma DB.
- שדה קלט לשאילתות משתמש.
- הצגת התשובה המקורקעת שנוצרה על ידי מערכת RAG.
- אופציה להציג את הקונטקסט שאוחזר (פיסות הטקסט הרלוונטיות), לשם שקיפות ואימות.

הקוד הבא הוא היישום המלא של האפליקציה. הוא מחולק לשני קבצים: rag_app.py (האפליקציה עצמה) ו-rag_engine.py (מנוע הRAG). נתחיל עם מנוע הRAG:

```
so tropmi
redaoLFDPyP tropmi sredaol_tnemucod.ytinummoc_niahcgnal morf
rettilpStxeTretcarahCevisruceR tropmi rettilps_txet.niahcgnal morf
amorhC tropmi serotsrotcev.ytinummoc_niahcgnal morf
amallOtahC ,sgniddebmEamallO tropmi amallo_niahcgnal morf
reveirteRyreuQitluM tropmi yreuq_itlum.sreveirter.niahcgnal morf
etalpmeTtpmorPtahC tropmi stpmorp.eroc_niahcgnal morf
resraPtuptuOrtS tropmi sresrap_tuptuo.eroc_niahcgnal morf
hguorhtssaPelbannuR tropmi selbannur.eroc_niahcgnal morf

ssalc lacoL GAR enigne gnisu amallO dna amorhC BD.
:enignEGARlacoL
"""
sedivorP tnemucod gnixedni dna yreuq gnirewsna.
"""

fed __tini__(fles, ledom_eman="2.3amall",
```

```
                    :("txet-debme-cimon"=ledom_debme
(3.0=erutarepmet ,eman_ledom=ledom)amallOtahC = mll.fles
  (ledom_debme=ledom)sgniddebmEamallO = ledom_debme.fles
                                 enoN = erotsrotcev.fles
                                   enoN = niahc_gar.fles

        ,0021=ezis_knuhc ,htap_fdp ,fles)tnemucod_xedni fed
                     :(003=palrevo_knuhc
                                                    """
                .erots rotcev otni tnemucod FDP a xednI

                                                  :sgrA
                         elif FDP ot htaP :htap_fdp
                     sknuhc txet fo eziS :ezis_knuhc
                sknuhc neewteb palrevO :palrevo_knuhc
                                                    """
                                            FDP daoL #
                         (htap_fdp)redaoLFDPyP = redaol
                              ()daol.redaol = stnemucod

                                     sknuhc otni tilpS #
        )rettilpStxeTretcarahCevisruceR = rettilps_txet
                            ,ezis_knuhc=ezis_knuhc
                        palrevo_knuhc=palrevo_knuhc
                                                     (
      (stnemucod)stnemucod_tilps.rettilps_txet = sknuhc

                                 erots rotcev etaerC #
              )stnemucod_morf.amorhC = erotsrotcev.fles
                                 ,sknuhc=stnemucod
                        ,ledom_debme.fles=gniddebme
                    "bd_amorhc/."=yrotcerid_tsisrep
                                                     (

                                      niahc GAR dliuB #
                                ()niahc_gar_dliub_.fles

                                    (sknuhc)nel nruter

                                :(fles)niahc_gar_dliub_ fed
  """.reveirter yreuq-itlum htiw niahc GAR eht dliuB"""
                         reveirter yreuq-itlum etaerC #
            )reveirter_sa.erotsrotcev.fles = reveirter
                            {5 :"k"}=sgrawk_hcraes
                                                     (
        )mll_morf.reveirteRyreuQitluM = reveirter_itlum
                             ,reveirter=reveirter
                                     mll.fles=mll
                                                     (

                               etalpmet tpmorp enifeD #
```

```
:txetnoc eht no YLNO desab rewsnA""" = etalpmet
{txetnoc}
{noitseuq} :noitseuQ
".txetnoc morf rewsna tonnaC" :yas ,txetnoc ni ton fI
"""
(etalpmet)etalpmet_morf.etalpmeTtpmorPtahC = tpmorp
niahc dliuB #
:(scod)scod_tamrof fed
(scod ni cod rof tnetnoc_egap.cod)nioj."n\n\" nruter
) = niahc_gar.fles
,scod_tamrof | reveirter_itlum :"txetnoc"}
{()hguorhtssaPelbannuR :"noitseuq"
tpmorp |
mll.fles |
()resraPtuptuOrtS |
(
:(noitseuq ,fles)yreuq fed
"""
.metsys GAR eht yreuQ
:sgrA
noitseuq resU :noitseuq
:snruteR
gnirts rewsnA
"""
:enoN si niahc_gar.fles fi
".tsrif tnemucod a xedni esaelP" nruter
(noitseuq)ekovni.niahc_gar.fles nruter
```

$_{e}ngine.py - RAG$gar :4.7 gnitsiL

קוד זה מגדיר מחלקה LocalRAGEngine שמכמסת את כל הפונקציונליות של מערכת RAG מקומית. המתודה index_document() מקבלת נתיב לPDF-, מבצעת את כל שלבי האינדקס (טעינה, פיצול, הטמעה, אחסון), ובונה את שרשרת הRAG- באמצעות Multi-Query Retriever. המתודה query() מקבלת שאילתה ומחזירה תשובה מקורקעת.

כעת נבנה את ממשק הStreamlit-:

```
ts sa tilmaerts tropmi
elifpmet tropmi
so tropmi
enignEGARlacoL tropmi enigne_gar morf
gifnoc egaP #
)gifnoc_egap_tes.ts
,"metsyS GAR lacoL"=eltit_egap
```


© Dr. Yoram Segal - כל הזכויות שמורות

```
                                                    ,"□"=noci_egap
                                                     "ediw"=tuoyal
                                                                  (

                                                          eltiT #
                        ("amallO htiw metsyS GAR lacoL □")eltit.ts
                                                  """)nwodkram.ts
                          .snoitseuq ksa dna tnemucod FDP a daolpU
                              .etavirp dna lacol si gnissecorp llA
                                                            """(

                          etats noisses ni enigne GAR ezilaitinI #
                          :etats_noisses.ts ni ton 'enigne_gar' fi
                  ()enignEGARlacoL = enigne_gar.etats_noisses.ts
                                eslaF = dexedni.etats_noisses.ts

                                     daolpu tnemucod rof rabediS #
                                                 :rabedis.ts htiw
                                ("daolpU tnemucoD □")redaeh.ts
                            )redaolpu_elif.ts = elif_dedaolpu
                                            ,"FDP daolpU"
                                             ['fdp']=epyt
                                                          (

            :dexedni.etats_noisses.ts ton dna elif_dedaolpu fi
                  :("...tnemucod gnixednI")rennips.ts htiw
                                elif pmet ot evaS #
                      )eliFyraropmeTdemaN.elifpmet htiw
                                  ,eslaF=eteled
                                  'fdp.'=xiffus
                                          :pmt sa (
              (()reffubteg.elif_dedaolpu)etirw.pmt
                              eman.pmt = htap_pmt

                                 tnemucod xednI #
            \.enigne_gar.etats_noisses.ts = sknuhc_mun
                 (htap_pmt)tnemucod_xedni

                               elif pmet pu naelC #
                                (htap_pmt)knilnu.so

                        eurT = dexedni.etats_noisses.ts
          ("!sknuhc {sknuhc_mun} dexednI"f)sseccus.ts

                                        seireuq rof aera niaM #
                                   :dexedni.etats_noisses.ts fi
                               ("snoitseuQ ksA □")redaeh.ts

                                  )tupni_txet.ts = noitseuq
                                ,":noitseuq ruoy retnE"
    "?gnitroper-non rof seitlanep eht era tahW"=redlohecalp
```

```
(
:noitseuq dna ("rewsnA teG")nottub.ts fi
:("...rewsna gnitareneG")rennips.ts htiw
(noitseuq)yreuq.enigne_gar.etats_noisses.ts = rewsna

(":rewsnA")redaehbus.ts
(rewsna)ofni.ts
:esle
("nigeb ot FDP a daolpU □")ofni.ts

retooF #
("---")nwodkram.ts
""")nwodkram.ts
**:kcatS ygolonhceT**
(amallO) 2.3 amalL :MLL -
(amallO) txet-debme-cimon :sgniddebmE -
BD amorhC :erotS rotceV -
tilmaertS + niahCgnaL :krowemarF -
(""
```

gnitsiL :5.7 gar$_{app.py - Streamlit}$

קוד זה בונה ממשק משתמש מלא ואינטואיטיבי. הוא משתמש בst.session_state- של Streamlit כדי לשמור את מצב האפליקציה (האם מסמך כבר אוינדקס) ואת מנוע הRAG- עצמו. הסיידבר מאפשר העלאת קובץ PDF, והאזור הראשי מאפשר שליחת שאילתות וקבלת תשובות.

כדי להריץ את האפליקציה, יש לשמור את שני הקבצים באותה תיקייה ולהריץ:

```
streamlit run rag_app.py
```

האפליקציה תיפתח בדפדפן, ונוכל להעלות מסמך PDF (למשל, דוח רגולטורי של בנק ישראל על חובות דיווח), לחכות לסיום האינדקס, ולאחר מכן לשאול שאלות כמו ``מהם הקנסות על אי דיווח?" --- והמערכת תחזיר תשובה מדויקת ומקורקעת בתוכן המסמך.

זוהי מערכת RAG מלאה, מקומית לחלוטין, המופעלת על ידי Ollama. הכול --- מהאינדקס ועד הגנרציה --- מתבצע על המחשב שלנו, ללא שליחת נתונים לשרתים חיצוניים, ללא עלות שוטפת, ובפרטיות מוחלטת. זוהי הדגמה חיה של ריבונות דיגיטלית מלאה: אנו בעלים על הנתונים, על המודלים, על התהליך ועל התוצאות.

7.5 דוגמת שימוש: ``מהם הקנסות על אי דיווח?"

כדי להמחיש את כוחה של המערכת, נבחן דוגמה קונקרטית. נניח שהעלינו לאפליקציה את הוראת בנק ישראל בנושא חובות דיווח של מוסדות פיננסיים, מסמך בן 50 עמודים המכיל פרטים מפורטים על נהלים, חובות, ועונשים. המסמך פוצל ל120- חלקים (Chunks), כל אחד בן כTokens 1200-, והוטמע למאגר הווקטורי.

המשתמש שולח את השאילתה: ``מהם הקנסות על אי דיווח?"

שלב 1 --- אחזור רב-שאילתות: הMulti-Query Retriever- יוצר חמש גרסאות של השאילתה:

- ``מהם העונשים על אי דיווח?"
- ``איזה קנסות חלים על הפרת חובת הדיווח?"

- ``מה התוצאות המשפטיות של אי דיווח?"
- ``אילו סנקציות מוטלות על מי שלא מדווח?"
- ``מה העונש הכספי על אי דיווח?"

כל שאילתה מוטמעת לוקטור ונשלחת לחיפוש במאגר. התוצאות מאוחדות.
שלב 2 --- אחזור קונטקסט: המערכת מזהה חמש פיסות טקסט רלוונטיות, ביניהן הפסקה הבאה (לצורך הדוגמה, בעברית):

> ``מוסד פיננסי שלא דיווח במועד כנדרש לפי סעיף 12(a) להוראה זו, ייקנס בקנס מנהלי של עד 50,000 שקלים חדשים לכל הפרה. במקרה של הפרות חוזרות, יוסמך המפקח להורות על השעיית הרישיון לתקופה של עד 90 יום."

שלב 3 --- בניית פרומפט: המערכת בונה פרומפט מלא:

```
Answer the question based ONLY on the following context:

[... 5 chunks of text, including the one above ...]

Question: What are the penalties for non-reporting?

If the answer is not in the context, say:
"I cannot answer based on the provided context."
```

שלב 4 --- גנרציה: מודל Llama 3.2 מקבל את הפרומפט המלא ויוצר את התשובה:

> ``על פי המסמך, מוסד פיננסי שלא דיווח במועד יוכל להיקנס בקנס מנהלי של עד 50,000 שקלים חדשים לכל הפרה. במקרה של הפרות חוזרות, המפקח מוסמך להורות על השעיית רישיון לתקופה של עד 90 יום."

שלב 5 --- הצגה למשתמש: התשובה מוצגת במסך, יחד עם אפשרות להציג את הקונטקסט שאוחזר.

התוצאה: המשתמש קיבל תשובה **מדויקת, מקורקעת ואמינה**, המבוססת על המסמך שהעלה. אם הוא רוצה, הוא יכול לאמת את התשובה על ידי עיון בפיסות הטקסט שאוחזרו, או אפילו לחזור למסמך המקורי ולבדוק את הסעיף 12(a). זוהי **שקיפות מלאה** --- המשתמש יודע מהיכן הגיעה התשובה, ויכול לסמוך עליה.

מערכת זו יכולה לשמש למגוון רחב של מקרי שימוש: ניתוח מסמכים רגולטוריים, שאילתות על מדיניות פנימית של חברה, חיפוש מידע בדוחות טכניים ארוכים, וכל תרחיש אחר שבו יש צורך לחלץ מידע מדויק ממקור ממשי. הכול באופן מקומי, פרטי וחינמי.

זוהי המימוש המלא של הריבונות הקונטקסטואלית: הידע שלנו, המודלים שלנו, התהליך שלנו, התשובות שלנו --- הכול תחת קורת גג אחת, ללא תלות בשום גוף חיצוני. זוהי החירות החישובית במלוא הדרה.

© Dr. Yoram Segal - כל הזכויות שמורות

פרק 8

מערכות סוכנים מתקדמות: מבינה יחידנית לאוטומציה קוגניטיבית

RAG-ו Ollama

ריבונות דיגיטלית בעידן הבינה המלאכותית

Author: Dr. Yoram Segal

All rights reserved

11/2025

תוכן העניינים

פרק 8

מערכות סוכנים מתקדמות: מבינה יחידנית לאוטומציה קוגניטיבית

8.1 מאינטראקציה לאוטומציה: המהפכה השקטה

היסטוריון העתיד, כאשר יבחן את העשור השני של המאה ה21, יבחין בנקודת מפנה מרתקת בתולדות הקוגניציה המלאכותית. שנים ארוכות שלטה תפיסה פשטנית: בינה מלאכותית נתפסה כישות יחידנית, מונוליתית, המגיבה לשאלות ומייצרת תשובות. כך פעלו ChatGPT, Claude, ומודלי השפה הגדולים האחרים. המשתמש היה שואל, המודל היה מייצר, והתהליך היה נגמר. אך מה שהחסיר מתפיסה זו היה המימד העמוק יותר של **דלגציה קוגניטיבית** -- היכולת של מכונה לא רק לענות, אלא **לנהל תהליכים מורכבים** באופן אוטונומי, כפי שמנהל אנושי מדלג משימות לצוות של מומחים.

הופעתן של **מערכות סוכנים** (Agent Systems) מסמלת מעבר פילוסופי מהותי: מהבינה המלאכותית ככלי המגיב לבקשות, לבינה מלאכותית כמנהלת המתזמרת תהליכים. במקום שמודל שפה יחיד ינסה לטפל בכל משימה בעצמו -- מחילוץ מידע ממסמכים, דרך ניתוח ועד להחלטה -- מערכת סוכנים מפצלת את התהליך לרכיבים מיוחדים, כאשר כל סוכן (Agent) הוא מומחה במשימה ספציפית אחת. זהו שינוי ארכיטקטוני עמוק, המדמה את האופן שבו ארגונים אנושיים פועלים: לא באמצעות אדם יחיד רב-יכולות, אלא דרך **רשת מתוזמרת של מומחים**.

מערכות אלה מגשימות את החזון של אוטומציה קוגניטיבית מלאה. כאשר משלבים את הכוח החישובי של Ollama -- המאפשר הרצת מודלי שפה מקומיים ללא עלות ובפרטיות מלאה -- עם ארכיטקטורת סוכנים מתוזמרת, נוצרת תשתית שיכולה לבצע תהליכים עסקיים מורכבים באופן אוטונומי, ללא צורך בהתערבות אנושית מתמדת. זהו המעבר מAI-מגיב לAI-מנהל, ממערכת שתלויה בנו למערכת שמשרתת אותנו בצורה יזומה ועצמאית.

במקביל, התפתחות זו מעלה שאלות עמוקות על טיב האינטליגנציה והאוטונומיה. האם רשת של סוכנים מיוחדים היא אינטליגנטית יותר ממודל יחיד גדול? האם הדלגציה היא סימן לחולשה או לעוצמה? ההיסטוריה האנושית מלמדת שהתמחות וחלוקת עבודה הן מנועי ההתקדמות הגדולים ביותר של הציביליזציה. כעת, עקרון זה חודר לעולם הקוגניציה המלאכותית.

8.2 ארכיטקטורת סוכנים: מהמונוליט לקהילה

8.2.1 המושג: מהו סוכן?

סוכן (Agent) בהקשר של בינה מלאכותית אינו פשוט מודל שפה שמריץ פעולה אחת. סוכן הוא **ישות אוטונומית בעלת זהות, תפקיד ומומחיות מוגדרים**, המסוגלת לקבל קלט, לעבד אותו על פי הוראות ספציפיות, ולהעביר את התוצאה הלאה. ההבדל המהותי בין סוכן למודל רגיל הוא בהתמחות: כל סוכן מתוכנן לבצע משימה אחת בצורה מושלמת, במקום לנסות לעשות הכול בצורה ממוצעת.

דמיינו רופא כירורג. הוא מומחה בניתוחים, אך לא מומחה באנסתזיה או בטיפול פוסט-ניתוחי. בחדר ניתוח מודרני, צוות רפואי שלם פועל יחד: מרדים, כירורג, אחות מכשירים, מנטר חיים. כל אחד מבצע את תפקידו המוגדר בצורה מושלמת, והתוצאה היא ניתוח מוצלח. כך פועלת מערכת סוכנים: **רשת של מומחים מתוזמרת**, שבה כל סוכן אחראי על חלק אחד מהתהליך הכולל.

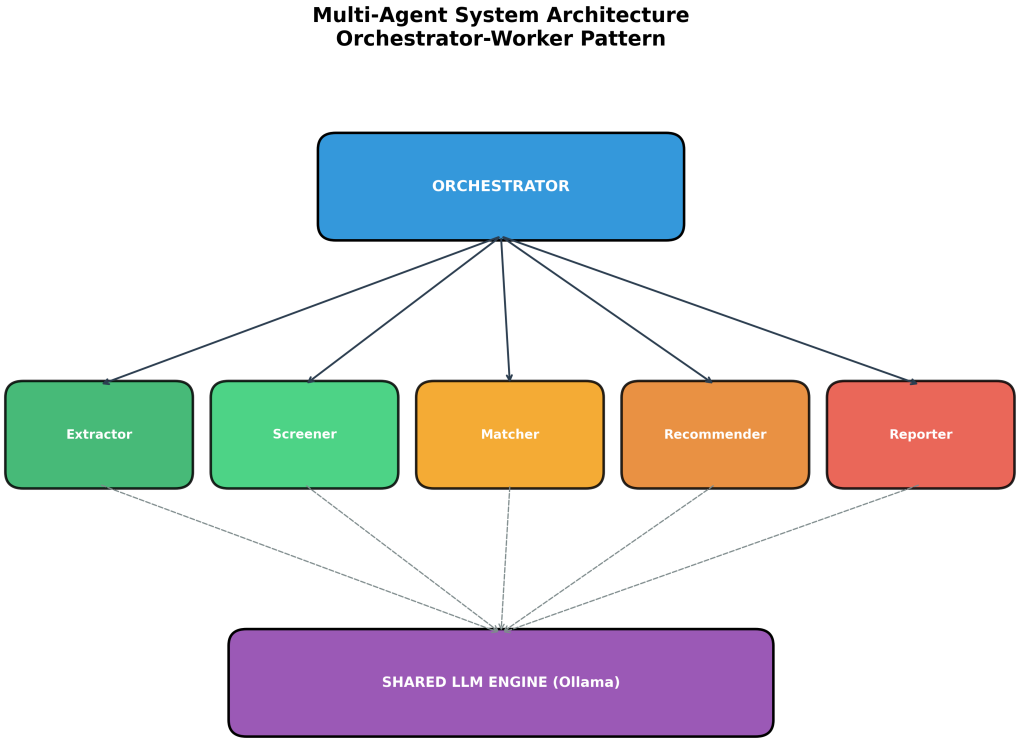
ארכיטקטורת סוכנים מורכבת ממספר רכיבים מרכזיים, כפי שמתואר באיור 8.1:

סוכן המתזמר (Orchestrator Agent): זהו המנהל המרכזי, האחראי על תזמון התהליך הכולל. הוא מחליט איזה סוכן יפעל מתי, מעביר מידע בין הסוכנים, ומוודא שהתהליך זורם לוגית משלב לשלב. המתזמר הוא המוח האסטרטגי של המערכת.

סוכנים מומחים (Specialist Agents): כל אחד מבצע משימה ספציפית בלבד. סוכן עשוי להיות מומחה בחילוץ מידע ממסמכים, אחר בסינון ובציון, שלישי בהתאמה, ורביעי בהמלצות. כל סוכן מקבל הוראות מפורטות (System Prompt) המגדירות את זהותו, תפקידו, ואת הפורמט שבו עליו להחזיר תוצאות.

זרימת קונטקסט (Context Flow): המידע זורם מסוכן לסוכן כשרשרת. התוצאה של סוכן אחד הופכת לקלט של הסוכן הבא. המתזמר מוודא שהמידע הנצבר בכל שלב (accumulated context) מועבר הלאה, כך שסוכנים מאוחרים בשרשרת יכולים להסתמך על עבודת הסוכנים שקדמו להם.

מנוע חישוב משותף: כל הסוכנים משתמשים באותו מנוע LLM בסיסי. במקרה שלנו, Ollama עם Llama 3.2 משמש כמוח החישובי המרכזי. כל סוכן מפעיל את אותו המודל, אך עם הוראות (System Prompt) שונות לחלוטין, מה שיוצר מומחיות מדומה דרך הנדסת זהות.



איור 8.1: ארכיטקטורת מערכת סוכנים: המתזמר מנהל זרימת עבודה מתוזמרת בין סוכנים מומחים, כאשר כל אחד מבצע משימה ספציפית ומעביר את התוצאה לשלב הבא.

איור 8.1 מציג את הארכיטקטורה הכללית של מערכת סוכנים. ניתן לראות כיצד המתזמר עומד במרכז, מתאם את כל הסוכנים המומחים, ומוודא שהמידע זורם בצורה לוגית ומסודרת. כל חץ מייצג

© Dr. Yoram Segal - כל הזכויות שמורות

העברת קונטקסט, וכל סוכן מסומן עם תפקידו הייחודי. המבנה הזה מאפשר מודולריות מלאה: ניתן להוסיף סוכנים חדשים, להחליף סוכנים קיימים, או לשנות את סדר הפעולות, ללא צורך לשנות את המודל הבסיסי עצמו.

8.2.2 מסגרות עבודה: Swarm והתזמור הקוגניטיבי

כדי לממש ארכיטקטורת סוכנים בפועל, נזקקים למסגרת עבודה (Framework) שמנהלת את התזמור הזה. אחת המסגרות המתקדמות והאלגנטיות ביותר היא Swarm -- ספרייה קלת משקל המאפשרת לבנות רשתות מתוזמרות של סוכנים המופעלים על ידי Ollama.

Swarm מתבססת על עקרון פשוט אך עוצמתי: כל סוכן הוא אובייקט Python עם שם, System Prompt ייחודי, ומודל בסיס משותף. המסגרת מספקת מנגנון העברת קונטקסט אוטומטי (context handoff), המאפשר לסוכן אחד להעביר את השליטה והמידע לסוכן הבא בשרשרת. המתזמר מתוכנת כך שיקרא לסוכנים לפי הצורך, יעביר את התוצאות ביניהם, ויוודא שהתהליך מתבצע לוגית.

היתרון המרכזי של Swarm הוא **הפשטות והשקיפות**. בניגוד למסגרות מורכבות יותר שדורשות תשתית ענן או תצורות מסורבלות, Swarm היא קוד פתוח, קלת הבנה, ופועלת באופן מקומי לחלוטין. היא משלימה באופן מושלם את Ollama, שכן שניהם חולקים את אותה פילוסופיה: אוטונומיה, פשטות, וריבונות מקומית.

דוגמה בסיסית לשימוש במסגרת ניתן לראות בקוד הבא:

```
{tnega_cisab:tsl}lebal\
tnegA ,mrawS tropmi mraws morf

amallO htiw tneilc mrawS ezilaitinI #
("2.3amall"=ledom)mrawS = tneilc

tnega tsilaiceps a enifeD #
)tnegA = tnega_rotcartxe
,"rotcartxE"=eman
.tsilaiceps noitcartxe atad a era uoY"""=snoitcurtsni
.txet morf noitamrofni derutcurts tcartxe ot si boj ruoY
,"""tamrof NOSJ ni stluser nruteR
"2.3amall"=ledom
(

tnega eht nuR #
)nur.tneilc = esnopser
,tnega_rotcartxe=tnega
slliks dna eman tcartxE" :"tnetnoc" ,"resu" :"elor"}]=segassem
[{"trepxe nohtyP ,eoD nhoJ :morf
(

(["tnetnoc"][1-]segassem.esnopser)tnirp
```

gnitsiL 1.8: הגדרת סוכן פשוט עם mrawS ו-amallO

כפי שניתן לראות בקוד **??**, הגדרת סוכן היא פשוטה וישירה. השדה instructions מגדיר את הזהות והתפקיד של הסוכן, בעוד model מציין את מודל הLLM הבסיסי. כאשר client.run() מופעל, הסוכן מקבל את ההודעה, מעבד אותה על פי ההוראות, ומחזיר תוצאה. זהו הבסיס לבניית מערכות מורכבות יותר.

8.3 מקרה מבחן מתקדם: סוכנות גיוס מלאכותית

8.3.1 תיאור הבעיה: מהמורכבות לאוטומציה

כדי להמחיש את הכוח של מערכות סוכנים, נבנה יישום מלא ומורכב: **סוכנות גיוס מלאכותית** (AI Recruitment Agency). תהליך הגיוס האנושי הוא אחד התהליכים העסקיים המורכבים והקריטיים ביותר בארגונים מודרניים. הוא כולל שלבים מרובים: קבלת קורות חיים, חילוץ מידע רלוונטי, סינון מועמדים על פי קריטריונים, התאמה למשרות פנויות, והמלצות סופיות. כל שלב דורש מומחיות אחרת, החלטות מורכבות, והערכה של גורמים רבים.

המטרה שלנו היא לבנות מערכת שמבצעת את כל התהליך הזה באופן אוטונומי לחלוטין, באמצעות חמישה סוכנים מיוחדים, כולם פועלים על גבי Ollama המקומי. המערכת תקבל קובץ קורות חיים בפורמט PDF, תעבד אותו שלב אחר שלב, ותייצר דוח המלצות מקיף ומנומק. כל זאת ללא עלות, באופן פרטי, ובמהירות.

תהליך הגיוס שלנו יחולק לחמישה שלבים מובחנים, כל אחד מנוהל על ידי סוכן ייעודי:

שלב 1: חילוץ (Extraction) -- סוכן מומחה בקריאת קורות חיים ממסמכי PDF וחילוץ המידע הרלוונטי לפורמט מובנה (JSON).

שלב 2: סינון וציון (Screening) -- סוכן המנתח את המידע המחולץ, מעריך את רמת הניסיון, הכישורים, וההשכלה, ומספק ציון כללי עם זיהוי נקודות חוזקה וחולשה.

שלב 3: התאמה למשרות (Matching) -- סוכן המשווה את פרופיל המועמד למאגר משרות פנויות ומציע את ההתאמות הטובות ביותר עם אחוזי התאמה.

שלב 4: המלצות (Recommendations) -- סוכן היוצר דוח סופי עם המלצות קונקרטיות, הערכת פוטנציאל, וציון ביטחון כללי.

שלב 0: תזמור (Orchestration) -- סוכן המנהל המרכזי המתזמר את כל התהליך, מוודא שכל סוכן מקבל את המידע הנכון במועד הנכון, ומצבר את התוצאות לדוח אחד שלם.

המערכת כולה תהיה מקומית, פרטית, וללא עלות תפעולית, ותדגים את העוצמה של אוטומציה קוגניטיבית מלאה.

8.3.2 ארכיטקטורת הסוכנים: חמישה מומחים וקצב אחד

נתחיל בהגדרת הסוכנים עצמם. כל סוכן יקבל System Prompt מפורט המגדיר את תפקידו, את פורמט התוצאות הצפויות, ואת הקשר שבו הוא פועל. הקוד הבא מציג את הגדרת כל אחד מהסוכנים:

```
{stnega_tnemtiurcer:tsl}lebal\
tnegA ,mrawS tropmi mraws morf
nosj tropmi

amallO htiw mrawS ezilaitinI #
("2.3amall"=ledom)mrawS = tneilc

VC morf ofni derutcurts stcartxE - rotcartxE :1 tnegA #
)tnegA = tnega_rotcartxe
,"rotcartxE VC"=eman
noitcartxe atad VC lanoisseforp a era uoY"""=snoitcurtsni
.tsilaiceps
:txet VC eht morf noitamrofni gniwollof eht tcartxE
eman lluF -
liamE -
enohP -
(noitarud ,elor ,ynapmoc htiw sboj fo tsil) ecneirepxe kroW -
(sraey ,snoitutitsni ,seerged) noitacudE -
```


© Dr. Yoram Segal - כל הזכויות שמורות

```
                                (slliks tfos dna lacinhcet) sllikS -
                                                snoitacifitreC -
          .tamrof NOSJ dilav ni noitamrofni detcartxe eht nruteR
                        ,"""llun esu ,gnissim si noitamrofni fI.
                                                "2.3amall"=ledom
                                                                (

                  etadidnac serocs dna sezylanA - reneercS :2 tnegA #
                                        )tnegA = tnega_reneercs
                                  ,"reneercS etadidnaC"=eman
  .tsilaiceps gnineercs RH decneirepxe na era uoY"""=snoitcurtsni
            :etaulave dna dedivorp eliforp etadidnac eht ezylanA
                                ecneirepxe tnaveler fo sraeY -
                              ecnaveler dna level noitacudE -
                                     htped slliks lacinhceT -
                                         noissergorp reeraC -
                  :yfitnedi dna (001-0) erocs gnineercs a edivorP
                                  (3 pot) shtgnerts yeK -
                        (yna fi) sgalf der ro snrecnoc laitnetoP -
                               :htiw tamrof NOSJ ni stluser nruteR
                                  ,"""{snrecnoc ,shtgnerts ,erocs}
                                                "2.3amall"=ledom
                                                                (

               sgninepo boj ot etadidnac sehctaM - rehctaM :3 tnegA #
                                        )tnegA = tnega_rehctam
                                       ,"rehctaM boJ"=eman
              .tsilaiceps gnihctam boj a era uoY"""=snoitcurtsni
           ,sgninepo boj fo tsil a dna eliforp etadidnac a neviG
                                :no desab sehctam tseb eht yfitnedi
                                                tnemngila sllikS -
                                                level ecneirepxE -
                                                    tif yrtsudnI -
                                        :edivorp ,hctam hcae roF
                                                      eltit boJ -
                                    (%001-0) egatnecrep hctaM -
                          (hctam doog a si siht yhw) gninosaeR -
                                :tamrof NOSJ ni sehctam 3 pot nruteR
                  ,"""{[{gninosaer ,tnecrep_hctam ,boj}] :sehctam}
                                                "2.3amall"=ledom
                                                                (

              snoitadnemmocer lanif sedivorP - rednemmoceR :4 tnegA #
                                        )tnegA = tnega_rednemmocer
                                  ,"rednemmoceR tnemtiurceR"=eman
         .tnatlusnoc tnemtiurcer roines a era uoY"""=snoitcurtsni
,(gnihctam ,gnineercs ,noitcartxe) sisylana suoiverp lla no desaB
                    :troper noitadnemmocer evisneherpmoc a edivorp
                                (secnetnes 3-2) tnemssessa llarevO -
                                        spets txen dednemmoceR -
                                    (%001-0) erocs ecnedifnoC -
```


© Dr. Yoram Segal - כל הזכויות שמורות

```
                                saera sucof weivretni detseggu S -
                            .egaugnal raelc ,lanoisseforp ni etirW
{saera_sucof ,ecnedifnoc ,spets_txen ,tnemssessa} :NOSJ sa nruteR
                                                              ,"""
                                                   "2.3amall"=ledom
                                                                  (

                  ssecorp eritne eht seganaM - rotartsehcrO :5 tnegA #
        :(tsil :sgninepo_boj ,rts :txet_vc)noitcnuf_rotartsehcro fed
                                                              """
                        .enilepip tnemtiurcer lluf eht setartsehcrO
                                                              """
             {sgninepo_boj :"sboj" ,txet_vc :"txet_vc"} = txetnoc

                                   VC morf atad tcartxE :1 petS #
            ("...atad VC gnitcartxE :1 petS [rotartsehcrO]")tnirp
                                   )nur.tneilc = esnopser_tcartxe
                                       ,tnega_rotcartxe=tnega
  noitamrofni tcartxE"f :"tnetnoc" ,"resu" :"elor"}]=segassem
                                     [{"{txet_vc}n\n\:VC siht morf
                                                                (
        ["tnetnoc"][1-]segassem.esnopser_tcartxe = atad_detcartxe
                              atad_detcartxe = ["detcartxe"]txetnoc
      srahc {(atad_detcartxe)nel} :detelpmoC [rotcartxE]"f)tnirp
                                                     ("detcartxe

                                       etadidnac neercS :2 petS #
          ("...etadidnac gnineercS :2 petS [rotartsehcrO]")tnirp
                                    )nur.tneilc = esnopser_neercs
                                        ,tnega_reneercs=tnega
etadidnac siht neercS"f :"tnetnoc" ,"resu" :"elor"}]=segassem
                                  [{"{atad_detcartxe}n\n\:eliforp
                                                                (
       ["tnetnoc"][1-]segassem.esnopser_neercs = tluser_gnineercs
                            tluser_gnineercs = ["gnineercs"]txetnoc
              ("detaulave etadidnaC :detelpmoC [reneercS]"f)tnirp

                                         sboj ot hctaM :3 petS #
      ("...sgninepo boj ot gnihctaM :3 petS [rotartsehcrO]")tnirp
      n\n\{atad_detcartxe}n\:eliforp etadidnaC"""f = tpmorp_hctam
        n\n\{(2=tnedni ,sgninepo_boj)spmud.nosj}n\:sboj elbaliavA
                                       """.sehctam tseb eht dniF
                                     )nur.tneilc = esnopser_hctam
                                        ,tnega_rehctam=tnega
        [{tpmorp_hctam :"tnetnoc" ,"resu" :"elor"}]=segassem
                                                                (
         ["tnetnoc"][1-]segassem.esnopser_hctam = tluser_gnihctam
                              tluser_gnihctam = ["gnihctam"]txetnoc
                 ("deifitnedi sehctaM :detelpmoC [rehctaM]"f)tnirp

                           snoitadnemmocer lanif etareneG :4 petS #
```

```
("...snoitadnemmocer gnitareneG :4 petS [rotartsehcrO]")tnirp
:enilepip lluf eht no desaB"""f = tpmorp_lanif

{atad_detcartxe} :ataD detcartxE
{tluser_gnineercs} :gnineercS
{tluser_gnihctam} :gnihctaM

""".noitadnemmocer tnemtiurcer lanif edivorP

)nur.tneilc = esnopser_dnemmocer
,tnega_rednemmocer=tnega
[{tpmorp_lanif :"tnetnoc" ,"resu" :"elor"}]=segassem
(
["tnetnoc"][1-]segassem.esnopser_dnemmocer = snoitadnemmocer
snoitadnemmocer = ["snoitadnemmocer"]txetnoc
("ydaer troper laniF :detelpmoC [rednemmoceR]"f)tnirp

txetnoc nruter

tniop yrtne eht si rotartsehcro ehT #
("=== dezilaitinI ycnegA tnemtiurceR IA ===")tnirp
(".enilepip tnega-5 htiw setadidnac ssecorp ot ydaeR")tnirp
```

gnitsiL 2.8: הגדרת סוכני הגיוס המלאכותיים

הקוד **??** מציג את ליבת המערכת. כל סוכן מוגדר עם System Prompt מדויק המתאר את תפקידו ואת פורמט הפלט הצפוי. הפונקציה orchestrator_function() מהווה את המתזמר -- היא מפעילה כל סוכן לפי הסדר הנכון, מעבירה את תוצאות כל שלב לשלב הבא, ומצבירה את כל המידע לדוח אחד שלם.

שימו לב לזרימת הקונטקסט: הסוכן הראשון (Extractor) מחלץ מידע גולמי. הסוכן השני (Screener) מקבל את המידע המחולץ ומנתח אותו. הסוכן השלישי (Matcher) מקבל גם את המידע המחולץ וגם את תוצאות הסינון, ומשווה למשרות. הסוכן הרביעי (Recommender) מקבל את התוצאות של כל הסוכנים הקודמים ומייצר המלצה מקיפה. כל שלב מסתמך על השלבים שקדמו לו, ובכך נבנית שכבת מומחיות מצטברת.

8.3.3 דוגמת ריצה: מקורות חיים לדוח המלצות

כעת נריץ את המערכת עם דוגמת קורות חיים אמיתית ונבחן את התוצאות בכל שלב. נשתמש בקורות חיים פיקטיביות של מדען נתונים בשם "דר' שרה כהן":

```
{nur_tnemtiurcer:tsl}lebal\
(etadidnac suoititcif) txet VC elpmaS #
""" = vc_elpmas
nehoC haraS .rD
moc.liame@nehoc.haras :liamE
7654321-05-279+ :enohP

:NOITACUDE
0202 ,noinhceT ,ecneicS retupmoC ni .D.hP -
7102 ,ytisrevinU vivA leT ,ecneicS ataD ni .cS.M -
5102 ,ytisrevinU werbeH ,scitamehtaM ni .cS.B -
```

```
                                                        :ECNEIREPXE KROW 31
                (tneserP-0202) learsI proChceT ,tsitneicS ataD roineS 41
                  scitylana evitciderp rof tnempoleved ledom LM deL - 51
               sdohtem elbmesne gnisu %52 yb nruhc remotsuc decudeR - 61
                           stsitneics atad roinuj 3 fo maet deganaM - 71
                         secnerefnoc reit-pot ni srepap 5 dehsilbuP - 81
                                                                      91
                                (0202-7102) IAputratS ,tsitneicS ataD 02
                          sisylana tnemitnes rof enilepip PLN tliuB - 12
             setenrebuK/rekcoD gnisu noitcudorp ot sledom deyolpeD - 22
             gnireenigne erutaef no maet tcudorp htiw detaroballoC - 32
                                                                      42
                                                            :SLLIKS 52
     rekcoD ,SWA ,krapS ,LQS ,hcroTyP ,wolFrosneT ,nohtyP :lacinhceT 62
        gnivlos-melborP ,noitacinummoC ,pihsredael maeT :sllikS tfoS 72
                                                                      82
                                                        :SNOITACIFITREC 92
                   (1202) ytlaicepS gninraeL enihcaM deifitreC SWA - 03
                         (2202) rotartsinimdA setenrebuK deifitreC - 13
                                                                  """ 23
                                                                      33
                                                  sgninepo boj elpmaS # 43
                                                       ] = sgninepo_boj 53
                                                                   } 63
                                  ,"tsitneicS ataD daeL" :"eltit"
                                 ,".dtL snoitavonnI IA" :"ynapmoc" 83
,esitrepxe PLN/LM ,ecneirepxe sraey +3 ,DhP" :"stnemeriuqer" 93
                                                ,"pihsredael maet
                                        "vivA leT" :"noitacol" 04
                                                            ,{ 14
                                                             } 24
                          ,"reenignE gninraeL enihcaM" :"eltit" 34
                            ,".cnI snoituloS duolC" :"ynapmoc" 44
        tnemyolped ,wolFrosneT ,nohtyP ,cSM" :"stnemeriuqer" 54
                                                ,"SWA ,ecneirepxe
                                          "afiaH" :"noitacol" 64
                                                            ,{ 74
                                                             } 84
                         ,"PLN - tsitneicS hcraeseR" :"eltit" 94
                             ,"sbaL hceT egaugnaL" :"ynapmoc" 05
   hcraeser ,hcroTyP ,snoitacilbup PLN ,DhP" :"stnemeriuqer" 15
                                                ,"dnuorgkcab
                                       "melasureJ" :"noitacol" 25
                                                            ,{ 35
                                                             } 45
                                  ,"tsylanA ataD roinuJ" :"eltit" 55
                                         ,"proC ataD" :"ynapmoc" 65
      ,"level yrtne ,nohtyP cisab ,LQS ,cSB" :"stnemeriuqer" 75
                                        "vivA leT" :"noitacol" 85
                                                             { 95
                                                             [ 06
```


כל הזכויות שמורות - Dr. Yoram Segal ©

```
                                                  rotartsehcro eht nuR #
                                                    (06*"=" + "n\")tnirp
                                  ("YCNEGA TNEMTIURCER IA GNINNUR")tnirp
                                                    ("n\" + 06*"=")tnirp

        (sgninepo_boj ,vc_elpmas)noitcnuf_rotartsehcro = stluser

                                                     stluser yalpsiD #
                                                    (06*"=" + "n\")tnirp
                                   ("STLUSER ENILEPIP TNEMTIURCER")tnirp
                                                    ("n\" + 06*"=")tnirp

                                       (":ATAD DETCARTXE [1 PETS]")tnirp
                                             (["detcartxe"]stluser)tnirp
                                             ("n\" + 06*"-" + "n\")tnirp

                                        (":STLUSER GNINEERCS [2 PETS]")tnirp
                                             (["gnineercs"]stluser)tnirp
                                             ("n\" + 06*"-" + "n\")tnirp

                                          (":GNIHCTAM BOJ [3 PETS]")tnirp
                                              (["gnihctam"]stluser)tnirp
                                             ("n\" + 06*"-" + "n\")tnirp

                                 (":SNOITADNEMMOCER LANIF [4 PETS]")tnirp
                                       (["snoitadnemmocer"]stluser)tnirp
                                                    (06*"=" + "n\")tnirp
```

gnitsiL 3.8: דוגמת ריצה של מערכת הגיוס

כאשר מריצים את הקוד **??**, המערכת מתחילה לעבוד שלב אחר שלב. הפלט הבא מדגים תוצאה טיפוסית של כל סוכן:

```
                                  {tuptuo_tnemtiurcer:tsl}lebal\
==============================================================
                                YCNEGA TNEMTIURCER IA GNINNUR
==============================================================

                   ...atad VC gnitcartxE :1 petS [rotartsehcrO]
                     detcartxe srahc 748 :detelpmoC [rotcartxE]

                  ...etadidnac gnineercS :2 petS [rotartsehcrO]
                      detaulave etadidnaC :detelpmoC [reneercS]

             ...sgninepo boj ot gnihctaM :3 petS [rotartsehcrO]
                         deifitnedi sehctaM :detelpmoC [rehctaM]

          ...snoitadnemmocer gnitareneG :4 petS [rotartsehcrO]
                  ydaer troper laniF :detelpmoC [rednemmoceR]

==============================================================
                                STLUSER ENILEPIP TNEMTIURCER
```

```
          ==============================================================

                                                  :ATAD DETCARTXE [1 PETS]
                                                                        }
                                        ,"nehoC haraS .rD" :"eman_lluf"
                                    ,"moc.liame@nehoc.haras" :"liame"
                                          ,"7654321-05-279+" :"enohp"
                                                  ] :"ecneirepxe_krow"
                                                                      }
                                    ,"learsI proChceT" :"ynapmoc"
                                ,"tsitneicS ataD roineS" :"elor"
                                      ,"tneserP-0202" :"noitarud"
                                                  ] :"sthgilhgih"
                                    ,"tnempoleved ledom LM deL"
                                        ,"%52 yb nruhc decudeR"
                                           ,"3 fo maet deganaM"
                                           "srepap 5 dehsilbuP"
                                                              [
                                                               ,{
                                                                }
                                          ,"IAputratS" :"ynapmoc"
                                        ,"tsitneicS ataD" :"elor"
                                         ,"0202-7102" :"noitarud"
                                                  ] :"sthgilhgih"
                                          ,"enilepip PLN tliuB"
                                      ,"noitcudorp ot deyolpeD"
                                          "gnireenigne erutaeF"
                                                              [
                                                                {
                                                                 ,[
                                                        ] :"noitacude"
"noinhceT" :"noitutitsni" ,"ecneicS retupmoC ni .D.hP" :"eerged"}
                                                   ,{0202 :"raey" ,
     vivA leT" :"noitutitsni" ,"ecneicS ataD ni .cS.M" :"eerged"}
                                         ,{7102 :"raey" ,"ytisrevinU
        werbeH" :"noitutitsni" ,"scitamehtaM ni .cS.B" :"eerged"}
                                        {5102 :"raey" ,"ytisrevinU
                                                                 ,[
                                                       } :"slliks"
 ,"krapS" ,"LQS" ,"hcroTyP" ,"wolFrosneT" ,"nohtyP"] :"lacinhcet"
                                                  ,["rekcoD" ,"SWA"
  ["gnivlos-melborP" ,"noitacinummoC" ,"pihsredael maeT"] :"tfos"
                                                                 ,{
                                                  ] :"snoitacifitrec"
                ,"(1202) ytlaicepS gninraeL enihcaM deifitreC SWA"
                       "(2202) rotartsinimdA setenrebuK deifitreC"
                                                                 [
                                                                   {

          --------------------------------------------------------------
```


כל הזכויות שמורות - Dr. Yoram Segal ©

```
                                                :STLUSER GNINEERCS [2 PETS]
                                                                        }
                                                            ,29 :"erocs"
                                                          ] :"shtgnerts"
                  ,"(noinhceT morf DhP) dnuorgkcab cimedaca gnortS"
        ,"maet ecneics atad gniganam ecneirepxe pihsredael nevorP"
    %52) tcapmi ssenisub elbarusaem htiw drocer kcart evisserpmI"
                                                "(noitcuder nruhc
                                                                ,[
                                                          ] :"snrecnoc"
     ,ecnanif) seirtsudni cificeps htiw ecneirepxe fo noitnem oN"
                                                ,"(.cte ,erachtlaeh
             "(sraey 3) ecneirepxe level-roines trohs ylevitaleR"
                                                                  [
                                                                    {

         ------------------------------------------------------------

                                                :GNIHCTAM BOJ [3 PETS]
                                                                        }
                                                            ] :"sehctam"
                                                                      }
            ,".dtL snoitavonnI IA - tsitneicS ataD daeL" :"boj"
                                                ,59 :"tnecrep_hctam"
       sraey +3 ,tem tnemeriuqer DhP .tif tcefreP" :"gninosaer"
.pihsredael maet nevorp ,detartsnomed esitrepxe PLN/LM ,ecneirepxe
                                ".(aera vivA leT) hctam noitacoL
                                                                ,{
                                                                 }
        ,"sbaL hceT egaugnaL - PLN - tsitneicS hcraeseR" :"boj"
                                                ,09 :"tnecrep_hctam"
hcroTyP ,snoitacilbup PLN htiw DhP .tif tnellecxE" :"gninosaer"
   ni senilepip PLN tliuB .dnuorgkcab hcraeser gnorts ,ecneirepxe
                                                    ".elor suoiverp
                                                                ,{
                                                                 }
     ,".cnI snoituloS duolC - reenignE gninraeL enihcaM" :"boj"
                                                ,58 :"tnecrep_hctam"
      /nohtyP ,(DhP sah) dedeecxe cSM .tif gnortS" :"gninosaer"
setenrebuK/rekcoD htiw ecneirepxe tnemyolped ,esitrepxe wolFrosneT
                                                ".deifitrec SWA ,
                                                                  {
                                                                    [
                                                                    {

         ------------------------------------------------------------

                                        :SNOITADNEMMOCER LANIF [4 PETS]
                                                                        }
etadidnac gnorts yllanoitpecxe na si nehoC haraS .rD" :"tnemssessa"
       nevorp ,esitrepxe cimedaca peed fo noitanibmoc erar a htiw
```


© Dr. Yoram Segal - כל הזכויות שמורות

```
,noinhceT morf DhP reH .ecneirepxe pihsredael dna ,tcapmi yrtsudni
   (noitcuder nruhc %52) stluser ssenisub elbarusaem htiw delpuoc
    rof erih eulav-hgih a sa reh snoitisop ,drocer noitacilbup dna
 lacinhcet htob setartsnomed etadidnac ehT .selor lacinhcet roines
       ,".snoitisop pihsredael rof yrassecen slliks tfos dna htped
                                                                  901
                                                        ] :"spets_txen"  011
  %59) elor tsitneicS ataD daeL rof ssecorp weivretni kcart-tsaF"  111
                                                          ,"(hctam
    %09) evitanretla sa noitisop tsitneicS hcraeseR rof redisnoC"  211
                                                          ,"(hctam
pihsredael dna stcejorp LM tnecer no evid-peed lacinhcet eraperP"  311
                                                       ,"hcaorppa
  "learsI proChceT morf secnerefer dna drocer noitacilbup yfireV"  411
                                                              ,[  511
                                                                  611
                                                     ,39 :"ecnedifnoc"  711
                                                                  811
                                                      ] :"saera_sucof"  911
        ,"yhposolihp tnemeganam maet dna elyts pihsredael eborP"  021
                ,"slaog reerac mret-gnol dna tif erutluc ssessA"  121
    niamod seriuqer elor fi ecneirepxe yrtsudni cificeps ssucsiD"  221
                                                       ,"esitrepxe
              "selor deilppa .sv hcraeser ni tseretni etaulavE"  321
                                                                [  421
                                                                  {  521
                                                                  621
        ============================================================  721
```

gnitsiL 4.8: פלט לדוגמה מריצת מערכת הגיוס

הפלט **??** מדגים את העוצמה של ארכיטקטורת הסוכנים. כל סוכן ביצע את תפקידו בצורה מושלמת:

הסוכן המחלץ לקח טקסט לא מובנה והפך אותו לאובייקט JSON מובנה עם כל השדות הרלוונטיים.

הסוכן המסנן העריך את המועמדת, נתן ציון גבוה (29), וזיהה נקודות חוזקה ספציפיות (רקע אקדמי חזק, ניסיון ניהולי, השפעה עסקית מדידה) לצד חששות קלים (מיעוט ניסיון בתעשיות ספציפיות).

הסוכן המתאים השווה את הפרופיל למשרות הפנויות ודירג את שלושת ההתאמות הטובות ביותר עם אחוזי התאמה גבוהים (59

הסוכן הממליץ סיכם את כל המידע לדוח מקיף, המציע צעדים הבאים קונקרטיים (ראיון מהיר למשרה הראשונה, בדיקת ממליצים), ציון ביטחון גבוה (39

כל התהליך התרחש באופן מקומי, פרטי, וללא עלות, והדגים אוטומציה קוגניטיבית מלאה של תהליך עסקי מורכב.

8.3.4 ניתוח התוצאות: מה למדנו על סוכנים?

התוצאות מלמדות על מספר עקרונות מרכזיים בארכיטקטורת סוכנים:

התמחות משפרת ביצועים: כל סוכן הצליח במשימתו בדיוק משום שהיה ממוקד. סוכן המחלץ לא ניסה גם לנתח וגם להמליץ -- הוא רק חילץ. התוצאה היא דיוק גבוה יותר בכל שלב.

זרימת קונטקסט מצברת ידע: ככל שהתהליך מתקדם, המידע מתעשר. הסוכן הממליץ לא ראה רק את קורות החיים הגולמיים, אלא גם את הניתוח של הסוכן המסנן ואת ההתאמות של הסוכן המתאים. זה מאפשר לו להסיק מסקנות מורכבות יותר.

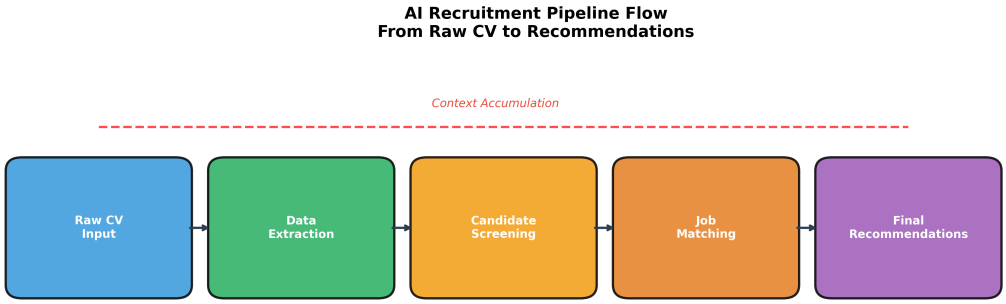
© Dr. Yoram Segal - כל הזכויות שמורות

המתזמר הוא המוח האסטרטגי: ללא תזמור נכון, הסוכנים לא יכלו לעבוד יחד. המתזמר הבטיח שכל סוכן קיבל את המידע הנכון במועד הנכון, ושהתוצאות נצברו בצורה לוגית.

מודולריות מאפשרת גמישות: ניתן בקלות להחליף סוכן אחד (למשל, לשפר את לוגיקת ההתאמה) מבלי לגעת בשאר המערכת. זה הופך את המערכת לגמישה ולקלה לתחזוקה.

פלט מובנה מאפשר אוטומציה נוספת: העובדה שכל סוכן מחזיר JSON מובנה מאפשרת למערכות אחרות (מסדי נתונים, דוחות, ממשקים) לעבד את התוצאות באופן אוטומטי, ללא צורך בהתערבות אנושית.

איור 8.2 ממחיש את זרימת המידע במערכת הגיוס, מקורות החיים הגולמיים ועד לדוח ההמלצות הסופי:



איור 8.2: זרימת מידע במערכת הגיוס המלאכותית: כל שלב מעשיר את הקונטקסט, והמידע זורם מסוכן לסוכן עד להמלצה הסופית.

איור 8.2 מציג בבירור כיצד המידע הופך ממסמך לא מובנה (קורות חיים) לנתונים מובנים (חילוץ), לניתוח איכותי (סינון), להתאמה קונטקסטואלית (משרות), ולבסוף להמלצה אסטרטגית. כל חץ מייצג העברת קונטקסט, וכל שלב מוסיף שכבת מומחיות נוספת.

8.4 אוטונומיה ארגונית: מהגיוס לשירות לקוחות

8.4.1 עלות תפעול אפסית: המשמעות הכלכלית

המערכת שבנינו מדגימה עיקרון כלכלי מהפכני: **עלות תפעול אפסית לאחר ההשקעה הראשונית**. בניגוד למערכות מבוססות ענן שגובות תשלום לכל קריאת API, מערכת הסוכנים שלנו פועלת על גבי Ollama המקומי. ברגע שהחומרה רכושה (מעבד מרכזי טוב וכרטיס גרפי עם זיכרון מספיק), ניתן להריץ את המערכת שוב ושוב, אלפי פעמים, ללא עלות נוספת.

דמיינו חברת גיוס קטנה שמעבדת 100 קורות חיים בחודש. שימוש במערכת מבוססת ענן (כמו GPT-4 של OpenAI) עלול לעלות כ-$0.03 לכל אלף tokens. קורות חיים ממוצעים הם כ-2,000 tokens, ותהליך הניתוח הכולל (חמישה שלבים) יכול לייצר עוד כ-5,000 tokens. סך הכול, כ-7,000 tokens לכל קורות חיים, כלומר כ-$0.21 לכל מועמד. 100 מועמדים בחודש = כ-$21 לחודש, או כ-$250 לשנה. עבור חברה קטנה זה לא נורא, אך עבור ארגון גדול המעבד אלפי קורות חיים בשנה, העלות הופכת למשמעותית.

מערכת מקומית עם Ollama מבטלת עלות זו לחלוטין. ההשקעה הראשונית היא בחומרה (כ-$1000-$3000 למערכת טובה), ולאחר מכן העלות השוטפת היא רק חשמל ותחזוקה בסיסית. התשואה על ההשקעה (ROI) מגיעה מהר מאוד, ולטווח ארוך העלות השולית לכל שימוש היא כמעט אפס.

יתרה מכך, המערכת המקומית מאפשרת **סקלה בלתי מוגבלת במסגרת החומרה הקיימת**. ניתן לעבד עשרות קורות חיים ביום, מאות, ואף אלפים (אם החומרה מספיקה), מבלי לדאוג מעליית עלויות. זוהי חירות כלכלית אמיתית.

8.4.2 פרטיות מלאה: הרקע האתי

מעבר ליתרון הכלכלי, מערכת הסוכנים המקומית מספקת **פרטיות מוחלטת**. קורות חיים מכילים מידע אישי רגיש: שמות, כתובות, מספרי טלפון, תיאורי תפקידים, והיסטוריה תעסוקתית. כאשר מעבדים קורות חיים דרך שירות ענן, המידע הזה עובר דרך שרתים מרוחקים, ונשמר (לפחות זמנית) במאגרי הנתונים של חברות הענן. גם אם החברות מתחייבות לפרטיות, האפשרות הטכנית של דליפה או שימוש לא מורשה קיימת.

מערכת מקומית מבטלת את הסיכון הזה לחלוטין. המידע לא עוזב את המחשב המקומי. אין העברת נתונים דרך האינטרנט, אין שמירה בענן, ואין חשיפה לגורמים חיצוניים. זהו יתרון קריטי במיוחד עבור תעשיות מוסדרות (כמו בנקאות, רפואה, או גיוס ביטחוני), שבהן פרטיות המידע היא דרישה חוקית ואתית.

יתרה מכך, המערכת המקומית מאפשרת **שליטה מלאה על הנתונים**. הארגון בוחר מה נשמר, מה נמחק, וכמה זמן המידע נשמר. אין תלות בספק חיצוני שעשוי לשנות את מדיניות הפרטיות שלו, להעלות מחירים, או אפילו להפסיק את השירות. זוהי ריבונות נתונים אמיתית.

8.4.3 יישומים נוספים: מהגיוס לשירות לקוחות

מערכת הסוכנים שבנינו אינה מוגבלת רק לגיוס. הארכיטקטורה הכללית של תזמור רב-שלבי עם סוכנים מומחים ישימה למגוון רחב של תהליכים עסקיים:

שירות לקוחות אוטומטי: מערכת סוכנים יכולה לקבל פניית לקוח, לזהות את סוג הבעיה (סוכן מסווג), לאחזר מידע רלוונטי ממסמכי מדיניות (סוכן RAG), לייצר תשובה (סוכן מגיב), ולהעביר למחלקה המתאימה אם נדרש (סוכן מתזמר). כל התהליך אוטומטי, מהיר, ופרטי.

ניתוח מסמכים משפטיים: מערכת יכולה לקבל חוזה, לחלץ סעיפים מרכזיים (סוכן מחלץ), לזהות סיכונים פוטנציאליים (סוכן מנתח סיכונים), להשוות לחוזים דומים (סוכן משווה), ולייצר דוח המלצות (סוכן ממליץ). זה מאיץ תהליכי בדיקת חוזים ומפחית טעויות.

ניהול פרויקטים: מערכת סוכנים יכולה לקבל תיאור פרויקט, לפרק אותו למשימות (סוכן מפרק), לאמוד זמנים (סוכן מעריך), להקצות משימות לחברי צוות (סוכן מקצה), ולעקוב אחר ביצוע (סוכן מנטר). התהליך הופך להיות מונחה-נתונים ואוטומטי.

מחקר ופיתוח: מערכת יכולה לקבל תיאור בעיה טכנית, לחפש פתרונות במאגרי מידע (סוכן חוקר), לנתח יתרונות וחסרונות (סוכן משווה), לייצר קוד אב-טיפוס (סוכן מפתח), ולהמליץ על הגישה הטובה ביותר (סוכן ממליץ).

כל אחד מהיישומים הללו חולק את אותו העיקרון: פירוק תהליך מורכב למשימות מיוחדות, כל אחת מבוצעת על ידי סוכן ייעודי, בתזמור מרכזי.

8.4.4 עתיד האוטומציה המקומית: מהחזון למציאות

מערכות סוכנים מקומיות מייצגות את הגבול הבא של אוטומציה ארגונית. בעשור הקרוב, צפוי שארגונים רבים יאמצו ארכיטקטורות דומות, לא רק בשל היתרונות הכלכליים והפרטיות, אלא גם בגלל **הגמישות והשליטה** שהן מספקות. ארגון שמפעיל מערכת סוכנים מקומית יכול להתאים אותה במדויק לתהליכים שלו, לשנות סוכנים לפי הצורך, ולשפר ללא הרף ללא תלות בספקים חיצוניים.

הדמוקרטיזציה של AI דרך כלים כמו Ollama ומסגרות עבודה כמו Swarm הופכת את האוטומציה הקוגניטיבית מטכנולוגיה שמורה לענקיות הטכנולוגיה לכלי נגיש לכל ארגון, קטן כגדול. חברות קטנות וסטארטאפים יכולות כעת לבנות מערכות מתוחכמות שהיו שמורות בעבר רק לתאגידים עם תקציבי IT ענקיים.

© Dr. Yoram Segal - כל הזכויות שמורות

יתרה מכך, מערכות אלה יכולות להשתפר עם הזמן דרך **למידה מנתוני השימוש**. כל ריצה של המערכת מייצרת נתונים -- מה עבד, מה לא עבד, איפה היו שגיאות. ניתן להשתמש בנתונים אלה כדי לכוון עדין (Fine-tune) את המודלים הבסיסיים או לשפר את הוראות הסוכנים, ובכך ליצור מעגל משוב חיובי של שיפור מתמיד.

העתיד הוא של **אוטונומיה קוגניטיבית מבוזרת** -- ארגונים שיפעילו צוותי סוכנים מקומיים, פרטיים, וממוקדים, המתקשרים זה עם זה בצורה יעילה ומתוזמרת. זוהי לא עוד חזון רחוק, אלא מציאות שכבר ניתן לממש היום, בזכות הטכנולוגיות שבחנו בספר זה.

8.5 סיכום: מהיחיד לקהילה

הפרק הזה הדגים את המעבר הפילוסופי והטכנולוגי מבינה מלאכותית יחידנית לבינה מלאכותית קהילתית. במקום מודל שפה אחד שמנסה לעשות הכול, בנינו רשת של סוכנים מומחים, כל אחד עם תפקיד ברור, הפועלים יחד תחת תזמור מרכזי. התוצאה היא מערכת חזקה יותר, גמישה יותר, ושקופה יותר.

מקרה המבחן של סוכנות הגיוס המלאכותית הוכיח שתהליכים עסקיים מורכבים יכולים להתבצע באופן אוטונומי, פרטי, וללא עלות תפעולית, על גבי תשתית מקומית. זוהי דוגמה ממשית ליכולות של Ollama בשילוב עם ארכיטקטורת סוכנים.

ככל שהטכנולוגיה תתקדם, צפוי שמערכות הסוכנים תהפוכנה מתוחכמות יותר, עם יכולות של תיאום דינמי, למידה מנתוני שימוש, ואפילו יצירה עצמית של סוכנים חדשים לפי הצורך. אך העיקרון המרכזי יישאר זהה: **דלגציה קוגניטיבית**, **התמחות**, **ותזמור** -- אותם עקרונות שהובילו לצמיחת הציביליזציה האנושית, מיושמים כעת בעולם הקוגניציה המלאכותית.

זהו סיום ההרצאה על מערכות סוכנים מתקדמות. בפרק הבא, נחזור לסכם את כל המסע -- מOllama כבסיס, דרך RAG כמנגנון אמת, ועד למערכות סוכנים כשכבת אוטומציה -- ונבחן את המשמעות הפילוסופית והמעשית של ריבונות דיגיטלית במאה ה21.

© Dr. Yoram Segal - כל הזכויות שמורות